

555.2

Network and Endpoint Analysis

Licensed To: Arvind Pariti <paritiarvind@gmail_com> April 13, 2026



PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE (DEFINED BELOW) ASSOCIATED WITH THE SANS INSTITUTE COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND THE ESCAL INSTITUTE OF ADVANCED TECHNOLOGIES, INC. /DBA SANS INSTITUTE ("SANS INSTITUTE") FOR THE COURSEWARE. BY ACCESSING THE COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this CLA. Courseware means all printed materials, including course books and lab workbooks, slides or notes, as well as any digital or other media, audio and video recordings, virtual machines, software, technology, or data sets distributed by SANS Institute to User for use in the SANS Institute course associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and User and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCESSING THE COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. USER FURTHER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND), SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree to the terms of this CLA, User should not access the Courseware. User may return the Courseware to SANS Institute for a refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium, whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. User may not sell, rent, lease, trade, share, or otherwise transfer the Courseware in any way, shape, or form to any person or entity without the express written consent of SANS Institute. Additionally, User may not upload, submit, or otherwise transmit Courseware to any artificial intelligence system, platform, or service for any purpose, regardless of whether the intended use is commercial, educational, or personal, without the express written consent of SANS Institute. User agrees that the failure to abide by this provision would cause irreparable harm to SANS Institute that is impossible to quantify. User therefore agrees to a base liquidated damages amount of \$5000.00 USD per item of Courseware infringed upon or fraction thereof. In addition, the base liquidated damages amount shall be doubled for any Courseware less than a year old as a reasonable estimation of the anticipated or actual harm caused by User's breach of the CLA. Both parties acknowledge and agree that the stipulated amount of liquidated damages is not intended as a penalty, but as a reasonable estimate of damages suffered by SANS Institute due to User's breach of the CLA.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. A written amendment or addendum to this CLA that is executed by SANS Institute and User may accompany this Courseware.

SANS Institute may suspend and/or terminate User's access to and require immediate return of any Courseware in connection with any (i) material breaches or material violation of this CLA or general terms and conditions of use agreed to by User, (ii) technical or security issues or problems caused by User that materially impact the business operations of SANS Institute or other SANS Institute customers, or (iii) requests by law enforcement or government agencies.

SANS Institute acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

The Apple® logo and any names of Apple products displayed or discussed in this book are registered trademarks of Apple, Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

VMware Workstation Pro®, VMWare Workstation Player®, VMWare Fusion®, and VMware Fusion Pro® are registered trademarks of VMware, Inc. Used with permission.

Governing Law: This CLA shall be governed by the laws of the State of Maryland, USA.

Courseware licensed to User under this CLA may be subject to export laws and regulations of the United States of America and other jurisdictions. User warrants he or she is not listed (i) on any sanction programs list maintained by the U.S. Office of Foreign Assets Control within the U.S. Treasury Department ("OFAC"), or (ii) denied party list maintained by the U.S. Bureau of Industry and Security within the U.S. Department of Commerce ("BIS"). User agrees to not allow access to any Courseware to any person or entity in a U.S. embargoed country or in violation of a U.S. export control law or regulation. User agrees to cooperate with SANS Institute as necessary for SANS Institute to comply with export requirements and recordkeeping required by OFAC, BIS or other governmental agency.

All reference links are operational in the browser-based delivery of the electronic workbook.

Licensed To: Arvind Pariti <paritiarvind@gmail_com> April 13, 2026 SEC555_2_K02_01

Network and Endpoint Analysis

DETECTION ENGINEERING AND SIEM ANALYTICS

© 2025 SANS INSTITUTE | ALL RIGHTS RESERVED

K02_01

This area is intentionally left blank.

Table of Contents

SMTP	3
DNS	17
Lab 2.1: Investigating DNS Logs	41
HTTP/HTTPS	42
Lab 2.2: Investigating HTTP Logs	69
Windows Logs	70
Lab 2.3: Investigating Windows Logs	94
Linux Logs	95
Host Firewalls and Login Events	118
Lab 2.4: Using Auditd	141
Contacts and Resources	142

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- SMTP
- DNS
 - ▢ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ▢ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ▢ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ▢ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will gain an understanding of the logging and monitoring of SMTP communications.

This area is intentionally left blank.

Common Service Examples

What network services can be easily exploited?

What do you mostly use in a corporate environment?

Examples of common network services include:

- DNS
- HTTP and HTTPS
- SMB
- SMTP
- SSH

We will be focusing heavily on SMTP, DNS, HTTP, and HTTPS

While many services exist, the ones most commonly seen in today's networks are SMTP, DNS, HTTP, and HTTPS. These are used by end users to browse the internet and receive email. As a result, we will be focusing heavily on these services since they are common attack vectors. Do note, however, that some of the techniques that we will cover are applicable across the board.

For example, when discussing frequency analysis, that should be a concept you apply everywhere in your environment.

Consider this: Did you previously see 20% encrypted traffic, and now you see 75%?

Is that normal, or perhaps an attacker is exfiltrating data over an encrypted connection?

Phishing Discovery via SMTP

From: Sandy Miller<sandy@labmeinc.com>
To: John Dones <john@labmeinc.com>
Subject: Client Call Today

Are they the same domain?

This search detects phishing attempts using SMTP

```
tags:smtp AND domain:labmeinc.com~  
-domain:labmeinc.com
```

Fuzzy search character (not supported on all SIEM)

It finds SMTP records similar to a known domain *but* not an exact match.

- **Should return nothing when you run it.**

Let's look at a simple spear phishing example.

The adversary did some OSINT research and discovered that Sandy Miller works with John Dones. Since John is the target, the attacker sent an email from sandy@**1**abmeinc.com (to make it resemble Sandy's real email address, which is sandy@labmeinc.com). Because there is only a single character difference, it is possible John will not realize that the email is not really from Sandy. If you look closely, the actual domain of labmeinc.com contains a lowercase letter **l**, which the attacker replaced with the number one **1**.

Assuming that SMTP logs are being collected, discovering phishing attempts that look similar to an organization's real email domain is fairly simple. In this example, the email domain in the SMTP log is fuzzy-searched for anything similar to **labmeinc.com** that is not an exact match to labmeinc.com. As a result, **1abmeinc.com** would be found. Think of this like when you are searching in your favorite search engine for restaurants. But you make a typo and enter "**restarants**". A fuzzy algorithm is able to match that you are most likely looking for **restaurants**, so it returns results for that term instead. An example of a deliberate use would be in a field called *operating systems*, where you know you have variances of Windows. You could use something like `os.name~windows` to allow searches to return Windows 11, Windows Server 2025, and so on.

A fuzzy search looks for things similar to whatever you are searching for by searching for values that are similar to a search string but off by a number of characters. Fuzzy searching works best with longer search strings because the longer the strings, the less likelihood for exact matches. If you had a small domain such as sec555.com, it might still work, but it is possible it may create false positives because you can get numerous items matching it. Note that not all SIEM tools support fuzzy searching.

Phishing Discovery via HTTP

From: Sandy Miller<sandy@labmeinc.com>
To: John Doe <john@labmeinc.com>
Email Body: Check out this **client's site** before our call
Links To: http://afecrej6h7cn5sdfhvjpg9evmj.com

Time ▾	frequency_score	query
March 14th 2025, 10:12:13:332	13.404	www.google.com
March 14th 2025, 10:12:13:332	5.778	afecrej6h7cn5sdfhvjpg9evmj.com

"But frequency_score is not a field in my logs." Not yet, it isn't. 😊

- Augmenting standard logs adds incredible power

Again, the same technique could be applied against HTTP, DNS, SMTP, and other event sources.

Continuing the spear phishing example, let us assume that John opened the email and inside was a link. Upon clicking this link, the browser navigates to http://afecrej6h7cn5sdfhvjpg9evmj.com. The domain would show up in DNS logs, and it would show up in HTTP logs. But how is this helpful?

Because of security devices, attackers often have to use randomness to evade filters. In this example, the domain afecrej6h7cn5sdfhvjpg9evmj.com is clearly random. Let's look at the URL in question and assess how ordinary it may be. Looking at the above output, **afecrej6h7cn5sdfhvjpg9evmj.com** has a **frequency_score of 5.778**. A normal record for www.google.com has a score of 13.404. This massive gap in score indicates that the domain afecrej6h7cn5sdfhvjpg9evmj.com is most likely random. The tool used to generate this will be discussed in more detail later.

This approach can be applied against HTTP, HTTPS, or DNS logs. In essence, you can harvest the URL from any related log source and just apply the pre-mentioned frequency check.

Now, I know you are thinking, "Where did the **frequency_score** field come from?" In this example, the logs have been augmented by performing a randomness check against the domain name and saving it into a field called frequency_score. This will be covered shortly. The point is that a common service log can be augmented and made defensible regarding active detection.

Note: The above is a fictitious URL, used solely for demonstration purposes.

Phishing Discovery via DNS

- The same technique can be applied to DNS:

```
tags:dns AND domain:labmeinc.com~  
-domain:labmeinc.com
```

Lucene or Elasticsearch
Query DSL (Domain Specific
Language)

- That search yields a match:

```
Mar 14th, 2025 10:12:13.332 labmeinc.com
```

- Should return nothing when you run it.**

Is this hit a true or false
positive?

To demonstrate that a given technique can apply to multiple logs or scenarios, take the same inbound phishing email. This time, the example uses DNS logs.

```
tags:dns AND domain:labmeinc.com~ -domain:labmeinc.com
```

The above query uses fuzzy searching to look at DNS logs for a domain similar to **labmeinc.com** but not actually labmeinc.com.

However, consider this: Does the technique apply as successfully to DNS logs as it does to SMTP logs?

- What would happen if you were looking at HTTP logs and the domain matched the corporate website, and someone tried to access the site but had a typo in their browser?

A typo would end up creating a false positive.

- What would happen if an external user tried to send you an email but had a typo?

The email would never arrive in the first place, which means this event would never make it to your logs, so no false positive would be created.

Applying this technique for detecting phishing attempts could be used against DNS, HTTP, and SMTP, but would likely create a lot of false positives for everything except SMTP. Again, think, understand, and apply.

Common SMTP Log Fields

HELO	From	To	Subject
Is Webmail	First Received	Reply Codes	Mail User Agent (MUA)
Source IP	Destination IP	File attachment name	File attachment size

Of all the different types of logs, SMTP is probably one of the sources most tailored to an organization. This is because there are so many different email servers, email proxies, and other email security devices available. Unfortunately, many organizations decide to collect from all sources, which makes them end up with lots of duplicate data. If you want to collect logs from your email server and a spam appliance, you should have a reason for collecting from both. It's possible that each has fields or values only available in one, but that is likely not the case.

This page shows some of the more common SMTP fields that a SIEM should be able to collect. Do note that names may differ across different products.

- **HELO:** Client sends this to the SMTP server to initiate communication.
- **From:** Denotes the sender email address.
- **To:** Denotes the recipient email address.
- **Subject:** This is the email subject.
- **Is Webmail:** States if the email was sent from a webmail interface.
- **First received:** Records the first (earliest) server hop the email goes through. Note that this is at the bottom of the **Received:** statements.
- **Reply codes:** SMTP reply codes.¹
- **MUA:** The client email application used to send emails (e.g., Microsoft Outlook or Mozilla Thunderbird).
- **Source IP:** Originating email server IP address.
- **Destination IP:** Destination email server IP address.
- **File attachment name and size:** Attached file name and size (if there is a file attached).

It's worth considering enriching your logs to make **display names** (e.g., Jane Doe) and **geoIP information** available, which can greatly help analysis.

1. <https://sec555.com/epc71>

Email Monitoring Methods

How can a SIEM detect what other mature devices might not?

Inbound monitoring possibilities

- Fuzzy searching
- Monitor for spikes of email volume from outside sources
- Look for external use of key employee names

Outbound monitoring possibilities

- Monitoring of authorized source devices
- Monitoring of SMTP user agents

With so much technology pre-built around SMTP, how can a SIEM possibly help?

If you stop to think about it, most of the existing technologies are designed to be prevention devices primarily. On top of that, they must be designed in a way that works across multiple environments. Thus, they are not tailored to your environment.

Some of the strongest capabilities can be found in the fact that analysts can tune systems according to their environment. A few methods to optimize your SIEM in relation to email monitoring are the following:

1. Inbound monitoring

- Using fuzzy searching to search for company look-alike domains, potentially used for phishing.
- Monitoring for spikes in email volume from outside sources, which can be indicative of a phishing campaign.
- Looking out for any key employee names (e.g., like Jane Doe - the CFO) used to send emails originating from a non-corporate email address.

2. Outbound monitoring

- Monitoring of authorized source devices, which are the ones you're expecting emails to be sent from.
- Monitoring of SMTP user agents for potentially suspicious ones being used.
- Monitor the volume of outbound email and identify if dramatic changes exist, which may be indicative of SMTP used for data exfiltration or command and control.

Fuzzy Phishing and Look-Alike Domains

Take legitimate company domains and look for variants

- Extremely effective against phishing domains and great for targeted attacks
- Best used in combination with email alerts or scripts

Look-alike domains

- Unicode domain names
 - **Homograph attack** uses valid Unicode that matches expected ASCII (e.g., microsoft.com, using a Cyrillic i instead of the Latin i)
 - Problem is that domain names support Unicode characters
- **Character replacement** (such as 1 for l—remember labmeinc earlier?)
- **Character transposition** (such as flipping two letters)
- Tools like dnstwist can generate domain permutations

As previously mentioned, fuzzy phishing uses a list of internal domains to look for phishing attempts against it. You can compile an allow list of internal domains that you populate, and then you tell the SIEM to do a fuzzy search and match any look-alike domains it uncovers in the logs. Those should be raised as alerts for your review. This is something that will require regular updates, but the only thing that has a considerable overhead is compiling and setting up the list for the first time. Maintaining it should be more straightforward.

Attackers using look-alike domains can use tools like **dnstwister**¹ to generate domains of interest. A search for avengersinc.com generated 344 look-alike domains available for the attacker.

Another tool, called **dnstwist**,² can also be used to identify look-alike domains and denotes the technique used each time:

- **Character omission** (removing one or more characters from the URL).
- **Character insertion** (inserting one or more characters at the URL).
- **Character replacement** (combination of insertion and omission where an existing character gets replaced by others).
- **Character transposition** (character positions get swapped).

Lastly, a great commercial solution is **DomainTools PhishEye**.³ It allows you not only to search for look-alike domains but also set alerts and associated notifications, matching your criteria.

1. <https://sec555.com/8zy96>
2. <https://sec555.com/jkpv6>
3. <https://sec555.com/ugm95>

Influx of Emails



While many phishing attacks may be targeted spear phishing attempts, it is common for a phishing email to be sent across a very large portion of company users. This method of attack has a higher chance of success even with a poorly designed phishing message. Instead of crafting a targeted email against a handful of people, a general email can be sent to thousands or tens of thousands, and it only takes one person to click a link or open an attachment for the attacker to win.

Because of this, it is possible to monitor for large phishing campaigns by looking for bulk emails by source. This technique is guaranteed to have false positives. Marketing or partner services are likely to generate bulk emails to your organization. Therefore, **the only way this method works is by applying it with an allow list approach**. Any authorized system that hits this list gets added to an allow list, and any source that is not authorized gets investigated.

Many detection techniques are inefficient because raw logs do not provide adequate information to filter out noise. Log enrichment adds context so that organizations can detect things they normally would not.

For example, if an organization appends geo information or whois data about an email sender in SMTP logs, then filtering out known email companies is possible. In the above example, SurveyMonkey is provided as a reference, since it is one of the most commonly used survey tools for a lot of companies. An SMTP log would normally contain the IP addresses of the sender, and that can be used to pull in information that the entity (based on its whois record or autonomous system number), being **SurveyMonkey**. Therefore, an organization can use enrichment context to filter out mass marketing firms and partners. This can lead to a significant noise reduction.

Monitor External Use of Employee Names

Many times, spear phishing impersonates executive staff to pressure a response.

- Example: Fake CFO sends an email to accounts payable to transfer money.

It's possible to use a list of names to look for phishing attempts.

- Look for display names of key staff and tag, alert, or trigger response action.

Because in most phishing attempts, attackers act as key executives (which is called *whaling*), it makes sense to monitor the display names for incoming emails. By monitoring for incoming emails with a display name of the CEO or the CFO (or other key personnel within the company, usually the C-level executives), but not actually coming from an email address associated with the company domain, this could be detected.

Would you ever accept an order to send \$100,000 to a bank account when that's originating from the CFO's Gmail account? Would you not just call them and ask if they legitimately intended to perform this transfer?

To materialize this technique, take the names of your executive team and start monitoring for inbound emails that use their names, originating from an email address that isn't their official company email account. That should raise an alert for further investigation.

Outbound Email Sources

Normally, email should come from authorized servers or subnets

- Exceptions may be scripts that send emails from hosts.

SIEM provides easy reporting and filtering

- Filter out email addresses used by scripts.
- Filter out authorized sources.
- Investigate outbound emails from unauthorized IPs.

This can be applied using flow data or connection logs

Within a given organization, there should be a finite list of authorized email sources. Normally, when a user's desktop is used to send an email to someone, the email is actually generated and sent from the desktop's email client to the company email server. Therefore, it is odd if SMTP traffic starts to be present where it should not be.

This is another allow list approach for looking for email sources where they are not expected. Because most organizations have a limited amount of email servers and new ones are not regularly stood up, this is fairly low maintenance, outside the initial setup.

Just remember to do your homework and exclude items such as:

- Scripts used to send emails from certain machines performing routine maintenance and sending reports (or any similar reason). For example, Python can be used to write a script sending an email by connecting to the recipient's SMTP server directly, without using the company mail server.
- Authorized email sources that are used in your environment.

Anything remaining can then be investigated to identify if this is expected traffic or not. Remember that at the beginning, you should expect a degree of false positives as you perform your initial walkthrough and eliminate items, but as that first wave passes, you should be in a good position to monitor your environment more efficiently.

SMTP User Agents and Clipping Level

Logs contain an SMTP Mail User Agent (MUA) field

- Specifies what client application claims to send an email

Can help filter out authorized outbound emails

- Great in combination with IP address filters
- Can also help find the source of infections

What are the maximum emails sent per system, per hour?

What if that were to increase exponentially?

- Is it a legitimate increase?
- Maybe an operational issue?
- Could it be SMTP command and control?
- Perhaps, phishing attacks using an authorized system?
- **Reminder:** Malware can often attack a company through another victim company.

An SMTP user agent describes the application being used to send email, like the below:

- Gmail/1.0,
- Yahoo Mail/1.0
- Apple Mail/15.0 (iPhone; CPU iPhone OS 15_0 like Mac OS X)
- Microsoft Outlook/16.0 (MS Windows:10; x64).

The user agent can be used to help filter out authorized outbound emails or can be used as a separate internal allow list approach.

User agents can also be helpful if an authorized system is compromised and starts sending emails via a different user agent. For instance, if you had an authorized web server that normally sent emails using Outlook but then started sending emails using Python, something is wrong.

The SMTP Mail User Agent is not required. Also, the contents can be set to anything. This means malware can mask as a legitimate email client. But that shouldn't stop you from still searching for it being suspicious to begin with.

Knowing normal makes abnormal stand out.

The hard part is knowing what is normal. Most organizations have no clue how many emails they send per hour, let alone how many per authorized system. Finding this amount and establishing a suitable clipping level allows for finding abnormal spikes or unauthorized use. You need to be careful to account for outliers or periods of overuse, but establishing an initial baseline of what's expected goes a very long way.

Summary

Identifying the sources of emails in your environment and associated patterns of expected volume can be really crucial to finding abnormal behaviors.

Use tools to uncover potentially typo-squatted domains that attackers may consider using against you. Consider proactively registering those domains before they use them.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- SMTP
- DNS
 - ☐ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ☐ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ☐ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ☐ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will gain an understanding of DNS, related attacks, and the importance of logging and monitoring.

This area is intentionally left blank.

Common DNS Record Types

Record Type	Description
A	Domain name to IPv4 address
AAAA	Domain name to IPv6 address
CNAME	Alias to A record (e.g., www.sans.org to sans.org)
MX	Maps domain to email server
NS	Designate the domain's authoritative DNS server
PTR	IP address to domain name (for reverse lookups)
SOA	Administrative domain information, e.g., refresh interval for secondary to pull records from primary DNS server, interval to retry pull if it fails etc.)
TXT	Sends text over DNS (special purposes, i.e. email authentication, DKIM, or sender approval, SPF)

It is important to know the main DNS record types available and how they are used:

- **A** = Domain name to IPv4 address.
- **AAAA** = Domain name to IPv6 address.
- **CNAME** = Alias to A record. Let's say you have sans.org and www.sans.org. Instead of creating two separate A records, one can be created for sans.org and a CNAME record for www.sans.org pointing to sans.org. If the IP address changes, you only have to change it for sans.org as www.sans.org is pointed to it automatically.
- **MX** = Maps a domain to an email server.
- **NS** = Authoritative name server records for the domain.
- **PTR** = Reverse lookup, allowing to resolve an IP address to a domain name.
- **SOA** = Administrative information about a domain (serial number of DNS zone file, refresh interval for primary to secondary server sync, retry interval to try again if a sync fails, expiration time of records from secondary server, TTL of caching domain information from a resolver agent).
- **TXT** = Sends text over DNS (special purposes such as email authentication, DKIM authentication, or sender approval, SPF).

Different techniques apply to different records. For example, when dealing with phishing attacks, MX records may be incredibly useful as they are involved with email routing. Looking for DNS tunneling? This is most likely being done by abusing TXT records. More to come. :)

Windows DNS Log Settings

The image shows two side-by-side screenshots of the Windows DNS server properties dialog box, specifically the 'Event Logging' and 'Debug Logging' tabs. The left screenshot shows the 'Event Logging' tab with 'All events' selected. The right screenshot shows the 'Debug Logging' tab with various options checked, including 'Log packets for debugging', 'Outgoing' and 'Incoming' packet directions, 'Queries/Transfers' and 'Updates' packet contents, and 'Request' packet type. Numbered callouts (1-6) highlight specific settings and actions. Blue callout boxes provide additional context for these settings.

Default setting in the DNS server Event Logging Tab

VS.

Select Log packets for debugging and all other options become enabled

Unselect Request to eliminate duplicates

Specify a location to store the logs

Set desired log file size

Collecting logs from Windows DNS can be a little confusing. If you open the properties on the DNS server, you are confronted with two tabs:

- **Event logging:** If you set event logging to all events in the Event Logging tab, you will receive logs of any DNS Server events, not client DNS query logging. These are logged in the traditional Windows Event format.
- **Debug logging:** To obtain query logs coming from clients, you have to enable debug logging on the debug logging tab.
 - Note that if you enable the Packet Type of **Request**, you will see all DNS queries before a response is generated. This creates unnecessary events and makes it appear that you have duplicate query records.
 - Instead, **keep Request unchecked and check Response**. This will log queries along with the response. Even if the query fails, this will still generate a log with a response of NXDOMAIN.

Remember to define a file path and filename for your output log and to modify the size, if desired. The above configuration can give you powerful visibility into your environment.

Here's a list of what DNS logs can give you:

- ✓ Low to moderate performance impact.
- ✓ Log at informational, warning, and error levels.
- ✓ They are enabled by default.
- ✓ Mostly used for troubleshooting, i.e., did a zone transfer fail, or did the DNS server experience any problems?
- ✓ Mostly used for troubleshooting operational or performance-related issues.

Windows DNS Analytical Log

Level	Date and Time	Source	Event ID	Task Category
Information	11/02/2025 04:25:21	DNS-Server	256	LOOK_UP
Information	11/02/2025 04:25:21	DNS-Server	256	LOOK_UP

Event 256, DNS-Server

General Details

QUERY_RECEIVED: TCP=0; InterfaceP=192.168.1.10; Source=192.168.1.20; RD=1; QNAME=sec555.com; QTYPE=65; XID=24897; Port=62321; Flags=256; PacketData=0x6141010000010000000000000067365633535303 [AdditionalInfo = VirtualizationInstanceOptionValue: ; GUID={685ABECF-E14F-44B0-A58C-C2151CAA2E50}]

Log Name: Microsoft-Windows-DNS-Server/Analytical

Source: DNS-Server

Event ID: 256

Level: Information

User: N/A

OpCode: Info

Logged: 11/02/2025 04:25:21

Task Category: LOOK_UP

Keywords: (1)

Computer: SEC555-SRV.domain.avengers

- Requires log agent with the capability to read ETL files or just export as .evtx and read

Actions

Analytical

Open Saved Log...

Create Custom View...

Import Custom View...

Clear Log...

Save All Events As...

File name:

Save as type: Event Files (*.evtx)

Save

Starting in Server 2012 R2 and later, Microsoft has developed a new method for DNS logging in Windows, called the **DNS Analytical Log**. This is a high-performance log system specific to DNS. This log is stored in an ETL (event trace log) file, so you must have a log agent that is capable of reading it to collect that. PerfView can read ETL files.¹ NXLog also supports .ETL file reading.² Note that Windows Server 2012 went EOL in 2023, so it's just mentioned here for awareness of when this feature was made available. DNS Analytical logs have very detailed information as they support debug level tracing (capable of assisting in obtaining packet-level information), but that also comes with potential performance hits.

Since this log isn't enabled by default, to enable this, open Windows Event Viewer and browse to Application and Service Logs > Microsoft > Windows > DNS-Server. Right-click on DNS-Server and click on View > Show Analytical and Debug Logs. As soon as it appears on the list, right-click on Analytical and select Enable Log.³ Remember to choose your settings for logging retention by right-clicking again on Analytical > Properties. The general tab has numerous retention options and allows you to also define the maximum event log size.

There's a great five-part blog in Microsoft's TechNet about the analytical logs that you should definitely read.

1. <https://sec555.com/dnezy>
2. <https://sec555.com/4o6lf>
3. <https://sec555.com/69geu>

References:

- Analytical Logs Part 1: <https://sec555.com/q3p6l>
- Analytical Logs Part 2: <https://sec555.com/vg7mq>
- Analytical Logs Part 3: <https://sec555.com/cx0me>

- Analytical Logs Part 4: <https://sec555.com/vuhrd>
- Analytical Logs Part 5: <https://sec555.com/jnrti>

© SANS Institute 2025

Licensed To: Arvind Pariti <paritiarvind@gmail_com> April 13, 2026

DNS Response Fields

Native

- Query Name
- Query Type
- TTL
- Data Length
- Type
- RDATA
- Response Codes

Value Add

- Frequency Score
- Parent Domain
- Child Domain
- Domain Length
- Domain Age
- Geo-Information

At a high level, DNS responses come with a few key fields, like:

- **Query name:** The domain name being queried.
- **Query type:** DNS record requested.
- **TTL:** Duration of time to cache the response.
- **Data length:** Size of resource data.
- **Type:** Record type (commonly NS).
- **RDATA:** The authoritative nameserver.
- **DNS response codes:** 0 (no error), others (indicating some error).

To add more value and weaponize your logs, you should consider adding the following fields:

- **Frequency score:** Entropy test for how random a field is (typical example is a URL's frequency).
- **Parent domain:** High-level domain name (example: **google.com** in mail.google.com).
- **Child domain:** Domains below parent domain (example: **mail** in mail.google.com).
- **Domain length:** Longer lengths may be indicative of suspicious domains.
- **Domain age:** New domains are often used by attackers.
- **Geo-information:** Location information can aid in identifying suspicious domains.

Identify Frequency: `freq_server.py` and `freq.py`

Previously hinted about frequency score

- Randomness calculation can be run and saved against any field

Mark Baggett, SANS instructor and author of SEC573, developed `freq.py` and `freq_server.py`

- **`freq.py`** is useful for manually checking for randomness
- **`freq_server.py`** allows for high-performance querying, is API-based, and was built with SIEM solutions in mind

A field called **frequency_score** was introduced earlier. This field contains a numeric float value that is used to express how random something is. This type of calculation is easy when using Mark Baggett's `freq.py` and `freq_server.py` scripts.¹ He gave a great presentation on them that is worth watching, to get acquainted with the tools in more detail.²

How this works is by mathematically calculating the frequency of occurrence of characters. For example, if the first character is the letter A, how likely is it that the second character that follows is going to be a T, and so forth? In the English dictionary, certain letters tend to follow other letters. If something is truly random, then this pattern will be broken.

The beauty of these scripts is that they allow you to create your own frequency tables. For example, you could have a frequency table based on the English dictionary as well as a frequency table of domain names generated by the Cisco Umbrella top 1 million sites. This allows different applications for different use cases.

- **`freq.py`** is useful for one-off manual checks or bulk checks against a text file.
- **`freq_server.py`** is designed as an API that can handle large-scale requests and is great for log augmentation.

1. <https://sec555.com/shn28>

2. <https://sec555.com/ukyfd>

freq.py: Frequency Score

- freq.py or freq_server.py can be used to identify resource entropy

```
python3 ./freq.py -m iuuabnbvjyew.com cisco.freq  
(10.2961, 16.4555)
```

Average
probability

Word
probability

URL to check

Frequency table (e.g., top 1
million Umbrella sites)

- The lower the score, the higher the chance of randomness.
 - If you are not allowed to put freq_server.py in place, you can use freq.py manually or scripted.
 - Example: Dump unique domains from previous day into dns.txt and compare them against your desired frequency table

```
freq.py -b dns.txt dns.freq  
[+] zln2vk7y4gxxbo11.com 3.03510406606  
[+] facebook.com 15.3311426794
```

Both **freq.py** and **freq_server.py** are used to generate an entropy score indicating randomness. With these scripts, the lower the score, the higher the likelihood something is random. When you run a domain against a frequency table, the result will be two numbers:

- **First number: Average probability:** Probability of pairs. Assume the word **queen**. The tool assesses the probability of q followed by u, u followed by e, e followed by e, and lastly e followed by n. It then adds them up and finds the average probability. **Lower than 5 usually indicates something to investigate further.**
- **Second number: Word probability (improved version of the average probability):** Probability of a whole word (e.g., hacker). **Lower than 4 usually indicates something to investigate further.**

In this example, **cisco.freq** and **dns.freq** are frequency tables used for calculating entropy. Mark's tool supports building custom frequency tables. For example, the English dictionary or random books you like can be fed through freq.py to generate a frequency table specific to words in the English dictionary. Then, when a string is used to calculate entropy against this, the frequency score will be based on the rate of occurrence of characters compared to the English dictionary. This makes freq.py powerful, as custom tables can be generated for specific use cases.

How about if you generate a list of all your system files, and when the attacker adds a new file with a weird name (meaning one you're not seeing frequently), then your SIEM tool raises an alert?

Note that your SIEM solution may not allow external API calls to freq_server.py, or you may not be authorized to implement freq_server.py. In these cases, it may still be possible to apply the technique in an out-of-band method using freq.py. For example, if you dumped a list of domains to a text file (manually or scripted), you could then run the text file through freq.py using the -b (bulk measure) switch. In this example, that yielded a suspicious URL of **zln2vk7y4gxxbo11.com**.

Newly Created Domain Access

Users occasionally browse new sites

- But really are creatures of habit

Monitoring newly created domain requests can be a powerful way to detect malware

- Can be done continuously or once every 24 hours
- Helpful to combine with top 1 million domain list, reputation scores, or known good list

Results often fit within a one-page report

If you think about it, network-based attacks occur either directly through an IP address or by using DNS names. Because of IP deny lists being commonplace, most of today's malware uses DNS names. This allows for another method of taking an opponent's strength and using it against them.

If attacks are going to be using DNS domains, then it is possible to catch them simply by inspecting newly created domains being used. Think about that. Let's look at a well-established domain's creation date, like `sans.org`:

```
whois sans.org | grep Creation  
Creation Date: 1995-08-04T04:00:00Z
```

As you can see, the creation date of that domain was several years ago. However, when attackers use domains for nefarious purposes, a look-alike domain will yield something just registered a few hours or days ago. Each time that is the case, security analysts should be aware that this may be worthy of further investigation. Of course, there can be a perfectly legitimate newly created domain present, but when you are investigating, you are expected to take multiple artifacts into consideration and make a decision. This is just one more item to consider when performing analysis. For example, would you expect 200 users to browse to a newly created domain in the same hour or day of it being created? Probably not.

This assessment can be done continuously using a SIEM alert engine, or it can be done once daily by having someone investigate a list of new domains that appeared the previous day. This technique is easier to apply if you first filter out a list of known good domains, using something like the top 1 million domains provided by Cisco Umbrella.

Reference:

- <https://sec555.com/kaf0w>

DNS Enumeration and NXDOMAIN status

Do **file.xyz.com**, **web.xyz.com**, or **vpn.xyz.com** exist? If not, generate an NXDOMAIN response code.

Some attacks involve asset discovery using DNS

Will generate a lot of NXDOMAIN requests

Possible reasons:

1. The user has a typo when accessing a site
2. A misconfigured application is running
3. Google Chrome is looking for DNS hijacking
4. **An infected device using domain generating algorithm (DGA)**

- Uses DNS brute forcing with tools such as DNSRecon or Nmap's dns-brute script

- If internally done, you will see these
- If external, you will only see these if you host the authoritative DNS servers for your domain

When a DNS query is performed for a name that does not exist or for some reason cannot be found, an NXDOMAIN response is generated. This response type should have a low number in your environment and is most commonly seen due to the following:

- User error.
- System misconfigurations.
- Since 2010, Google Chrome and Chromium-based browsers use random domain names (usually between 7 and 15 characters) to test for DNS hijacking, at system startup and upon DNS setting reconfiguration.¹
- However, if NXDOMAIN responses begin presenting in high occurrences, it may indicate malware is using a domain generating algorithm (DGA) (covered later in more detail).

Sometimes, attackers will use DNS enumeration to discover what assets are available for later attacks. For example, they may request the names: file, Web, VPN, etc., against your externally facing domains. By doing so, they can discover if you have any externally facing servers of interest, such as a VPN appliance. Unfortunately, if this method of discovery is used externally, you would not be able to detect it unless the authoritative name server for your domain is maintained by your organization and capable of generating query logs, which you need to review to be able to identify this.

On the flip side, if an attacker lands an initial foothold on your network, they may use DNS internally to discover assets. If so, this will generate a burst of NXDOMAIN response codes. This is not a common practice for modern malware or adversaries. However, many tools, such as Nmap, default to using DNS servers for lookups, unless explicitly disabled.

This means that if an attacker or malware makes a mistake, a lot of DNS traffic and NXDOMAIN responses will be generated.

1. <https://sec555.com/630a7>

Fast Flux



Since IP deny lists are commonly utilized, malware authors adapted and began using DNS to bypass filters. One such method is using fast flux, where a domain exists, but the IP addresses associated with the domain are constantly changing. This can be one or more IP addresses per A record that are changing every minute to a couple of minutes (in this example, the TTL is set to 150 seconds, instead of the typical 14,400 seconds). For example:

Domain name	TTL	Type	Record
rainbowbank.com	150	A	1.1.1.1
rainbowbank.com	150	A	2.2.2.2
rainbowbank.com	150	A	3.3.3.3
rainbowbank.com	150	A	23.124.228.102

Let's look at the steps outlined in this example:

1. The victim tries to access rainbowbank.com (the attacker's malicious website), perhaps by clicking on a link contained in an email. A DNS request goes out to the DNS server handling resolution for rainbowbank.com.
2. The DNS server provides a response for where to go, so that rainbowbank.com is accessed. The response points to one of the IP addresses that are infected machines and are a part of the attacker's botnet. The malware on these bots allows for self-registering against the fast flux system's domain.
3. The victim gets pointed to one of the bots to retrieve the rainbowbank.com web page.
4. That infected machine relays traffic to the actual backend attacker services (attacker's malicious web server), using a proxy script the attacker installed upon compromise.
5. The attacker's server is hosting a phishing web page, which is provided as a response.
6. That phishing web page ultimately gets served to the victim via the botnet.

By using infected computers in this manner, the attacker's backend servers and services are hidden and

protected while making it more difficult for defenders to take that infrastructure down. It's also worth noting that the attacker doesn't need to own the DNS server used in the above example. What they would commonly do is:

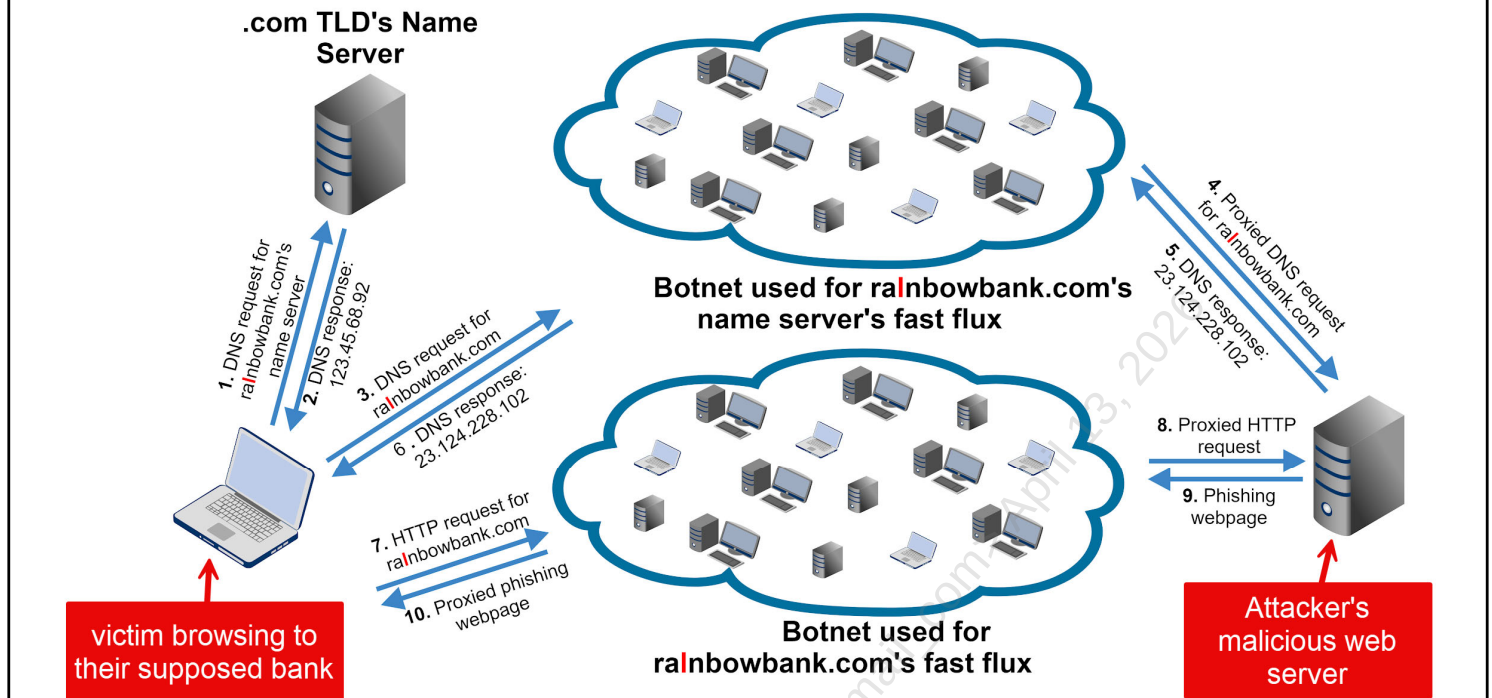
- ✓ Register rainbowbank.com
- ✓ Set the name servers to ns1.evil-dns.net, which they control.
- ✓ Frequently rotate A records for rainbowbank.com to point to new botnet nodes every few minutes.

They don't really need to run a full public DNS server. They just need **any authoritative server that can respond to lookups for their domain, with rapidly changing IPs.**

Image reference:

- <https://sec555.com/4k2se>

Double Fast Flux



In a fast flux scenario, it is very difficult to get rid of all the hosts in a botnet, so typically, the DNS server offering domain resolution would be taken down to solve this problem. But then, attackers started using the same fast flux technique to protect their DNS servers, which made double fast flux emerge.

Double fast flux is fast flux, but with the inclusion of additional layers. Typically, this extra complexity uses infected machines to handle A records (like in the case of single fast flux) **and then another set of infected machines to handle name server records**. Therefore, some machines are acting as DNS proxies to hide the attacker's DNS servers from being discovered, and another set of systems are acting as proxies for other services, such as an attacker's web server or botnet control servers. Essentially, in fast flux, the only thing changing is A record entries. **In double fast flux, A record entries and name server entries change.** Let's look at the diagram:

1. The victim performs an initial DNS request to find rainbowbank.com and gets directed to the .com TLD server.
2. The .com TLD server provides a response, which points to the name server handling resolution for rainbowbank.com.
3. The request is now sent to the attacker's name server botnet (which contains numerous machines acting as DNS nameservers) so that defenders can't take them all down. In a fast flux scenario, that would be one nameserver that could be taken down. But in a double fast flux scenario, the attacker uses a whole botnet to handle this.
4. The request now gets proxied by the botnet and routed to the attacker's actual name server.
5. A DNS response is then provided, which will travel through the name server botnet.
6. The DNS response gets delivered to the victim, and from now on, the previous fast flux scenario is followed again, in steps 7-10.
7. The victim gets pointed to one of the bots to retrieve the rainbowbank.com web page.
8. That infected machine relays traffic to the actual backend attacker services (attacker's malicious web server), using a proxy script the attacker installed upon compromise.

9. The attacker's server is hosting a phishing web page, which is provided as a response.
10. That phishing web page ultimately gets served to the victim via the botnet.

You can think of double fast flux as a two-tier fast flux operation. One tier is used for the attacker's name server and another is used for the attacker's malicious web page.

Again, defenders discovered a way to combat this: Use internal DNS servers to act as the authoritative name servers for the fast flux domain to prevent future connections. This, then, led attackers to switch to using domain generation algorithms (DGA), analyzed shortly. Regardless, today, both DGA and fast flux are used by malware.

Image reference:

- <https://sec555.com/4k2se>

Fast Flux Detection

Can be difficult to detect as DNS load balancing is common

- Requires looking at frequency and answer count

Possible detects

- Look for *repetitive* DNS calls with TTLs < 300
- Look for answers with a count > 12
- If used for botnet control, look for persistent callbacks
- Subscribe to TI feeds
- DNS protection services

In many ways, fast flux can look like normal DNS traffic as it closely resembles DNS load balancing or the use of content delivery networks. Both have widespread use on the internet. That means that a domain would be hosted in multiple servers, which provides increased availability and performance.

The main difference is the repetitive DNS queries and the possible use of higher-than-normal answer counts. While it is common for popular sites, such as social media, to have low TTLs and multiple calls throughout the day, it does not generate hundreds of DNS queries and responses and does not do so 24/7 as a botnet command and control (C2) commonly would.

Therefore, consider:

- ✓ Monitoring number of DNS queries by source IP address with short TTLs (indicating that the attacker is using short TTLs to force another DNS response to be used shortly, pointing to another IP address, instead of waiting for several minutes, as in the normal scenario of DNS load balancing).
- ✓ Monitoring number of DNS queries with more than 12 answers provided (which you can adjust to another threshold, if you desire). This can again indicate fast flux since you wouldn't commonly expect multiple answers to be provided to any legitimate DNS query. How many IP addresses do you expect a legitimate website to respond to? Certainly, it won't be using 50 or 60 different servers.
- ✓ Monitoring DNS connections for any persistent connections (indicating connectivity to the attacker's malicious infrastructure).
- ✓ Subscribing to threat intelligence feeds to obtain up-to-date information about known fast-flux IP addresses and name servers.
- ✓ Setting IDS/IPS rules for failed connection attempts across large number of IP addresses.
- ✓ Using a DNS protective service, like Cisco Umbrella or Cloudflare gateway.

DGA (Domain Generation Algorithms)

Domain generation algorithms are used by malware C2

- Generate hundreds or thousands of domains (i.e., abxkjfodiueuiyutfreiughfdjkgjh.com)
- Used to avoid DNS deny lists
- Allows changing IPs frequently

Generates a large number of random fake DNS names

- But C2 is established only using a **small working** subset

Infected machines constantly pull new DNS records

- Resulting in a lot of NXDOMAIN errors

A more modern use of DNS for hiding is the use of DGA (domain generation algorithms) by malware.¹ Attackers create malicious code to allow hundreds or even thousands of fictitious different domain names to be generated and accessed. Out of all of those, only a handful comprise a subset with actually working malicious domains, resolving in systems that the attacker's command and control infrastructure uses. That allows for additional malware and instructions to be downloaded to the victims. These are used to avoid IP and DNS deny lists and can be extremely difficult to catch.²

Ways to Detect

- ✓ **Domain randomness:** These domains, typically, have high entropy to bypass security controls. As stated before, randomness is an attacker strength that can become their weakness when used with frequency analysis.
- ✓ **NXDOMAIN errors:** Infected machines calling out to a DGA-protected system will generate a large amount of NXDOMAIN records as they are trying to perform DNS lookups of non-existent domains. That is something you can monitor and raise alerts for.
- ✓ **Threat intelligence feeds:** Use threat intelligence feeds to identify known DGA domains.
- ✓ **Reverse malware analysis:** Performing reverse malware analysis can help you identify DGA operation, and you can then use the domains the malware uses in your SIEM and other security controls.

1. <https://sec555.com/f4z03>

2. <https://sec555.com/rdqkt>

NXDOMAIN Detection

Monitoring NXDOMAIN responses by IP will detect:

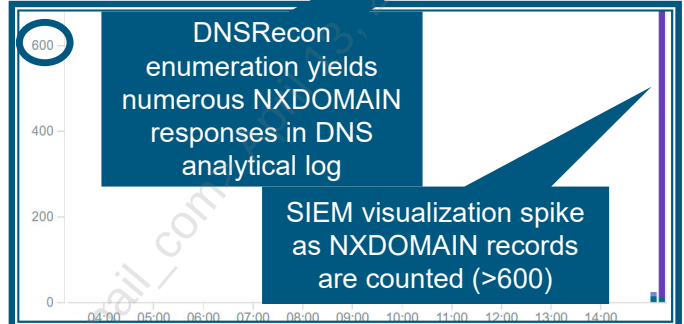
- DGA use
- DNSRecon
- Misconfigured systems

Subdomains of domain.avengers			
sec555-srv	Host (A)	192.168.1.10	
www	Alias (CNAME)	SEC555-SRV.domain.avengers	
www.test.int	Alias (CNAME)	SEC555-SRV.domain.avengers	
acg	Alias (CNAME)	SEC555-SRV.domain.avengers	
acv	Alias (CNAME)	SEC555-SRV.domain.avengers	
add	Alias (CNAME)	SEC555-SRV.domain.avengers	
ado	Alias (CNAME)	SEC555-SRV.domain.avengers	

```

# dnsrecon -d domain.avengers -D ./dnsmap.txt -t brt --xml ./dnsrecon.xml
[*] Using the dictionary file: ./dnsmap.txt (provided by user)
[*] brt: Performing host and subdomain brute force against domain.avengers..
[+] CNAME acg.domain.avengers SEC555-SRV.domain.avengers
[+] A SEC555-SRV.domain.avengers 192.168.1.10
[+] CNAME acv.domain.avengers SEC555-SRV.domain.avengers
[+] A SEC555-SRV.domain.avengers 192.168.1.10
[+] CNAME add.domain.avengers SEC555-SRV.domain.avengers
[+] A SEC555-SRV.domain.avengers 192.168.1.10
[+] CNAME ado.domain.avengers SEC555-SRV.domain.avengers
[+] A SEC555-SRV.domain.avengers 192.168.1.10
[+] 8 Records Found
  
```

12/02/2025 07:30:20	0A58	PACKET	000001A9C0FEC8A0	UDP	Snd	192.168.1.30	3fb4 R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A58	PACKET	000001A9C0FCD3B0	UDP	Snd	192.168.1.30	a8e1 R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A50	PACKET	000001A9C0EBF4F0	UDP	Snd	192.168.1.30	dfeb R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A50	PACKET	000001A9C0FEC8A0	UDP	Snd	192.168.1.30	9e07 R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A50	PACKET	000001A9C0F6F560	UDP	Snd	192.168.1.30	b710 R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A50	PACKET	000001A9C0EBF4F0	UDP	Snd	192.168.1.30	4806 R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A50	PACKET	000001A9C0FEC8A0	UDP	Snd	192.168.1.30	4ff9 R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A50	PACKET	000001A9C0F6F560	UDP	Snd	192.168.1.30	6da7 R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A50	PACKET	000001A9C0EBF4F0	UDP	Snd	192.168.1.30	fcba R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A50	PACKET	000001A9C0FEC8A0	UDP	Snd	192.168.1.30	af9b R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A50	PACKET	000001A9C0F6F560	UDP	Snd	192.168.1.30	4c5e R Q [8385 A DR NXDOMAIN]	A
12/02/2025 07:30:20	0A58	PACKET	000001A9C0EBF4F0	UDP	Snd	192.168.1.30	15ce R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A58	PACKET	000001A9C0FCD3B0	UDP	Snd	192.168.1.30	15ce R Q [8385 A DR NXDOMAIN]	AAAA
12/02/2025 07:30:20	0A58	PACKET	000001A9C0F6F560	UDP	Snd	192.168.1.30	2dbd R Q [8385 A DR NXDOMAIN]	AAAA



By simply monitoring NXDOMAIN responses by IP addresses, you will be able to uncover the use of DGA, DNSRecon, and misconfigured systems. Again, NXDOMAIN is a normal occurrence, but the frequency/volume of the occurrence is typically low.

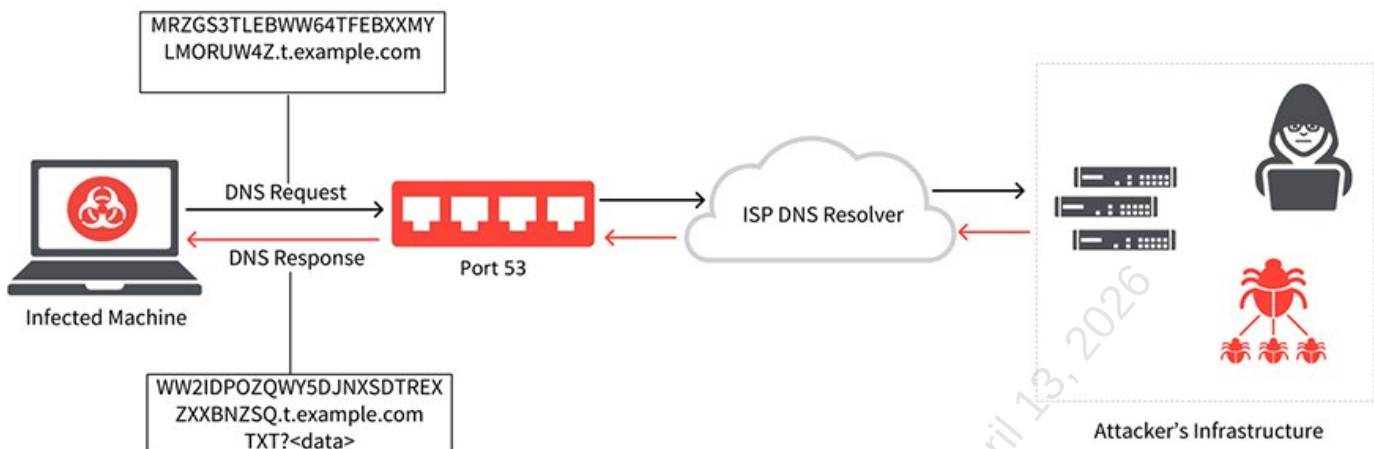
In this example, you see the following:

1. At the upper left-hand side, there's a snapshot of the DNS configuration for a domain called "**domain.avengers**", where there are a few subdomains configured, such as:
 - acg.domain.avengers
 - acv.domain.avengers
 - add.domain.avengers
 - ado.domain.avengers
2. The attacker is using DNSRecon¹ to perform a brute force enumeration of **domain.avengers** using a custom DNS name list called **dnsmap.txt**, as shown at the lower left.
3. At the upper right-hand side, you can see what that looks like at the DNS server. Numerous NXDOMAIN records appear in the DNS analytical log for requests originating from 192.168.1.30.
4. Those are also counted as visualized in the SIEM tool with a purple line, shown at the lower right-hand side. There are more than 600 requests originating from the same source IP address, which is what generated this spike.

To simply monitor NXDOMAIN responses, consider setting a threshold, such as show all systems that have generated more than 50 NXDOMAIN responses within a given period of time.

1. <https://sec555.com/0ghfb>

DNS Tunneling



DNS tunneling is highly successful in exfiltrating data and controlling bots

Works if you allow port 53 outbound to anywhere

DNS tunneling uses the DNS protocol to send data

DNS tunneling uses the DNS protocol to communicate directly with an attacker's external DNS server. This is how:

1. The attacker performs a domain registration, e.g., example.com. The name server of that domain (the attacker's external DNS server) points to the attacker's machine. There, a malicious DNS tunnelling program is installed (e.g., Iodine or DNScat2).
2. The attacker sends malware to a victim on the target network (malicious DNS tunnelling client software), usually present on a company's internal network (behind any firewall). Since most of the time, DNS requests and responses are allowed to and from end systems (like an infected desktop or laptop on the company network), the infected machine sends a DNS request query to the company's ISP DNS resolver (which is a server that relays requests for IP addresses to root and top-level domain servers). That DNS query contains encoded data in the subdomain of the DNS request. Something like, `NDY1ODQ1Nzg0NTEyNjU5OA.example.com` (which is an example of a card being exfiltrated using Base64 encoding).
3. The DNS resolver sends the victim's query to the command-and-control server of the attacker, where the tunnelling program is installed. That means a connection is established from the victim machine to the attacker's machine through the DNS resolver. This tunnel can be used to exfiltrate data or for other malicious purposes. Because there is no direct connection between the attacker and victim, it is more difficult to trace the attacker's computer.

Key advantages of this technique include:

- ✓ **Firewall evasion**, since UDP 53 is usually open.
- ✓ **Living off the land**, since DNS traffic is noisy by nature, so it's hard to identify anything malicious.
- ✓ **Bidirectional channel**, since the attacker can both send and receive data.

Image reference:

- <https://sec555.com/pmy4o>

DNS Tunneling Detection

Abnormal DNS requests

- Limit which external DNS servers can be used
- Monitor for large number of requests from a single IP address
- Monitor for use of special DNS query types, such as TXT records
- Monitor NXDOMAIN records
- Monitor the size of outbound DNS queries (i.e., more than 40 characters)
- Monitor domain name entropy (look for high degree of randomness, which is indicative of encoded data)

A more advanced use of DNS tunneling works if an organization allows external DNS recursion, even if only to authorized DNS servers. Under normal conditions, an internal DNS server will reach out to an authorized DNS server, such as Google's 8.8.8.8, to perform recursive DNS lookups. That means that the initial client DNS request gets relayed from DNS server to DNS server, until a final response is provided about the whereabouts of that domain. Because of this, malware can use DNS tunneling through legitimate and authorized servers. In plain English, that means **if you allow DNS externally, this attack will work.**

Lenny Zeltser has written a great article demonstrating both DNS tunneling techniques and how to perform them.¹

The most common tools available to create a DNS tunnel are dnscat2² and iodine.³ They are commonly used by penetration testers to demonstrate an organization's lack of detection around DNS tunneling. Other tools include:

- DNSExfiltrator⁴ (DNS tunneling using PowerShell)
- SoDNS⁵ (designed for arbitrary command execution)
- Dns2tcp⁶ (TCP over DNS tunneling)
- DET (Data Exfiltration Toolkit,⁷ performs data exfiltration over ICMP, HTTP, and DNS)

Given that most organizations cannot detect DNS tunneling, you would figure it is difficult to find. However, it is not impossible. Consider the below:

- ✓ Monitor for large volumes of DNS requests from a given source IP address.
- ✓ Monitor NXDOMAIN responses by IP address.
- ✓ Since the goal is to communicate or send large amounts of data, DNS tunneling is often performed over DNS TXT records. Monitoring the number of TXT records requested by a given source IP and the size of outbound DNS requests is a quick way to detect tunneling.
- ✓ Monitor DNS traffic and identify if outbound requests are originating directly from client devices

instead of DNS resolvers. The expected path should be a client device making a DNS query to your internal company DNS server, which in turn can follow its normal resolution path.

- ✓ Ideally, organizations would limit external DNS from authorized internal servers to authorized external servers and then establish monitoring. This, combined with DNS query logging, will catch misuse of DNS.

1. <https://sec555.com/41z79>
2. <https://sec555.com/ercos>
3. <https://sec555.com/e36q5>
4. <https://sec555.com/53eyp>
5. <https://sec555.com/qkhuf>
6. <https://sec555.com/e5f8z>
7. <https://sec555.com/sxygi>

Outbound Without DNS

Take all unique external destination IP addresses and compare against DNS query IP address answers

Exceptions will include:

- Cached DNS resolution
- Microsoft IP addresses
- Content Delivery Network IP address such as Akamai
- Root DNS server IP addresses
- Hardcoded IP addresses
- DoH (DNS over HTTPS)
- Vendor stuff (e.g., Defender for Endpoint)

Add each as exceptions (consider using ASN numbers)

Each time traffic is generated to an external IP address, you should be checking to see if that matches responses provided to DNS queries generated. On a normal network, you would only expect the below not to have a DNS request and response with a matching IP address:

- Cached DNS resolution (a DNS request will only be required when the cache expires).
- Microsoft IP addresses.
- Content Delivery Network (CDN) networks.
- Root DNS server IP addresses.
- Hardcoded IP addresses (commonly seen in IoT, legacy applications, and embedded software).
- DoH (The initial DNS lookup can happen inside an encrypted tunnel, so you won't see it in DNS logs).
- Occasional vendor stuff.
- **Malware.**
- **Policy violations.**

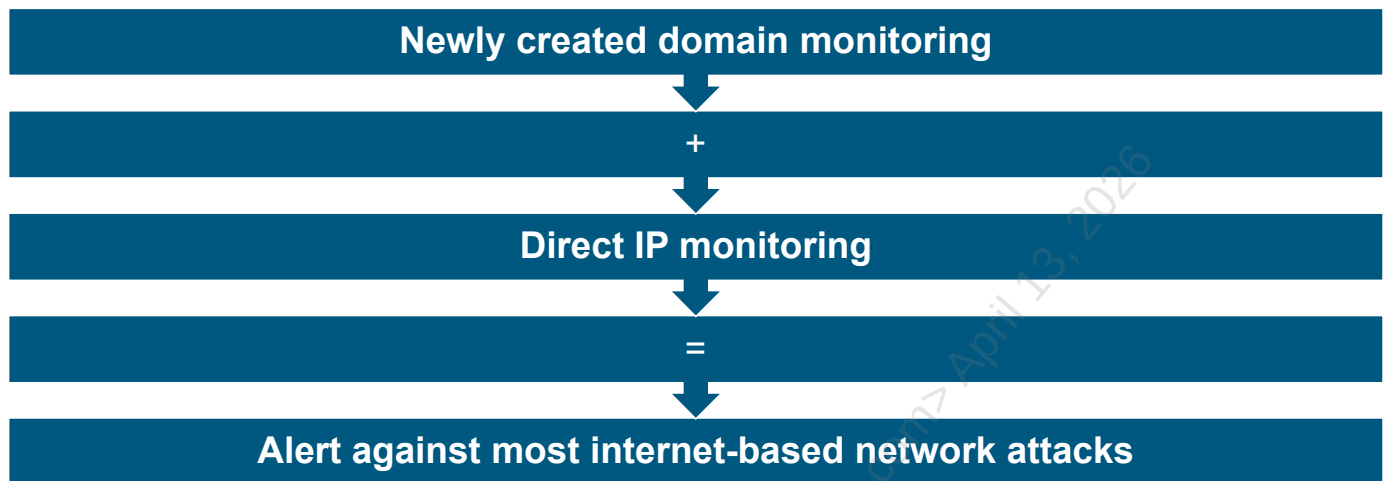
It is easy to filter out all these so that all that is left are:

- Malware
- Policy violations

To make this filtering easier, ensure you use ASN numbers for things like CDN networks.

Catching Internet-Based Malware

- It is easy to set up new domain access monitoring.
- It is fairly easy to find direct IP calls to the internet.



Take the time to implement:

- Newly created domain monitoring, and
- Direct IP monitoring

You will then effectively detect most internet-based network attacks. This is because every new connection, whether by DNS domain or by IP address, will end up on a dashboard for review.

But you might still wonder what kind of internet-based attacks this would miss.

The above two techniques would not see traffic from known good domains.

For example, modern malware occasionally will use things like social media sites for command and control. **If these are authorized sites, you would not see the attack.** Basically, an attacker can use authorized sites to attack or control systems, and these two techniques would be oblivious to it.

If you are trying to implement monitoring for this, you will need to use key filtering information such as:

- ASN.
- Reverse DNS.
- Top 1 million URLs (which, of course, you can customize to your own and add more or trim the list down to fit your needs).

Summary

Understanding how DNS works and what logs should be enabled for visibility are key differentiators in increasing your environmental visibility and catching attackers.

Be aware of common DNS attacks (such as fast flux, DGA, and DNS tunnelling) and enrich logs to detect rare domains. Create baselines of traffic from authorized DNS sources to quickly identify what stands out.

This area is intentionally left blank.

Investigating DNS Logs

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- SMTP
- DNS
 - ▢ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ▢ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ▢ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ▢ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will understand how HTTP and HTTPS logging work and what techniques can be used for efficient monitoring.

This area is intentionally left blank.

HTTP: Direction Matters

Outbound HTTP

Access to external web server/service

- **Expected Use:** Web Client
- **Unexpected Uses:**
 - Command and control
 - Remote access Trojan or DDoS
 - Stage 2 malware downloads
 - Data exfiltration (large uploads)

Inbound HTTP

Access to internal web server/service

- **Expected Use:** Web Server
- **Unexpected Uses:**
 - Brute force logins
 - SQL injection
 - Command injection
 - XSS/XSRF attacks

One thing to note is that depending on the direction of an HTTP connection, things end up being different.

For example, if HTTP is being used from an external system inbound into the network, then under normal conditions, this is most likely an HTTP connection into an organization's web server. If, however, an HTTP connection starts from the inside of the network and then reaches out to an external system, it is most likely a web client reaching out to an external web server, such as an internal web browser reaching out to a website. **This ultimately means the techniques used around HTTP apply differently based on direction.**

Techniques revolving around attacks against an internal web server will apply to inbound connections. Examples include:

- **Brute force authentication attempts** (attacker trying to break into an account).
- **Web application attacks** detailed in the OWASP Top 10¹ (such as SQL/command injection, XSS, XSRF, etc.).

Techniques around internal systems talking to the internet in an unauthorized manner will be applied to outbound connections. Examples of these include:

- **C2 traffic** (traffic from compromised internal machines to the attacker's command and control infrastructure).
- **RAT (Remote Access Trojan) or Application DDoS (distributed denial of service) attacks:** Both of these generate large volumes of outbound network traffic. For example, a tool like Slowloris² can perform an HTTP DDoS, with vast amounts of HTTP requests.

1. <https://sec555.com/aojze>
2. <https://sec555.com/ytu7c>

HTTP Log Sources

Inbound

- Web Application Firewall
 - Provides uniform web logs for varying services
- Malware Detonation Device
- Apache, IIS, Nginx/IDS

Outbound

- Web Proxies
- Next-gen Firewall/IDS/IPS

HTTP log includes more info than just port 80 access

Special sources

- **Zeek/Packetbeat:** Log both inbound and outbound HTTP
- **Script:** Does anything it is told to do
- **Cloud Logs/APIs/Applications:** Vary dramatically in quality/quantity

Some of the most common sources for HTTP logs are web servers and web proxies. Essentially, if a company uses a web server and has no proxy, it would most likely use the IIS or Apache logs from the web server. However, if they're using a web proxy (like Squid, Zscaler, Bluecoat, or any other vendor), then those logs would be the ones collected and analyzed.

Having said that, other sources for HTTP logs can include:

- Advanced firewalls (also known as next generation or next gen or NGFW).
- Network IDS (Intrusion Detection System) or IPS (Intrusion Prevention System).
- Malware detonation devices (sandboxes like CAPE).
- Network extraction of logs (from a tool like Zeek or Security Onion).

Notice that an HTTP log is not simply a log of who or what accessed TCP port 80. HTTP logs are logs that include additional fields such as virtual host, user agent, and a lot more that can help you identify the activity that took place on the network.

Web Server Log Formats

Common Log Format

- Text file format for web servers like Apache (e.g., Apache access logs)

Combined Log Format

- Default for Nginx (Common Log Format + referrer + user agent)

W3C Extended

- Default for IIS
- Includes field header and allows adding/removing fields

The three most typical web server log formats are: Common Log Format, Combined Log Format, and W3C extended.

- **Common Log Format** is an initial format that includes the following 7 fields (in a text-based file):

```
10.0.0.101 - Nick [10/Mar/2025:13:55:36 -0700] "GET /index.html HTTP/1.0" 200 2326
```

1. Remote host address (10.0.0.101)
2. Remote logname (usually unused or blank and represented by -)
3. Authenticated user (Nick). If the user isn't authenticated, this will be represented by -
4. Timestamp (10/Mar/2025:13:55:36 -0700)
5. Request (method, resource, and protocol) (GET /index.html HTTP/1.0)
6. HTTP response status code (200)
7. Response size in bytes (2326)

- **CLF (Combined Log Format)** is basically the same as the Common Log Format but adds two very important fields in the logs: Referrer (stating what page was used to direct the user to the resource being accessed) and user agent (denotive of the client browser and operating system information).
- **W3C extended** format was proposed by W3C (World Wide Web Consortium) and designed to allow more granular control of web logs. It allows for inclusion or exclusion of fields, proxy logging, and more.

Web Server Default Logs

Apache on Ubuntu (Server time zone set to UTC)

```
10.0.0.107 - - [06/Mar/2025:14:41:55 +0000] "GET /favicon.ico
HTTP/1.1" 404 652 "https://10.0.0.10/" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/54.0.2840.87 Safari/537.36"
```

Nginx on Ubuntu (Server time zone set to Pacific)

```
10.0.0.107 - - [06/Mar/2025:06:59:26 -0800] "GET /favicon.ico
HTTP/1.1" 404 209 "http://10.0.0.110/" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/54.0.2840.87 Safari/537.36"
```

IIS on Windows Server (Server time zone set to Central)

```
2025-03-06 14:24:48 192.168.1.10 GET /favicon.ico - 80 -
192.168.1.30 curl/8.8.0 - 404 0 2 2
```

This page includes default logs for Apache, Nginx, and IIS for accessing the file **favicon.ico**. For Apache and Nginx, the logs came from an Ubuntu system, while the IIS logs came from a Windows 2022 Server.

In these logs, each system was set to a different time zone. For Apache and Nginx, the time zone is specified in the log by default. **For IIS, the log does not specify the time zone, so be careful and know thy logs.**

Apache log breakdown:

```
10.0.0.107 - - [06/Mar/2025:14:41:55 +0000] "GET /favicon.ico
HTTP/1.1" 404 652 "https://10.0.0.10/" "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/54.0.2840.87
Safari/537.36"
```

- **10.0.0.107**: Remote client IP address.
- **First -**: The RFC1413 identity of the client, which is not used, so you see the -.
- **Second -**: The username. If there's a -, it means the user hasn't authenticated.
- **06/Mar/2025:14:41:55 +0000**: Access date, time, and time zone.
- **GET /favicon.ico HTTP/1.1**: Client's request line.
- **404**: HTTP response from server (in this case, indicative of "Not Found").
- **652**: HTTP response size in bytes (without the HTTP headers).
- **https://10.0.0.10**: Referrer URL (originating page of the client request).
- **Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/54.0.2840.87 Safari/537.36**: User-agent string identifying the browser and client OS (Google Chrome version 54.0.2840.87, Windows NT 10.0; Win64; x64).

IIS log breakdown:

2025-03-06 14:24:48 192.168.1.10 GET /favicon.ico - 80 - 192.168.1.30
curl/8.8.0 - 404 0 2 2

- **2025-03-06 14:24:48**: Date and time of request.
- **192.168.1.10**: IIS server IP address (s-ip).
- **GET**: Client request method (cs-method).
- **/favicon.ico**: Requested resource file (cs-uri-stem).
- **First** -: Query string not present (if it was, it would be something like ?id=478) (cs-uri-query).
- **80**: Server port (s-port).
- **Second** -: Username (here, the dash indicates unauthenticated user) (cs-username).
- **192.168.1.30**: Remote client IP address making the request (c-ip).
- **curl/8.8.0**: User-agent string (here, the CURL CLI utility was used to make this request) (cs(user-agent)).
- **Third** -: Indicates there's no referrer (if there was, it would be present here) (cs(referrer)).
- **404**: HTTP status code (here, "Not Found") (sc-status).
- **0**: Windows sub-status code used for more information (sc-substatus).
- **2**: Win32 status code (2 indicates "Not Found")(sc-win32-status).
- **2**: Time taken to process this request (in milliseconds) (time-taken).

Apache/Nginx Logging

Both generate two logs: **access.log** and **error.log**

The access log is for default logging

The error log records errors



Both allow for granular log formatting and support

Adding/removing fields

Changing the format of fields

Logging with JSON

Apache and Nginx default to creating two logs: **access.log** and **error.log**. The access.log is the default log of requests to the web server, and error.log is a record of errors or abnormalities discovered by the web servers. For HTTP analysis, access.log is what we are primarily analyzing.

Both web servers allow granular customization of logs. They support:

- Adding and removing fields
- Changing the format of fields, such as making dates follow YYYY-MM-DD format
- Tuning the output, such as making the log output in JSON format.

For a sample of customizing Apache log formats, see Apache's official website.¹

1. <https://sec555.com/o5pga>

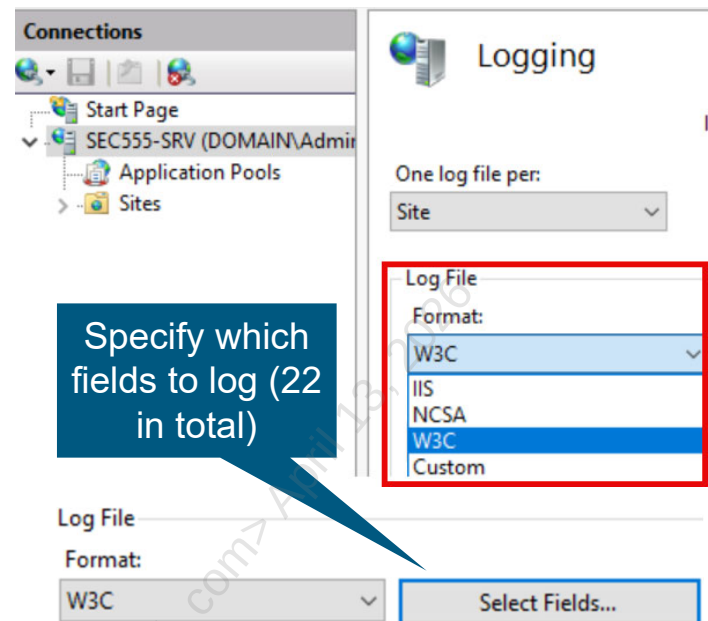
IIS Logging

Supports multiple log formats

IIS: Comma-separated list of fixed fields

NCSA: Log file with fixed set of basic fields

W3C: Extendable log format (recommended and default)



IIS defaults to W3C but has a few other logging options:

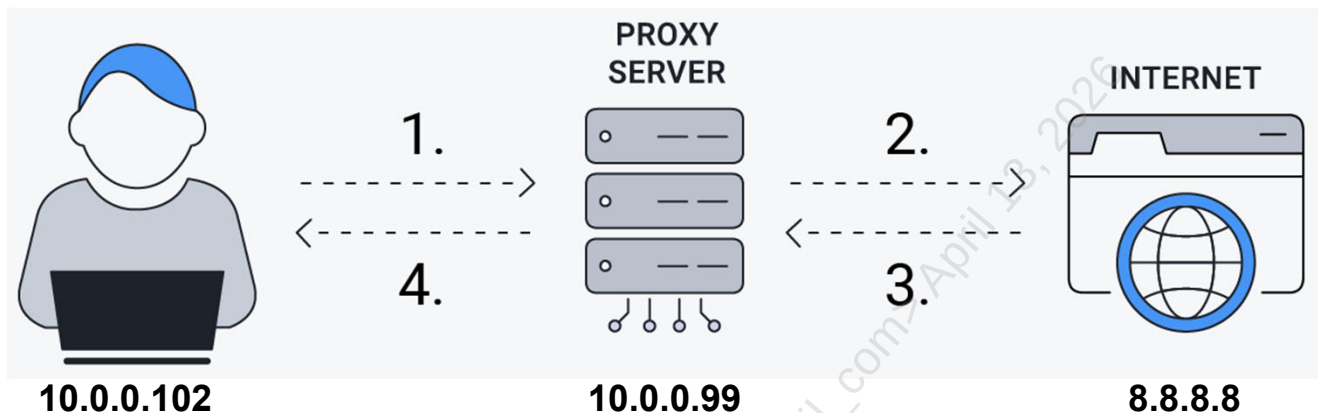
- **IIS:** This is handled by HTTP.sys, and is a *fixed* ASCII text-based format. That is very limiting, as you can't customize the fields that are logged. Fields are comma-separated, and time is recorded as local time. This is mainly used for simple monitoring.
- **NCSA:** Same as the IIS format, this is also handled by HTTP.sys, and is also a *fixed* ASCII text-based format. As noted, that is very limiting, as you can't customize the fields that are logged. Fields are separated by spaces, and time is recorded as local time with the Coordinated Universal Time (UTC) offset. This is mainly used for basic logging and legacy systems.
- **W3C (default and preferred option):** This is handled by HTTP.sys, and is a **customizable** ASCII text-based format, which gives you the power to specify what fields are logged (out of a total of 22). Fields are separated by spaces, and time is recorded in UTC. Note that if you want to collect logs using the Azure Monitoring Agent, they need to be in W3C format. This is the most powerful option and is typically used for flexible analysis and SIEM integration.
- **Custom:** When this option is selected, the **Logging** page becomes disabled. That's because the IIS Manager can't be used for custom logging. The recommendation is to remain with the default W3C, as it allows the most granular control, and the other options are fairly basic and limited.

A key challenge for SIEM solutions is collecting and managing logs from disparate systems.

Unfortunately, this is true across version changes of IIS. This means consistency must be accounted for in log collection as well as configuration. It is possible to alter IIS on the various versions, so the logs all include the same fields and format using Microsoft command line tools or PowerShell (only on newer IIS versions).

Web Proxy X-Forwarded-For (XFF)

- Keep in mind that a proxy terminates a client connection and starts a new one.
 - Means that the source IP address looks like the proxy
- X-Forwarded-For keeps track of original IP address.



Using a proxy can be great for detection and prevention, but it does add a layer of complexity when performing analysis.

For example, assume there's a laptop located at 10.0.0.102, which is connecting to 8.8.8.8 through a proxy at 10.0.0.99. In this scenario, the Google DNS server at 8.8.8.8 will log that 10.0.0.99 is what accessed it, not 10.0.0.102.

This is because a web proxy is an intentional machine-in-the-middle. The laptop has a connection to the proxy (connection 1), and the proxy has a separate connection to the web server (connection 2). This means two separate sockets, with their own IP addresses and ports, are used to make any given connection.

So, if this is the case, how can you tell that 10.0.0.102 is the system actually accessing 8.8.8.8? The answer is the **X-Forwarded-For (XFF)** field. This header field contains the source IP address of the system that made the original request. Keep in mind that some systems require modification of logs to include the XFF field. Zeek, by default, will extract this. Web servers often will not. For instance, **IIS requires enabling advanced logging and then adding the field X-Forwarded-For to log this.**

The X-Forwarded-For field is arbitrary and can be forged (using ZAP¹ or Burp Suite² or even the browser developer tools). **If the XFF field is present, a proxy is being used.** Note that this can be used to find unauthorized proxies. You can create a list of authorized proxies in your environment and monitor for anything else that you see appearing.

1. <https://sec555.com/u7evq>
2. <https://sec555.com/5gmzq>

Image reference:

- <https://sec555.com/9r64i>

HTTP Log Example Fields

Timestamp	Source IP	Source Port	Destination IP	Destination Port
Request Method	Virtual Host	Referrer	URI	User-Agent
Request Bytes	Response Bytes	Status Code	Substatus Code	User
Proxy	Server Name	Duration	Cookie	MIME Type

This page demonstrates the most common HTTP fields included in various HTTP logs (from a server and/or web proxy):

- **Timestamp:** Date and time of client request (some fields may also contain the time zone).
- **Source IP address:** The remote client IP the request originated from.
- **Source port:** The source port of the remote client making the request.
- **Destination IP address:** The destination web server.
- **Destination port:** The destination web server's port.
- **Method:** The request method used (GET, POST, etc.)
- **Virtual host:** The specific website being accessed.
- **Referrer:** The URL of the referring page that was used.
- **URI:** The requested resource path.
- **User-agent:** Denotes what the client used to access the page (e.g., CURL or Mozilla Firefox).
- **Request bytes:** Request length in bytes.
- **Response bytes:** Response length in bytes.
- **Status code:** HTTP server status code.
- **Sub-status code:** Used for more detailed error information in case of an error.
- **User:** The username of the requesting user (if they have authenticated to access a resource).
- **Proxy:** The proxy field contains the IP address of the proxy device that sent the request to the server.
- **Server name:** Web server's hostname.
- **Duration:** How much time (in ms) did the server take to process the request?
- **Cookie:** The value of the client's cookie.
- **MIME type:** The response type sent by the server (e.g., HTML, image, JSON, etc.).

Common HTTP Request Methods

GET: Typically used to retrieve data

HEAD: Used to retrieve HTTP headers only

POST: Typically used to submit data, such as a form

PUT: Used to create or update a specified resource

DELETE: Used to delete a specified resource

Request methods are simply a way for a web client to tell its destination what action it wishes to perform. Some of the most common ones are:

- **GET:** The most commonly seen type of request sent to a web server is the GET method to retrieve a resource of interest.
- **HEAD:** Useful if you want to do a test of a web page's operation or check the web server information.
- **POST:** Used when submitting form data over a web browser.
- **PUT:** Most often seen when uploading a file to a server (commonly used by API services but often dropped by web application firewalls).
- **DELETE:** Used to remove a resource from a web server.

Other less common methods, not mentioned above, are:

- **TRACE:** Used to take input and send it back to the client for debugging.
- **OPTIONS:** Lists out all the VERBS supported for a given site. It doesn't retrieve data but only supported methods from the server (e.g., GET, POST) and other supported capabilities.
- **CONNECT:** Allows tunneling a connection through another endpoint, such as a proxy, and is commonly used in HTTPS connections via proxy.
- **PATCH:** Allows for partially updating an object (in contradiction to PUT, which only allows replacing the entire resource with a new one of your choice).

Define Your Thresholds

Is it normal to have 500 GET requests in 1 minute?

GET per minute by source

- Informational (0–200)
- Notice (201–300)
- Warning (301–400)
- Alert (401 or more)

Is it normal to have 400 POST requests in 1 minute?

POST per 5 minutes by source

- Informational (0–100)
- Notice (101–150)
- Warning (151–200)
- Alert (201 or more)

Let's talk about what normal looks like.

- **Browsing to msn.com:** Generates 90 GET requests in less than a second and then nothing else.
- **Downloading an .ISO file from ubuntu.com:** Generates a handful of GET requests (a couple requests) in less than a second.
- **Submitting a website form:** As simple as a single POST request in less than a second.

What does abnormal look like?

- **Failed SQL injection test:** 228 GET request attempts in 1 minute.
- **Scan/Crawl of web server:** ~1,000 GET requests in 5-minute period.
- **Scan/Crawl of web server using POST:** >100 POST requests in 5 minutes.

There are numerous other examples that can be identified, but it's all about knowing your environment and establishing your thresholds. Would it be normal in your environment to have 550 GET or 250 POST requests in 1 minute?

A simple technique any SIEM can perform is to monitor for a threshold. For instance, it is simple to set up a rule that states if more than X specific number of GET or POST requests are found when grouped by a given field (such as source IP address), raise an alert. This technique applies very differently to workstation subnets versus server subnets. However, it can work across both if a high enough threshold is specified.

HTTP Response Status Codes

1XX Informational

2XX Success

3XX Redirection

4XX Client error

5XX Server error

Example

A lot of 404 responses?
Is it a typo or something else?

- Web scanners
- Vulnerability scanners
- Website misconfiguration

Another component of HTTP communication that is important to understand is HTTP status codes. When an HTTP request is made, the response to the request includes an HTTP status code indicating information about the request. Response codes are grouped in five categories:¹

- **100–199** (Informational responses)
- **200–299** (Successful responses)
- **300–399** (Redirection messages)
- **400–499** (Client error responses)
- **500–599** (Server error responses)

A few examples:

- A successful web request will have a responding status code of **200 OK**.
- A request to a non-existent page would generate an HTTP response status code of **404 Not Found**.
- A request to get a web page that now resides at another URI will generate an HTTP response status code of **308 Permanent Redirect**.
- A request to browse to a resource without having access will result in an HTTP status code of **403 Forbidden**.

Similarly to having thresholds for GET requests, it's advisable to carve out what abnormal numbers for certain HTTP responses would be, like excessive 403 or 404 responses. For example, a scan of a web server using Nikto² generates hundreds of 404 responses, as the scanner is trying to fetch pages and test for the presence of components to flag vulnerabilities.

1. <https://sec555.com/uxs2z>
2. <https://sec555.com/b8nvk>

Naked IP and Mitigation

Naked IP: Using an IP address to navigate to a destination, instead of a URL

- Not normal from inside the network to outside

Exceptions include vendor configurations, such as AV updates

- Easy to filter out using IP or ASN numbers

Naked IPs are common for malware

- The goal is to alert or have an empty dashboard for the use of naked IPs from internal to external

End users browse the internet using DNS names. If you want to browse to sans.org, you use that, not an IP address like 45.60.103.34. Therefore, someone or something accessing the internet using a direct IP address is odd. Making an HTTP request using an IP address is sometimes referred to as a **naked IP address request**.

Monitoring for naked IP address requests, typically, will point out a few vendor platforms. For instance, endpoint security systems have historically been notorious for downloading antivirus definitions or sending data over naked IP address requests.

Filtering out "valid" naked IP address requests is quite simple, especially if you can filter out the data based on ASN numbers. In the event your system does not support ASN lookups, you can use reverse DNS against naked IP addresses. Performing a reverse DNS lookup against an IP address specified in a naked IP address request will often point back to a culprit vendor or company and allow for easily filtering them out.

Even though a lot of malware uses DNS domains, it is still common to see malware use naked IP address requests. Thus, this technique can work quite effectively, especially if you filter and group results per destination IP address. Why would you see tons of traffic going to IP address X.X.X.X, from multiple nodes on your network, if you haven't excluded that as a legitimate destination?

Tagging can also be used for better logic control. For example, if a naked IP comes in, you may want to add a tag of `naked_ip`. However, if a naked IP address comes in and a reverse DNS or ASN lookup shows that it is a CDN network, you could select not to add the `naked_ip` tag or add `naked_ip` but with an extra tag such as `cdn`, `noise`, etc.

URL Length Thresholds

Verify lengths in your environment and set thresholds.

- Possible size to start with is > 140–150

Simple rules are often enough to catch what you want.

- Many things need tuning
- Some web servers use long URLs
- This technique does not work against outbound connections due to ads and CDN networks

Web page visits often fall within a short length. Even with parameters, they tend to fall below a certain threshold. For example, look at the login page of sec555.com, and you will notice that even the longest requests wouldn't typically be more than, say, about 140 to 150 characters:

- `http://sec555.com/login?id=admin`
- `http://sec555.com/login?id=nickjones`
- `http://sec555.com/login?id=mariabennedict`

But under attacks, such as SQL injection, these URL lengths get very large. The longest string within a *failed* SQL injection attack can easily be more than 600 characters.

A technique that works fairly well on internal or DMZ-facing web servers is simple URL length checks. Oftentimes, the length of URLs tends to fall below a certain threshold, such as 100 characters in length.

The technique of monitoring URL lengths is often used by web application firewalls. However, many organizations do not have a web application firewall. For maximum effectiveness, a web application firewall must be properly configured. Those organizations deploying WAFs are often only protecting a subset of their web servers.

For URL length checks to be most effective, a threshold needs to be set and monitored. A possible starting size to monitor is 140-150 characters in length. This is because different sites have different use cases, and some may exceed 100 characters while others may always be less than 50 characters.

This technique is also intended for monitoring organization-controlled web servers. This could work for all HTTP connections, but online advertisements tend to have lengthy parameters for tracking end-user activity. If you take the time to filter out URLs from advertisements, this technique could be applied across all HTTP URLs.

Common User Agents

Google Chrome

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/132.0.0.0 Safari/537.36
```

Microsoft Edge

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 Edg/133.0.0.0
```

Mozilla Firefox

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0)  
Gecko/20100101 Firefox/135.0
```

Opera

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 OPR/117.0.0.0
```

This slide represents what user agents look like using modern internet browsers. All of these were captured on a Windows 11 System, as Wireshark¹ intercepted the traffic over the wire, but you can get more examples if you need to.²

User agents tend to be verbose in giving out information. For instance:

- **Windows NT** specifies the operating system version. Notice that Windows NT 10 is stated in all the above; however, traffic was generated and captured on a Windows 11 system. Note that modern browsers, like Chrome or Firefox, don't distinguish that in their user agent strings. That is extremely useful to remember when performing analysis.
- **Win64 or x64** specifies that the system is running a 64-bit operating system.
- Application versions, such as Chrome/132.0.0.0, identify exactly what version of Chrome is being used.
- Some user agents will also include other details such as the .NET version a system has installed.

How about if you were analyzing some log output and you saw curl/8.8.0? That wouldn't be your typical browser. Granted, a power user like an admin or developer could be doing a test of a web page, but if you saw that originating from a human resource's desktop, you should be investigating that activity further, as it most likely indicates something suspicious taking place.

1. <https://sec555.com/uij9t>
2. <https://sec555.com/4gck5>



This area is intentionally left blank.

TLS Encryption

Encryption is becoming more prevalent and standardized.

- Great as sensitive information is being protected
- Introduces a challenge as it is hard to protect against what you cannot see

Many security systems are blind due to encryption.

- Thus, use of HTTPS is an attacker favorite

With TLS 1.2 and lower, only the payload is encrypted.

- **The certificate is not**
- There are lots of details within a certificate

Certificate Viewer: scarlet-dragonfly.com

General	
Details	
Issued To	
Common Name (CN)	scarlet-dragonfly.com
Organisation (O)	<Not part of certificate>
Organisational Unit (OU)	<Not part of certificate>
Issued By	
Common Name (CN)	WE1
Organisation (O)	Google Trust Services
Organisational Unit (OU)	<Not part of certificate>
Validity Period	
Issued On	Thursday 30 January 2025 at 19:58:31
Expires On	Wednesday 30 April 2025 at 21:58:19
SHA-256 Fingerprints	
Certificate	b51e55d6f982a8b5d00f09b7a34220d3b86a864f75
Public key	f254cac74f08b65b81b243edf9794299c77c6fc98c

A majority of the internet traffic today uses encryption to the point where, by August 2024, there were 317 million certificates.¹ As time passes, numbers increase. The challenge with encryption is that it introduces overhead and reduces defender visibility as to potentially malicious traffic coming in. Defenders love and at the same time hate encryption. We love it because it keeps sensitive information secure from interception, but we also hate it because it's not possible to perform traffic inspection for any malicious indicators.

You will often find that many organizations still do not have the authorization to perform SSL/TLS inspection. This can be due to a lack of senior management approval, or, in some areas, legal restrictions (e.g., imagine inspecting all the encrypted banking traffic. Not good, is it?). However, even if SSL/TLS inspection is disabled, there are techniques that still work and add further defense in depth.

When accessing a website over HTTPS, if you are using TLS version 1.2 or lower, the contents being transported over the network are encrypted. However, the certificate information is not encrypted by design. Because of this, a lot of information can be used within the certificates to identify malware.

The above certificate belongs to the author's company, Scarlet Dragonfly Ltd.² Notice how much detail is present in the certificate? We can use that to our advantage, and that is exactly what we will be doing shortly.

Note: Only TLS versions 1.2 and 1.3 should be used today, as everything before 1.2 is considered deprecated (TLS 1.1 was deprecated in 2021).

1. <https://sec555.com/tr5g6>
2. <https://sec555.com/lej4p>

TLS 1.3 and Certificate Log Sources

Focusing on analyzing certificates, not payloads

- With TLS 1.3, payload/certificate analysis doesn't work by default
- Both options require **TLS Inspection** for TLS 1.3

SSL/TLS Inspection can be combined with decrypt port mirroring

- Which provides full payload analysis

Options for certificate extraction are limited

- **Zeek:** Network extraction with lots of information
- **Suricata:** Network extraction with less information
- **Commercial Solutions:** Limited options and varied success

Certificate analysis is not commonly used

- Should be done, as it is easy, and it works
- Next-gen firewalls use this for SSL inspection without decryption

The value of having certificate information is tremendous. Yet, there are not many options available for extracting log information on certificates. It is not that it is difficult or that it is computationally expensive. It is simply that techniques for analyzing certificates have not gained enough attention yet.

While it would be possible to log certificate information at a proxy or a next-gen firewall, these solutions typically do not offer that capability. Network extraction using Zeek¹ or Suricata² tends to be the primary way to gather this information.

Next-gen firewalls have started using this technique to catch malware (e.g., Palo Alto PA-series, Cisco Firepower, Fortinet FortiGate, Check Point firewalls, Sophos XG, Juniper SRX). Also note that there are certain TLS inspection settings that can allow us to analyze the certificate information but do not decrypt the data.

1. <https://sec555.com/rn32w>
2. <https://sec555.com/w64lx>

Common Certificate Fields

Timestamp	Source IP	Source Port	Destination IP	Destination Port	Basic Constraints
Key Algorithm	Key Length	Key Type	Not Valid Before	Not Valid After	Serial Number
Signing Algorithm	Subject	Version	Common Name	Organization	Organizational Unit
Locality	State	Country Code	Email	Issuer Info	And more

This page demonstrates some of the common fields included in certificates. All of these, and a little bit more, come with the default Zeek x509 logs. Let's see what they are:

- **Timestamp:** The date and time the certificate was generated.
- **Source IP Address:** Sender IP address.
- **Source Port:** Sender port number.
- **Destination IP Address:** The IP address of the destination server.
- **Destination Port:** The server's port number (typically **443** for HTTPS)
- **Basic Constraints:** Defines whether the certificate is for a Certificate Authority, a website, or a user.
- **Key Algorithm:** What is the cryptographic algorithm used (e.g., RSA, ECC, DSA)?
- **Key Length:** The size of the cryptographic key in bits (e.g., RSA key 2048-bit).
- **Key Type:** Indicates the key type (X.509 certificates contain the public key).
- **Not Valid Before:** The date when the certificate starts being valid.
- **Not Valid After:** The expiration date of the certificate.
- **Serial Number:** Unique serial number identifying the certificate, which the CA assigns.
- **Signing Algorithm:** The cryptographic hash function and signature algorithm (e.g., **SHA-256 with RSA**).
- **Subject:** The entity the certificate is issued to (usually a website, user, or service).
- **Version:** The X.509 version (commonly v3).
- **Common Name (CN):** The domain name or entity name (e.g., www.scarlet-dragonfly.com).
- **Organization (O):** The company or entity that owns the certificate (e.g., Scarlet Dragonfly).
- **Organizational Unit (OU):** A sub-division of the organization, if defined (e.g., Security Engineering).
- **Locality (L):** The city where the company or entity is located (e.g., Palo Alto).
- **State (ST):** The certificate company or entity's state or province (e.g., California).
- **Country Code (C):** The country of the entity or company (e.g., UK, US).
- **Email:** The contact email address associated with the certificate.
- **Issuer information:** The entity that signed and issued the certificate (e.g., DigiCert, GlobalSign)

Malware and Attack Tools Break Patterns

Malware and numerous attack tools, like Metasploit's Meterpreter, tend to break "normal" certificate patterns

- Partly due to laziness
- Partly due to intended stealth

For example:

- Skips steps in the SSL/TLS handshake
- Contains minimal certificate information

Key principle: Turn an enemy's strength into a weakness

A key tool used by penetration testers is Metasploit.¹ Metasploit is a framework for penetration testing, and one of the most critical components is Meterpreter.² Meterpreter is an exploit payload used for command and control and is extremely feature-rich.

Meterpreter code is open-source, so anyone can freely use its code to mirror capabilities. This means malware can easily build up from functionality that is readily available. As a result, it should be treated as a minimum standard of what you should be able to catch. This means you should be familiar with the functionality and what occurs in order to catch it. Vendors treat Meterpreter as a whack-a-mole game. This means they develop signatures that can detect some techniques, but we need to be more robust.

Both Meterpreter and malware wish to avoid detection. To do this, they often will use encryption to blind security devices, but the methods used to encrypt do not always follow normal patterns. Identifying these differences and monitoring for them can provide opportunities for detection.

Remember: You may not know what happened, but you will at least know that something did.

1. <https://sec555.com/u8zw1>
2. <https://sec555.com/xqh7w>

Zeek vs. Meterpreter SSL Log

# certificate_common_name	Q Q [sec555.com
# certificate_issuer	Q Q [CN=Go Daddy Secure Certificate Authority - G2,OU=http://certs.godaddy.com,
# certificate_organization_unit	Q Q [Domain Control Validated
# certificate_subject	Q Q [CN=sec555.com,OU=Domain Control Validated
# cipher	Q Q [TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
# client_certificate_chain_fuids	Q Q [(empty)
# ve	Q Q [seco256r1
# issuer_common_name	Q Q [Go Daddy Secure Certificate Authority -
# issuer_country_code	Q Q [US
# issuer_locality	Q Q [Scottsdale
# issuer_organization	Q Q [GoDaddy.com\
# issuer_organization_unit	Q Q [http://certs.godaddy.com/repository/
# issuer_state	Q Q [Arizona
# ssl_common_name_frequency_score	Q Q [17.018
# validation_status	Q Q [ok

Which one should I trust?

# certificate_common_name	Q Q [/!tsow
# certificate_issuer	Q Q [CN=!tsow
# certificate_subject	Q Q [CN=!tsow
# cipher	Q Q [TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
# client_certificate_chain_fuids	Q Q [(empty)
# issuer_common_name	Q Q [!tsow
# ssl_common_name_frequency_score	Q Q [3.549
# validation_status	Q Q [self signed certificate

This page demonstrates a legitimate Zeek SSL log for accessing <https://sec555.com> on the left-hand side vs. an illegitimate certificate for a reverse HTTPS Meterpreter connection on the right-hand side.

Let's look at the sec555.com legitimate certificate first.

First of all, note the frequency score is high, which in this case means that it is probably not randomly generated.

Also, the certificate's validation status is marked as **ok**, which is a great indication that the certificate is not self-signed and is an actual legitimate one.

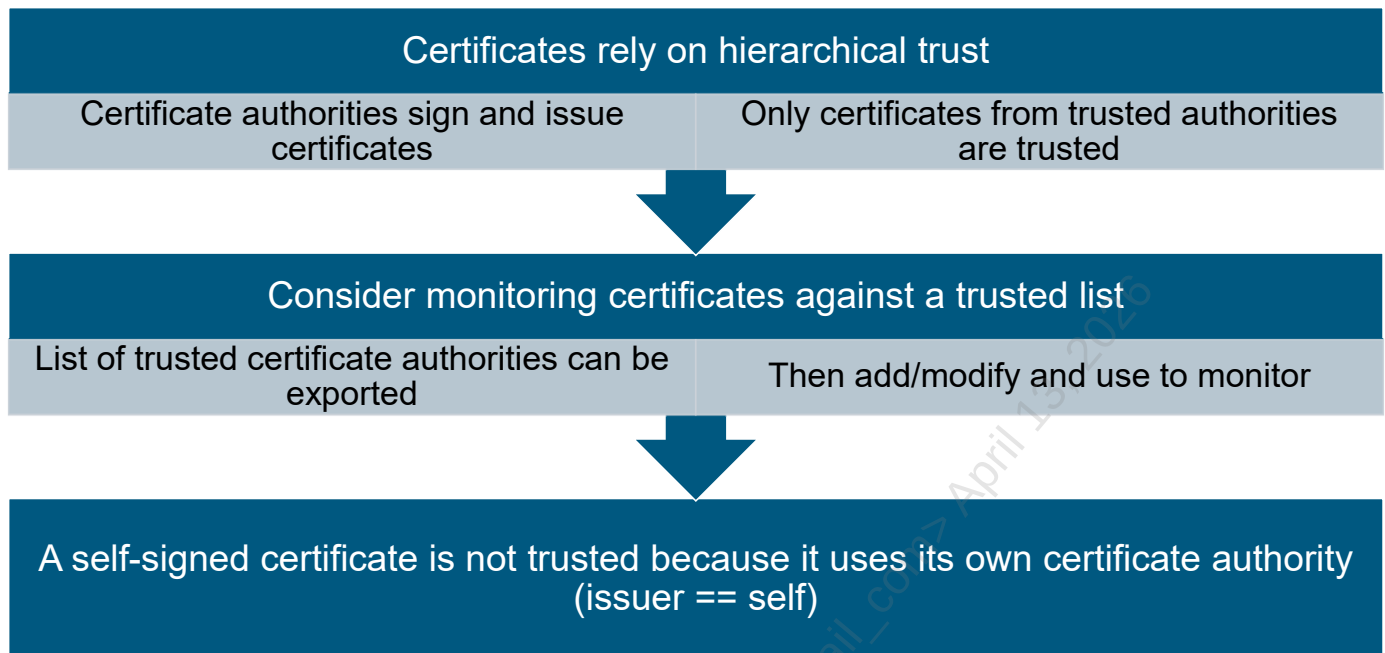
Lastly, the issuer certificate specified all the following fields, which are not something that a malicious certificate would always have populated:

- Organization unit
- Organization
- Locality
- State
- Country code

Now if you look at the Meterpreter certificate, you will notice that a self-signed certificate is identified as being present. Also, note that this time the frequency score is really low, indicating that something possibly malicious is in play.

Lastly, many of the other fields normally in existence (and highlighted earlier) are missing this time around.

Compiling Lists of Legitimate CAs



Certificates should be generated by trusted root certificate authorities. These are vetted entities trusted by your OS and browsers.

Interestingly enough, based on market statistics, 90% of all certificates seem to have been issued by **just six CAs**:¹

1. Let's Encrypt
2. GlobalSign
3. IdenTrust
4. Sectigo
5. DigiCert Group
6. GoDaddy Group

Therefore, it is possible to export a list of trusted root certificate authorities and then have your SIEM look for SSL/TLS certificates that are not issued by entities existing in your trusted list.

A self-signed certificate should never be trusted because that means that it's issued and used by the same entity. So, in essence, it doesn't undergo any checks. That's exactly what was depicted earlier being in use by Meterpreter. Sadly, more than 90% of phishing sites in 2024 used HTTPS, meaning a certificate was in place.² And the problem keeps growing.

1. <https://sec555.com/4n2yt>
2. <https://sec555.com/gxqts>

Be Aware of Let's Encrypt

The internet is moving toward 100% encryption

- Standard sites do not want to buy certificates

Let's Encrypt is a free, automated, and open CA

- Based on non-profit crowdfunding
- Sponsored by many major corporations
- Provides automated certificate issuance and renewal
- Uses domain validation

Already is being used by attackers and malware

One of the major holdbacks in shifting to 100% website encryption is the cost of a certificate.

Let's Encrypt¹ is an organization that was developed to eliminate this cost by providing free and automated certificates.² It does this by allowing a client tool called **certbot** to issue and renew certificates automatically, roughly 30 days before the expiration date stated in the certificate. These certificates are valid for exactly 90 days.

Arguably, this increases security as it introduces encryption and helps mitigate machine-in-the-middle attacks. However, it also decreases security as it allows malware and phishing campaigns to use trusted certificates. Because of the automation, there is a high likelihood that malware and adversaries will start using free certificates such as the ones through this company.

Most people think a certificate guarantees security because of the underlying encryption. But if you really think about it, a certificate is issued merely to show that a public key of an entity belongs to that entity. I.e., if talking about a web server, its certificate simply states that the public key contained in it belongs to the server in question. It doesn't, however, stipulate anything about the legitimacy of the server's owner or the provided security. It could very well be a malicious attacker owning that server.

1. <https://sec555.com/bfmsy>
2. <https://sec555.com/9u3zv>

Validation of Certificate Fields

Sometimes, simply verifying the field content is enough

- A finite list of valid country codes (same with states)

Searching for invalid country code finds things

Malware sometimes generates a name but leaves out things like .com or .net

- Internally, this may be normal
- Externally, not so much

Even searching for common names, missing a period may often work

issuer_common_name	issuer_common_name_frequency_score	certificate_common_name	certificate_common_name_frequency_score
www.rp44tyu3jp7sfat.com	7.4918991267	www.kqwm2iwsvh4xd2q.net	2.38362707549
www.oxj2hqwf5g.com	6.69161908874	www.42ixw6g5fu44w7sth.net	3.6429459776
www.y6bn3trq5cesxk.com	5.67222914792	www.ryfg74xnxjg42ln3.net	3.07996625182
www.m6hoayo5cga.com	8.10067483174	www.dctpbpif6zy54mspih.net	3.23019049697

Generally, HTTPS is used to access websites, and as a result, the certificates are expected to follow a domain name. Therefore, finding a certificate with a common name missing a period is abnormal. Yet malware, or even tools such as Meterpreter, are guilty of this. The slightest detail may sometimes make a substantial difference.

For example, consider the country code field. Compile a list of legitimate country codes to have in an allow list.¹ Create an alert when a certificate uses a country code not present on the allow list.

Certain techniques are complementary to each other. For example, missing fields will find malware that tries to fill out the bare minimum required, so it leaves certain fields unfilled. Yet, it does nothing for malware where all the fields are filled out.

Look at the image. Frequency analysis is applied since it can find randomly generated fields, even if it does nothing for empty fields. Combining one technique with the other helps address any weaknesses that a single technique may have.

The end goal is to make evasion so difficult that adversaries cannot hide. Yes, any individual detection technique can be avoided. But ultimately, with the application of many techniques and processes, it is extremely difficult not to trigger at least one alert.

1. <https://sec555.com/mklzr>

Summary

Remember the importance of establishing patterns of expected traffic (volume of HTTP requests and responses in your network) to identify violations. Having a sound understanding of log formats for common web servers greatly helps you choose wisely. Inspect URL length and identify abnormal activity.

Inspecting SSL/TLS certificates can provide great insight. Finding certificate uncommon names, countries, states, and missing or randomly filled fields can go a long way to finding sources of evil. Be very sceptical about self-signed certificates and caution your users against them.

This area is intentionally left blank.

Investigating HTTP Logs

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines

- SMTP
- DNS
 - ▢ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ▢ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ▢ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ▢ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will gain insight into the intrinsics of Windows logging and appropriate audit policies.

This area is intentionally left blank.

Network vs. Endpoint: Visibility Comparison

Network

- ✓ Small number of network devices
- ✓ Easier management
- ✗ Large amount of data
- ✗ Encryption is a challenge

Endpoint

- ✓ Encryption problem solved
- ✓ Great level of insight
- ✗ Large number of endpoint devices
- ✗ Management can be a problem
- ✗ Narrowed visibility

It is important to note that the level of detail involved in endpoint analytics is much greater than network monitoring. Put plainly, endpoints can perform and see things that network monitoring never will. This is because endpoints deal with processes, storage and files, memory, as well as network traffic. **They are the ultimate destination of whatever is happening.** The number of data points available on any given endpoint can quickly be overwhelming.

Take, as an example, a remote access Trojan. On the network, you may be able to detect what IP addresses and ports are in use. If traffic is unencrypted, you may even be able to see what commands are being issued. Yet on the endpoint system, you can see all of this regardless of encryption, plus you can see what process(es) the Trojan is using, what user and privileges are being utilized, if files or registry keys are used for persistence, and much more.

It is much easier to monitor for unauthorized network connections and patterns of activity using network equipment. This is because they are purpose-built, mature, and there are a lot less of them to manage than there are endpoints. Even with only network data, there is a large visibility footprint—even if it is only high level.

On the flip side, endpoints can provide highly targeted visibility points. For example, you can look for unauthorized programs being launched, services being created, and other indicators of activity. This type of visibility is only offered on the endpoints themselves. A major drawback, though, is that management and scalability are much more difficult. There are more data points available, it is hard to keep every device configured properly, and a deeper understanding of the many endpoints is required.

Windows Logs

Multiple built-in log sources exist across Windows

- 2003/XP and earlier use Windows Event Log (EVT)
- 2008/Vista+ use Windows XML Event Log (EVTX)
- 2000+ supports Event Tracing for Windows (ETW)
 - Used to generate Event Trace Logs (ETL)
 - Commonly used for debugging or troubleshooting
 - High performance and often use memory buffers
 - Over 1,000+ trace logs in Windows 10/11

Windows logging has been around for a while. Therefore, it is no surprise that it has changed over time. As a historical comparison of the evolution of logs, consider these facts:

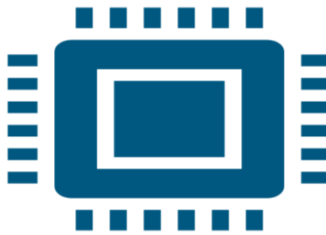
- Beginning in Windows NT 4.0, Microsoft introduced the Windows event log format of .EVT (Event Viewer Log).
- Then, in Windows Vista/Server 2008, this format was upgraded from EVT to Windows XML event log (EVTX). These are the two main log types for Windows systems.
- Microsoft also introduced event tracing for Windows, starting with Windows 2000. This provided means for advanced, in-depth logging typically used for debugging or troubleshooting.
- Over time, Microsoft has continued to increase the available default trace logs. For example, Windows Vista had over 400, yet Windows 10/11 have over 1,000.

Windows Events



Microsoft events are stored in a proprietary binary format.

- Requires Windows Event Viewer or special agent to read



The new format is still binary but is XML-based.

- Supports up to **32,766** bytes (syslog UDP is 1,024)
- Allows for custom parameters in the message section



Events are broken up by:

- **Channels:** A group of logs, such as Security or System
- **Event IDs:** Unique IDs to filter on the same channel

Windows events are stored in channels, which are simply groups of logs. The three most common are Application, Security, and System, but there are many more. Within these channels, events are given unique IDs to search and filter on. **Event IDs are not unique across channels.**

Prior to Vista, Windows events were stored in a proprietary format. This format effectively made it so that Windows Event Viewer was the only method to view the logs. This doesn't mean that vendors and the open-source community were not able to develop methods to read this binary format. It just means native support is limited to Windows Event Viewer and a backward-compatible PowerShell cmdlet called **Get-EventLog**.¹

However, pulling event data or user data at a field level is not possible with the old EVT format. For this, EVTX is needed, as well as a tool that can read it. The PowerShell cmdlet for EVTX format logs is **Get-WinEvent**.²

Windows events allow for logs up to 32 KB. However, the actual allowed size limit is 32,766 bytes.

1. <https://sec555.com/ehdt3>

2. <https://sec555.com/s97uh>

EVTX Log Sample

```

- <Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event">
- <System>
  <Provider Name="Microsoft-Windows-Security-Auditing" Guid="{54849625-5478-4994-A5BA-3E3B0328C30D}" />
  <EventID>4688</EventID>
  <Version>2</Version>
  <Level>0</Level>
  <Task>13312</Task>
  <Opcode>0</Opcode>
  <Keywords>0x8020000000000000</Keywords>
  <TimeCreated SystemTime="2016-12-21T05:16:22.907843100Z" />
  <EventRecordID>109809</EventRecordID>
  <Correlation />
  <Execution ProcessID="4" ThreadID="8276" />
  <Channel>Security</Channel>
  <Computer>CIT01LPT.test.int</Computer>
  <Security />
</System>
- <EventData>
  <Data Name="SubjectUserSid">S-1-5-21-2635542286-2942777934-2742232658-1105</Data>
  <Data Name="SubjectUserName">jhenderson</Data>
  <Data Name="SubjectDomainName">TEST</Data>
  <Data Name="SubjectLogonId">0x1b67d0</Data>
  <Data Name="NewProcessId">0x...</Data>
  <Data Name="NewProcessName">C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data>
  <Data Name="TokenElevationType">%%1938</Data>
  <Data Name="ProcessId">0xf38</Data>
  <Data Name="CommandLine">C:\Windows\system32\WindowsPowerShell\v1.0\PowerShell.exe -executionpolicy bypass -file "C:\sec555.ps1"</Data>
  <Data Name="TargetUserSid">S-1-0-0</Data>
  <Data Name="TargetUserName"></Data>
  <Data Name="TargetDomainName"></Data>
  <Data Name="TargetLogonId">0x0</Data>
</EventData>

```

Main fields

Custom data and fields

Field name

Field value

Starting with Windows Vista, Windows events are XML-based. This means fields are stored in parameters rather than having one large message with a bunch of data. This makes event fields specific and readable. The most common type of Windows logs today is the Windows XML event logs. These are log files stored with the file extension of .evtx. While the name states it is XML, under the hood, it is still a binary file. The XML comes from the Windows EvtRender API, which converts between binary and XML when the file is read.

The change from EVT to EVTX is significant. By adding XML capabilities to events, it allows Windows a granular layout by allowing custom fields, also referred to as properties, in the EventData and UserData sections. This means additional filtering and searching capabilities—assuming logs that are collected parse out these fields. Note that while there are tools available to convert from EVT to EVTX, the conversion does not add extra data or custom fields. Instead, these tools convert files so that they can be read by more modern applications. One example is the Windows built-in command line tool called wevutil or even Windows Event Viewer, which can load an EVT file and then save it as EVTX.

The image shows an example of a Windows log taken from a Windows 11 system. The top breaks out the System Properties section. These are the main fields for every event. The bottom shows the EventData section. This is where custom fields such as NewProcessName are stored.

To view the XML of a Windows login to a Vista or later operating system, first open Windows Event Viewer and then open the log you wish to analyze. The log will have a tab for "General" and a tab for "Details". Click on "Details" and then switch from "Friendly View" to "XML View".

XML means that the parameters are defined and extractable with the right agent. Unfortunately, many native SIEM agents will only pull fields that they want to collect. This means you may only have 50–100 Windows fields when, in fact, there can be more than a thousand in a large environment.

Field Structure

XML contains mainstream fields

- EventData and UserData are further structured into other XML fields.
- XML does not require traditional parsing.
 - Older collection methods (syslog) may cause visibility loss.
 - For example, logs from Windows devices are likely to have over 500 fields.

Use of XML structure provides robust logging capabilities

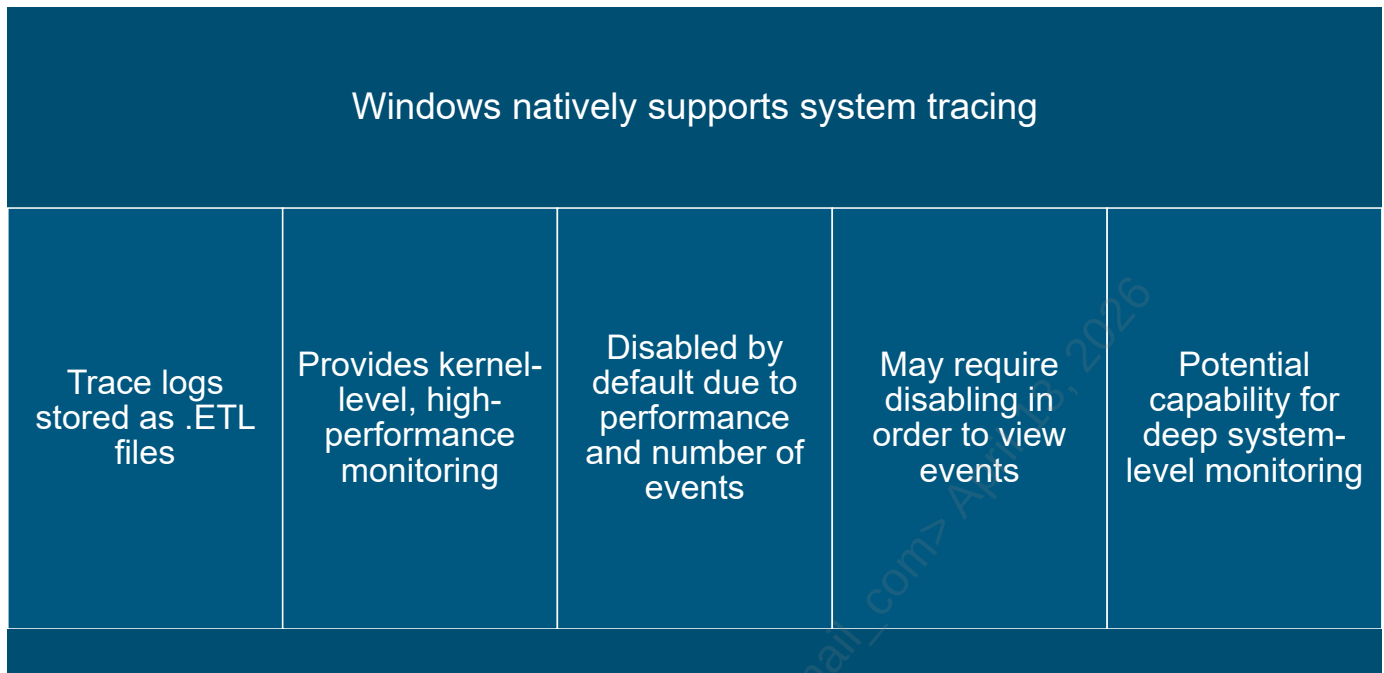
Microsoft made a smart decision when they decided to add the XML schema capabilities in EVTX files. This effectively allows intelligent, structured logs that can contain more data, yet have more capabilities as far as precision and filtering are concerned.

Because EVTX events use XML, an infinite number of fields can be created. This is important to note as traditional log agents often will parse out the most critical fields. This can result in having approximately 50 to 100 Windows fields to search on. While this simplifies searching and filtering, it also is blinding, as there are easily over 500 fields, and they may be necessary for certain techniques to work.

It is important to know if this is a limitation of your environment. If so, new events with new fields will not be usable until custom parsers are built by your vendor. This typically takes a considerable amount of time, unless you are willing to pay for immediate professional services.

Oddly enough, since fields are stored in XML, it is possible to auto parse out all new fields. Yet some products do not support this (often if they are transporting the log over syslog).

Event Tracing for Windows (ETW)



Microsoft implemented trace logs to provide highly detailed logs that can maintain high-volume logging. These logs are managed by the event tracing for Windows subsystem and are stored as an .etl file extension. Typically, these are recommended to be enabled only when needed for troubleshooting and then disabled afterward due to performance overhead. When viewing trace logs, **it is sometimes required first to disable tracing prior to being able to view the logs**. Note that turning tracing back on clears the logs.

Microsoft is investing more time into providing additional trace logging as well as minimizing the performance impact. For example, starting in Server 2012 R2 and later, Microsoft supports Windows DNS Server debug logging at little to no noticeable impact. Should a Windows DNS Server be running commodity hardware and be handling around 100,000 queries per second, the performance impact is expected to be only 5%.

While the use cases for regularly collecting and analyzing ETL files are rare, it should be noted that many trace logs exist that may be of use.

Reference:

- <https://sec555.com/qzn2f>

Channel

"A channel is basically a sink that collects events."¹

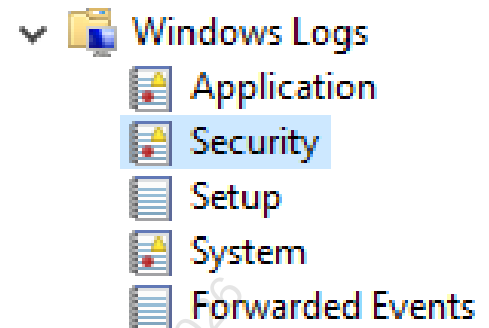
Channels act as receivers of specific events

- Act as a high-level category

Four categories of channels

- Admin
- Operational
- Analytic
- Debug

Disabled by default



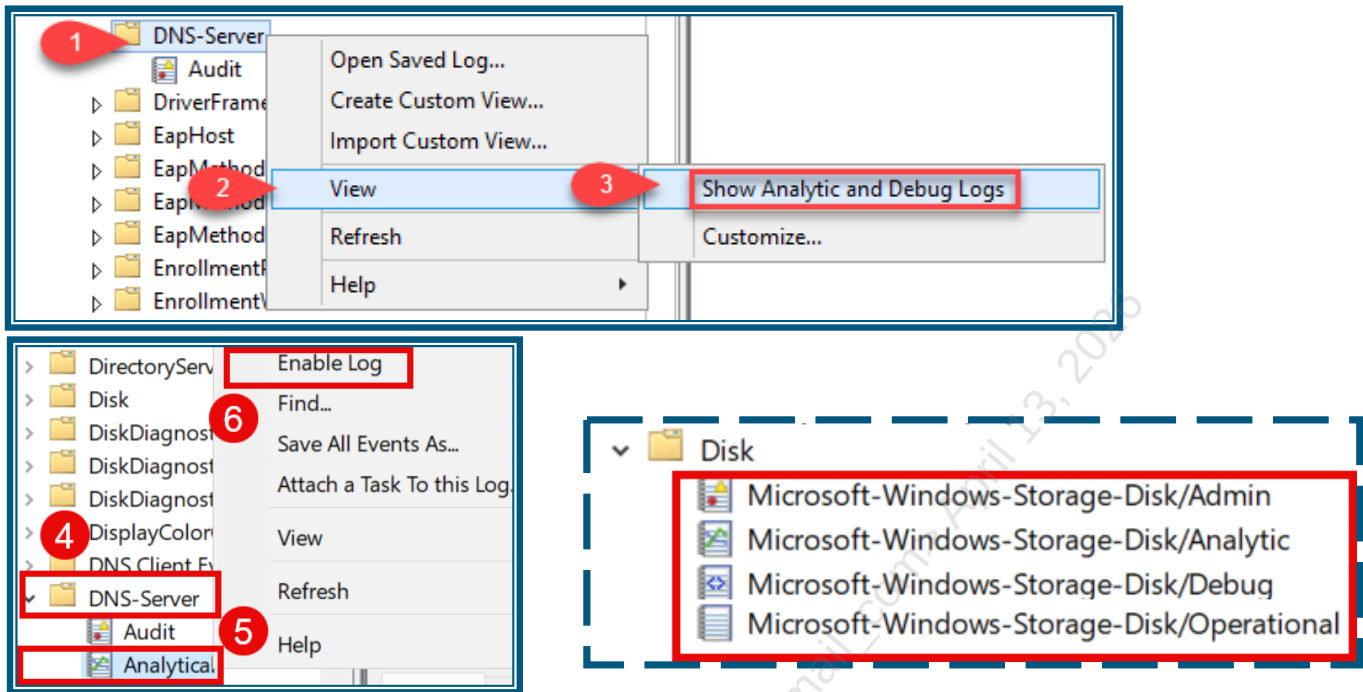
When people think of Windows logs, they often envision the Windows Event Viewer. The most common places folks look are the **Application**, **Security**, and **System** logs. These are examples of channels. They are a location for created logs to go to. This is what is referred to as a "sink."²

Under the hood, there are four channel types: **admin**, **operational**, **analytic**, and **debug**. The purpose of these channels is to describe the type of logs they contain:³

- **Admin type channels:** Relate to end users, administrators, and support personnel. Events written to the Admin channels are expected to have a well-established solution that can be followed. Mostly useful for investigating application crashes, as it logs critical errors and warnings. SIEM tools often use this.
- **Operational type channels:** Relate to analyzing and diagnosing a problem or occurrence. They can trigger actions or tools based on the problem or activity. Example: Adding a printer. Mostly used for service monitoring.
- **Analytic type channels (disabled by default):** Relate to events that are published in high volume. They mostly relate to program operation and are indicative of issues that cannot be solved by a user action. Mostly used for detailed DNS event tracing.
- **Debug type channels (disabled by default):** They are used solely by developers to identify problems for debugging.

1. <https://sec555.com/zopwi>
2. <https://sec555.com/zopwi>
3. <https://sec555.com/tqvik>

Analytic and Debug Logs



This page shows you how to enable debug logs. First, go to **Start Menu > Event Viewer**. After the window opens up, follow the steps below:

1. Navigate to **Application and Services Logs > Microsoft > Windows > DNS Server** (you can choose another log source if you desire).
2. Right-click on the selected log source and select **View**.
3. Then click on **Show Analytic and Debug Logs**.
4. After that happens, notice you now get a new option of Analytical Logs.
5. Right-click on **Analytical Logs** and select **Enable Log**.

If you then take a look at another log source, like the Disk, you see that all four channel types appear underneath it.

Basic Audit Policy

Local Security Policy

File Action View Help

Security Settings

- Account Policies
- Local Policies
 - Audit Policy**
 - User Rights Assignment
 - Security Options
- Windows Defender Firewall with Advanced Security
- Network List Manager Policies
- Public Key Policies
- Software Restriction Policies
- Application Control Policies

Policy	Security Setting
Audit account logon events	No auditing
Audit account management	No auditing
Audit directory service access	No auditing
Audit logon events	No auditing
Audit object access	No auditing
Audit policy change	No auditing
Audit privilege use	No auditing
Audit process tracking	No auditing
Audit system events	No auditing

Audit account logon events Properties

Local Security Setting Explain

Audit account logon events

Audit these attempts:

☐ Success

☐ Failure

OK Cancel

Recommendations for all these settings are provided by the Center for Internet Security.

You can configure any options you want from an audit policy point of view by going to the Start menu and opening the Local Security Policy manager. As you see in the image on the left, when the window comes up, go to **Local Policies > Audit Policy** and that will show you all the system options for managing that device. Notice that the default values when the system comes up are set to **No auditing** across the board. If you select one item of interest (e.g., **Audit account logon events**), you just select what you require to be logged (e.g., **Success**, **Failure**, or **Both**) and then click **OK**.

If you want to make changes domain-wide, you can make similar changes to a domain controller via the Group Policy GPO settings and then save those to reflect all machines in that Organizational Unit, Domain, or other group the policy applies to. Most organizations struggle with what to log for¹ or what to initially switch on.²

An effective audit policy needs to be enabled. One of the quickest ways to find recommendations on which audit policies need enabling and why is to read the CIS benchmarks based on Windows.³ Every setting includes a detailed breakdown of what it does, why it is important, and what the recommendation is for certain settings. This is great for understanding what the impact of a specific setting is. In fact, the details provided can be considerably better than the description Microsoft provides under Group Policy Management. Some of the additional information provided includes the specific event IDs generated, the vulnerability and impact of enabling or disabling the policy, and suggested countermeasures.

The recommendations provided are generally acceptable. However, consideration should be given to your environment and collective settings. For example, there are some events for which you may want to record both successful attempts and failures, yet the recommendation is for success only.

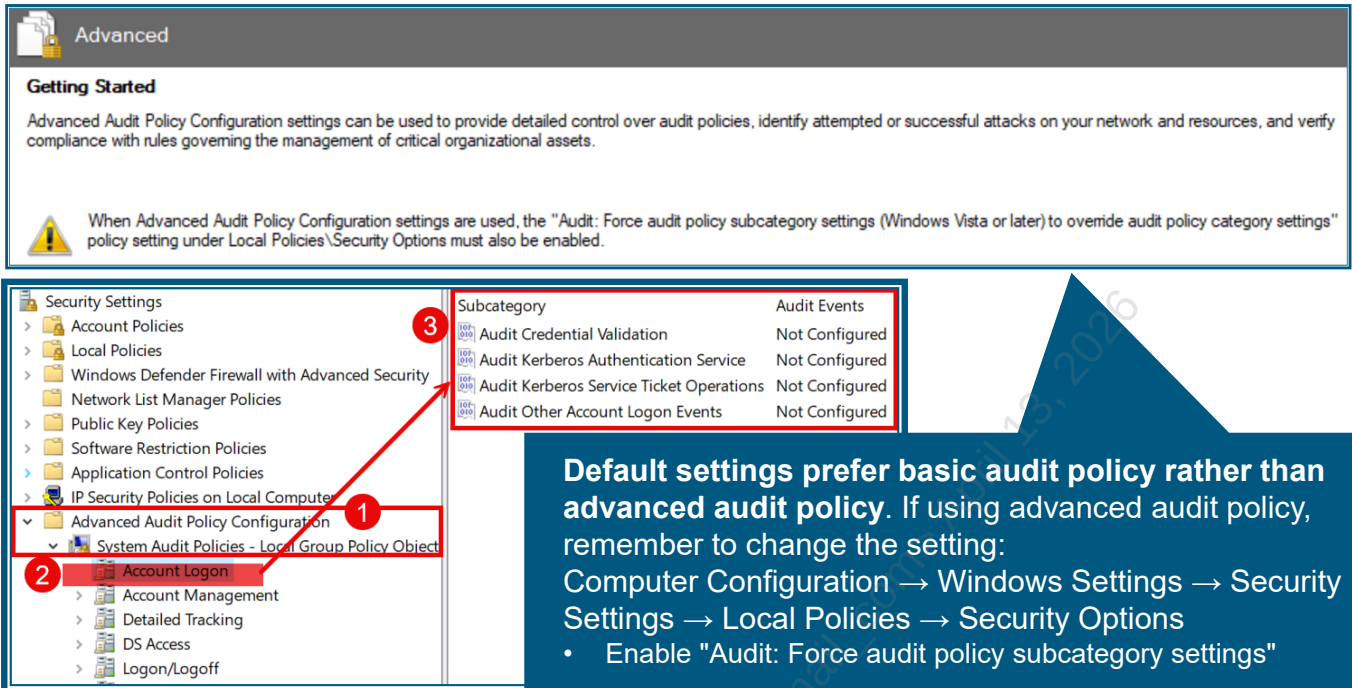
Logoff events are one example where the recommendation is to only record successful logoff attempts. The reasoning is that the setting provides an attacker with the ability to generate millions of failure events. If the "Audit: Shut down system immediately if unable to log security audits" is enabled, this can grant attackers the ability to perform a denial-of-service attack. If this setting is off,

you instead run the risk of logs stopping once full, or of logs constantly being rotated out if "**Overwrite events as needed (oldest events first)**" is set. "Overwrite events as needed" is the default behavior.

An additional resource with amazing recommendations is the malware archeology cheat sheet collection, which can greatly help and is highly recommended.⁴

1. <https://sec555.com/ytiaac>
2. <https://sec555.com/vjz75>
3. <https://sec555.com/rljby>
4. <https://sec555.com/2obgd>

Advanced Audit Policy



Advanced

Getting Started

Advanced Audit Policy Configuration settings can be used to provide detailed control over audit policies, identify attempted or successful attacks on your network and resources, and verify compliance with rules governing the management of critical organizational assets.

When Advanced Audit Policy Configuration settings are used, the "Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings" policy setting under Local Policies\Security Options must also be enabled.

Security Settings

- Account Policies
- Local Policies
- Windows Defender Firewall with Advanced Security
- Network List Manager Policies
- Public Key Policies
- Software Restriction Policies
- Application Control Policies
- IP Security Policies on Local Computer
- Advanced Audit Policy Configuration
 - System Audit Policies - Local Group Policy Object
 - Account Logon
 - Account Management
 - Detailed Tracking
 - DS Access
 - Logon/Logoff

Subcategory	Audit Events
Audit Credential Validation	Not Configured
Audit Kerberos Authentication Service	Not Configured
Audit Kerberos Service Ticket Operations	Not Configured
Audit Other Account Logon Events	Not Configured

Default settings prefer basic audit policy rather than advanced audit policy. If using advanced audit policy, remember to change the setting:
 Computer Configuration → Windows Settings → Security Settings → Local Policies → Security Options

- Enable "Audit: Force audit policy subcategory settings"

When using the advanced audit policy, it is important to know that the default behavior is for basic audit policy settings to override advanced audit policy settings. Fortunately, Windows warns you of this when you visit the advanced audit policy (as per the top image).

It also shows you how to change this behavior by setting "**Audit: Force audit policy subcategory settings**" to Enabled.

Keep in mind that using advanced audit will also clear the simple policy settings.

Advanced audit policies give administrators far more granular control of the environment. They have almost 60 different configurational options in comparison to the 9 that exist in basic audit policy configuration. The bottom image shows 4 different settings available under **Account Logon**, in comparison to the single setting present in the basic audit policy.

TechNet has an article that contains some great videos explaining the configuration steps for each advanced audit policy.¹

1. <https://sec555.com/951yi>

Account Management

Used to track changes to groups, users, and computers

Needed to monitor key groups for modifications

Such as new members added to Domain Admins

Powerful when combined with change control system

Subcategory

	Audit Application Group Management
	Audit Computer Account Management
	Audit Distribution Group Management
	Audit Other Account Management Events
	Audit Security Group Management
	Audit User Account Management

Audit Events

Not Configured
Not Configured
Not Configured
Success and Failure
Success and Failure
Success and Failure

These types of events are useful to keep track of unauthorized changes. For example, it is a best practice to monitor additions to any sensitive security group, such as **Domain Admins** and **Enterprise Admins**. While out of the gate, something such as monitoring Domain Admin additions are high value, more can be achieved if logs from a change control system can be used to verify security group changes.

Consider this: If employees are added to a group called "Sensitive Data access", hopefully, it required someone to authorize the addition. If the authorization record can be used, it can verify that all modifications came with prior consent. Then, should an adversary add an account to this group, it would be red-flagged as no prior authorization was given.

There are six possible subcategories:

- **Audit Application Group Management:** Monitors changes to application groups (e.g., group creation/change/deletion and member addition/removal from group).
- **Audit Computer Account Management:** Monitors changes to computer accounts (account creation/change/deletion).
- **Audit Distribution Group Management:** Monitors changes to Active Directory distribution groups.
- **Audit Other Account Management Events:** Monitors special changes such as a password hash of a user account being accessed or changes to the password policy or lockout policy.
- **Audit Security Group Management:** Monitors changes to standard security groups (e.g., group creation/change/deletion and member addition/removal from security group).
- **Audit User Account Management:** Monitors changes to user accounts (e.g., user account creation/change/deletion/rename/disablement/enablement/lockout/unlock).

Reference:

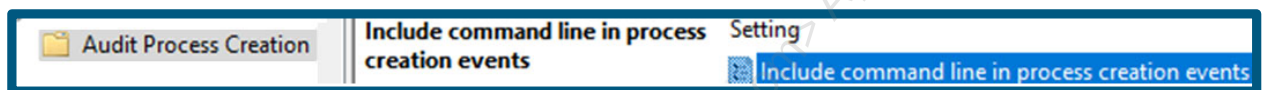
- <https://sec555.com/28q5x>

Detailed Tracking

Can generate vast amounts of logs. Therefore, proceed with caution!

- Has built-in monitoring of processes
- Default behavior does not include command-line logging

Subcategory	Audit Events
 Audit DPAPI Activity	Not Configured
 Audit PNP Activity	Not Configured
 Audit Process Creation	Not Configured
 Audit Process Termination	Not Configured
 Audit RPC Events	Not Configured
 Audit Token Right Adjusted	Not Configured



Be careful when enabling logs under detailed tracking as they may have performance implications.

However, there are a few critical events analysts should regularly monitor. One is process creations, which involves logging all new processes being launched. Windows can natively log this if **Audit Process Creation is enabled**. These types of logs can also be collected from application control suites. It is important to note that the default behavior of Audit Process Creation is to generate an event on each new process being launched, but that it does not include any command-line parameters involved with the process starting.

To enable auditing process creation and include command line parameters, enable "**Audit Process Creation**" but also enable the policy "**Include command line in process creation events**" located at Computer Configuration > Administrative Templates > System > Audit Process Creation.

Another thing worth tracking is new devices being plugged in, such as plug-and-play devices (this is under **Audit PNP Activity**). Starting with Windows 10 and Server 2016, Microsoft allows logging of plug-and-play devices.

Logon/Logoff

Many attacks involve credential theft and reuse.

- Therefore, tracking logons is critical.

Too many failed logon attempts show brute forcing, misconfigurations, or password spraying.

Too many successful logons may mean the attacker has free reign.

Subcategory

 Audit Account Lockout
 Audit User / Device Claims
 Audit Group Membership
 Audit IPsec Extended Mode
 Audit IPsec Main Mode
 Audit IPsec Quick Mode
 Audit Logoff
 Audit Logon
 Audit Network Policy Server
 Audit Other Logon/Logoff Events
 Audit Special Logon

Audit Events

Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured
Not Configured

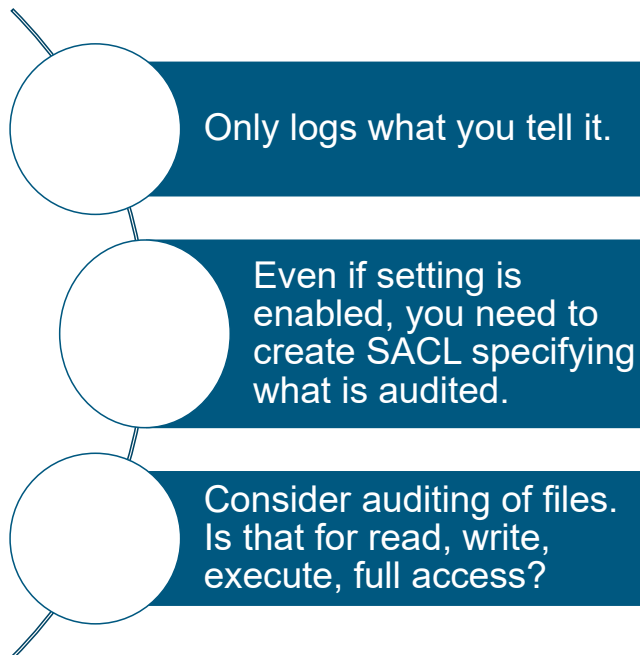
Monitoring logons is a critical component in continuous monitoring. At the most basic level, organizations should look for repetitive logon failures or successes to find brute force attempts or compromised accounts spreading internally. Once this and other high-value quick wins are implemented, then organizations should advance their monitoring to include things such as baselining user activity by monitoring logons, logoffs, times of occurrence, and systems typically logged into per user. This is an area where machine learning has greatly improved and could be of value. Some useful subcategories are:

- **Audit Account Lockout:** This records both successful and failed logons when a logon request is made against a locked-out account.
- **Audit Logoff:** This records both successful and failed logoff attempts. This is useful for context. For instance, was user JD still logged on at the time of an incident or not?
- **Audit Logon:** This records successful and failed logon attempts. Failed logon attempts are useful for catching brute force attempts, and successful logon attempts are great for profiling user activity or finding credential theft. This may manifest as credentials being used during abnormal hours or creating large numbers of sessions.
- **Audit Network Policy Server:** Allows you to audit RADIUS and Network Access Protection user access requests. An audit event can be generated with each NAP request (e.g., grant, deny, discard, quarantine, lock, unlock).
- **Audit Other Logon/Logoff Events:** This records successful and failed logon/logoff events for things such as terminal services, locking and unlocking a workstation, and can detect a Kerberos replay attack.

Reference:

- <https://sec555.com/gr32x>

Object Access



Subcategory	Audit Events
Audit Application Generated	Not Configured
Audit Certification Services	Not Configured
Audit Detailed File Share	Not Configured
Audit File Share	Not Configured
Audit File System	Not Configured
Audit Filtering Platform Connection	Not Configured
Audit Filtering Platform Packet Drop	Not Configured
Audit Handle Manipulation	Not Configured
Audit Kernel Object	Not Configured
Audit Other Object Access Events	Not Configured
Audit Registry	Not Configured
Audit Removable Storage	Not Configured
Audit SAM	Not Configured
Audit Central Access Policy Staging	Not Configured

Object access controls log quite a few things. If you wish to audit files, registry keys, or network shares, you must enable the auditing capability first. For example, after turning on Audit File System, it grants Windows the ability to audit things like files or folders being accessed, but only if a SACL is placed on them telling what to audit. In the past, there was a wide misconception that enabling this would generate an event for every file accessed. That SACL will need to specify if an object will be audited for failure, success, or both, and then what action upon the object will be taken. For example, is it read, write, execute, or any access? You can be really granular and also designate specific users and groups, since you have total auditing control. So, this approach can have minimal impact in your log volume but still provide quite beneficial value.

This audit policy also controls things such as Windows Firewall logging to a log channel and file access on removable drives.

In Active Directory, many things are considered objects (users, groups, files, folders, registry keys, etc.). This policy even controls certificate-related events.

auditpol.exe

Non-domain joined systems can be configured with auditpol.exe.

- **List all policy settings:**

```
auditpol /get /category:*
```

- **Set policy settings:**

```
auditpol /set /subcategory:
"file system" /success:enable
/failure:enable
```

- **List all users with a defined audit policy:**

```
auditpol /list /user
```

```
auditpol /get /category:*
```

System audit policy	Setting
Category/Subcategory	
System	
Security System Extension	Success and Failure
System Integrity	Success and Failure
IPsec Driver	No Auditing
Other System Events	Success and Failure
Security State Change	No Auditing
Logon/Logoff	
Logon	Success and Failure
Logoff	Success and Failure
Account Lockout	Success and Failure

Not all systems are typically joined to a domain. Some are intended not to be joined for security purposes (such as DMZ systems). In order to effectively maintain log settings, audit policies need to be pushed out. Since group policy is not available, an alternative method, such as using auditpol.exe, is needed.¹ This command-line utility can retrieve and set audit policy settings. Because of this, it is fairly easy to automate and push out a standardized audit policy for non-domain-joined systems. Open a PowerShell window and use it.

If you need to check the policy settings for a given category (like the Account Management category), you can use something like:

```
Auditpol /get /category:"Account Management"
```

Category/Subcategory	Setting
Account Management	
Computer Account Management	Success
Security Group Management	Success
Distribution Group Management	No Auditing
Application Group Management	No Auditing
Other Account Management Events	No Auditing
User Account Management	Success

Using `auditpol /get /category:*` shows you the values for all the policies. There are other options for setting the audit policy, backing it up, restoring, and more, included in the reference below.

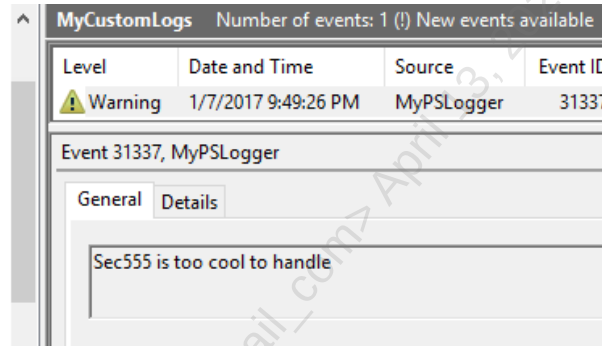
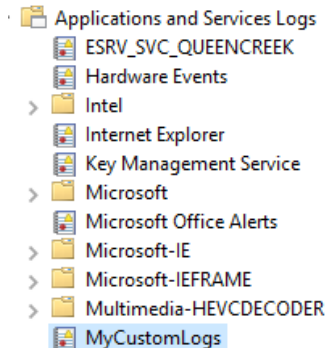
1. <https://sec555.com/7jmvi>

Custom Logging with PowerShell

- PowerShell makes writing custom logs easy.
 - Simply use Write-EventLog to generate a custom log.
 - If you want a custom channel, first, use New-EventLog.

```
New-EventLog -LogName MyCustomLogs -Source "MyPSLogger"
```

```
Write-EventLog -LogName MyCustomLogs -Source "MyPSLogger" -EventId 31337 -EntryType Warning -Message "Sec555 is too cool to handle"
```



This page shows the creation of a new event log channel called MyCustomLogs, as well as a single log generated.

One of the easiest ways to generate a custom log is to use PowerShell. PowerShell, in itself, is powerful and can do just about anything. Taking that functionality and adding a simple function to create logs opens the door for all sorts of opportunities. Let's say, for instance, you have a program that only outputs logs to a file, yet the log agent or collection method you have at your disposal does not handle files well. To resolve this, you could write a PowerShell script that monitors the log file and turns it into Windows event logs.

Another example would be if you had a recurring task that you would like to log. Instead of outputting to a log file, simply dump data into an event log channel. Since PowerShell also has the capability to create a new channel as well as specify custom Event IDs, logs become easy to read and sort.

If you are planning on using a custom event log, you first must create it using **New-EventLog**. Failure to do so will result in an error and no log being generated when invoking Write-EventLog.

Sysmon

Free download from
Windows Sysinternals

- Written by Mark Russinovich and Thomas Garnier
- Runs as a Windows system service and device driver
- Provides process hashes and parent processes for analysis

Monitors

- Processes
- Network connections
- Driver and DLL loading
- Registry key modification
- Modifications of file creation times
- Process access
- WMI Monitoring

Sysmon was designed to generate logs of interesting activity commonly associated with malicious or anomalous activity. By themselves, the logs are useless; however, when looked at by an analyst, they can be extremely powerful. For example, Sysmon can monitor all processes being launched, along with the parent process that spawned the new process, and can provide a hash of the new process. On top of this, the log also includes any command-line parameters along with other useful information. Collecting this data centrally and evaluating it lets you know exactly what is taking place in your organization.

Similar to NetFlow, Sysmon can optionally log all network connections being made. However, unlike NetFlow, Sysmon records the process that either made the network request or is receiving the network request. This level of detail is great for troubleshooting, incident response, and establishing firewall rules for a default deny policy.

Under the hood, Sysmon utilizes built-in Windows API calls and ETW tracing to generate logs. This allows for an acceptable performance overhead. Keep in mind that it will generate a lot of logs. However, it may not make sense to try and collect all of these logs centrally unless major filtering is done. Sysmon allows fine-grained filtering on what to log. For example, you can create a configuration rule to log all network connections unless they come from the process iexplore.exe.

Sysmon is built for both desktops and servers. It can be deployed to Windows 7 and newer, as well as Windows Server 2012 and newer operating systems. Mark Russinovich did a great presentation on malware hunting with Sysinternals tools, which is worth watching.¹

1. <https://sec555.com/id3s4>

Sysmon Event Log Example

Event 1, Sysmon

General	Details
Process Create: UtcTime: 2017-01-02 16:45:55.172 ProcessGuid: {6a32f033-83c3-586a-0000-0010aa59c4ad} ProcessId: 3048 Image: C:\Windows\System32\timeout.exe CommandLine: timeout 1 CurrentDirectory: C:\Program Files\Intel\SUR\WILLAMETTE\ User: NT AUTHORITY\SYSTEM LogonGuid: {6a32f033-05f4-585e-0000-0020e7030000} LogonId: 0x3E7 TerminalSessionId: 0 IntegrityLevel: System Hashes: SHA1=B55BE2059C5FEE93C803CC853360766707BE8AF4 ParentProcessGuid: {6a32f033-83c0-586a-0000-00105a17c4ad} ParentProcessId: 19216 ParentImage: C:\Windows\System32\cmd.exe ParentCommandLine: C:\Windows\system32\cmd.exe /c ""C:\Program Files\Intel\SUR\WILLAMETTE\svc_install.bat" S > log_start.txt 2> &1"	
Log Name:	Microsoft-Windows-Sysmon/Operational
Source:	Sysmon
Event ID:	1
Level:	Information
User:	SYSTEM
OpCode:	Info
More Information:	Event Log Online Help

In this slide, the parent process of C:\Windows\System32\cmd.exe is running the batch script at C:\Program Files\Intel\SUR\WILLAMETTE\svc_install.bat. This is being executed by the NT AUTHORITY\SYSTEM account, and a hash of the executable being spawned (C:\Windows\System32\timeout.exe) is provided.

Collecting this level of detail allows for studying and understanding what is happening in your environment. For example, if you received this log, would you know what WILLAMETTE is? It sounds odd; possibly it is malicious, but it is in the C:\Program Files\Intel folder, so maybe it is not. Investigating this would reveal that it is an Intel service for providing power savings and is installed as part of the Intel Driver Update Utility.

Sysmon Configuration

Granular logging available

- Uses XML config
- Can include or exclude based on
 - Path
 - Process/Image
 - Digital Signature
 - Integrity Level

```
<Sysmon schemaversion="3.20">
  <!-- Capture all hashes -->
  <HashAlgorithms>sha1,md5</HashAlgorithms>
  <CheckRevocation></CheckRevocation>
  <EventFiltering>
    <!-- Log all drivers except if the signature -->
    <!-- contains Microsoft or Windows -->
    <DriverLoad onmatch="exclude">
      <Signature condition="contains">microsoft</Signature>
      <Signature condition="contains">windows</Signature>
    </DriverLoad>
    <!-- Log all process creations unless they have a -->
    <!-- integrity level of medium -->
    <ProcessCreate onmatch="exclude">
      <IntegrityLevel>Medium</IntegrityLevel>
    </ProcessCreate>
    <!-- Do not log process termination -->
    <ProcessTerminate onmatch="include" />
    <!-- Log network connection if the process isn't InternetExplorer -->
    <NetworkConnect onmatch="exclude">
      <Image condition="end with">iexplore.exe</Image>
    </NetworkConnect>
  </EventFiltering>
</Sysmon>
```

Configuring Sysmon to log or not log something can be done with basic command line switches or by using the advanced configuration. The advanced configuration requires the use of XML configuration files. When deploying Sysmon, it is usually recommended to use the advanced configuration files, as the level of granularity is often used to include or exclude certain things.

The following commands can be used to find more information on building a Sysmon configuration file:

```
sysmon.exe --help
sysmon.exe -? config
```

The picture on this page also shows an example of a functional XML configuration file used for Sysmon version 5. The main thing to pay attention to is the use of `onmatch="include"` and `onmatch="exclude"`. In the case of `onmatch="exclude"`, all items are logged except the ones specifically excluded. So, a statement reading `<ProcessCreate onmatch="exclude"></ProcessCreate>` would log all process creations as no exclusions are specified. In contrast, `onmatch="include"` only includes items specified. The configuration of `<ProcessCreate onmatch="include"></ProcessCreate>` would log nothing because nothing is specified to be included for logging.

If attempting to build a new Sysmon configuration, consider using a community-provided configuration as a starting point.

References:

- <https://sec555.com/yezoc>
- <https://sec555.com/qxsih>

Sysmon Modular

- Project by Olaf Hartong
 - Adds MITRE tags
 - Modular design
 - Easy to support

```
<Sysmon schemaversion="4.30">
  <EventFiltering>
    <RuleGroup name="" groupRelation="or">
      <ProcessCreate onmatch="include">
        <Rule name="Eventviewer Bypass UAC" groupRelation="and">
          <ParentImage name="technique_id=T1548.002,technique_name=Bypass User Access Control"
            condition="image">eventvwr.exe</ParentImage><!-- eventvwr.exe launching child process other than
            mmc.exe -->
          <Image condition="is not">c:\windows\system32\mmc.exe</Image>
```

5_process_ended	schema update to 4.30	2 months ago
6_driver_loaded_into_kernel	schema update to 4.30	2 months ago
7_image_load	ATT&CK Sub-Techniques retagging	5 days ago
8_create_remote_thread	added clipboard monitor	2 months ago
9_raw_access_read	schema update to 4.30	2 months ago
attack_matrix	minor change	15 months ago
.gitignore	revocation check added	2 years ago
Merge-SysmonXml.ps1	Updates for v11	2 months ago
README.md	Small tweak	4 days ago
license.md	Create license.md	2 years ago
sysmonconfig.xml	Updated after successful CICD run	3 days ago

Olaf Hartong created a GitHub project called Sysmon Modular.¹ Sysmon Modular provides a breakout of each Sysmon functionality into subfolders. The subfolders contain inclusion and exclusion configuration files, which also include MITRE technique information. Having multiple configuration files that are based on each capability of Sysmon simplifies the organization to support and maintain.

Deploying Sysmon Modular requires running a PowerShell command that combines all the individual configuration files into a single XML configuration. Installing Sysmon or updating it requires running sysmon.exe or sysmon64.exe and referencing the generated XML configuration file.

This is a great tool that you will have an opportunity to use in detail during an upcoming lab.

1. <https://sec555.com/fqoeg>

Summary

Windows logs come in multiple formats, such as EVT, EVTX, and ETL. Current operating systems use EVTX, which is XML-based. This allows for unlimited fields and granular logs to be present.

What gets logged is controlled by basic and advanced audit policies, which you should configure with best industry practices in mind, such as using CIS templates. Custom logs can be created by software or scripts, such as using Sysmon or PowerShell.

This area is intentionally left blank.

Investigating Windows Logs

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines

- SMTP
- DNS
 - ☐ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ☐ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ☐ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ☐ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will gain an understanding of the importance of using firewalls as a detection tool (contradicting their preventive nature) and the significance of connection logging.

This area is intentionally left blank.

The Art of Logging

NIST defines a log in SP 800-92 (Guide to Computer Security Log Management) as *"a record of the events occurring within an organization's system and networks."*¹

Linux auth.log sample

```
Dec 29 14:36:00 slingshot sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1001)
```

1

2

3

4

1. Timestamp (**Dec 29 14:36:00**)
2. Source system (**slingshot**)
3. Source process (**sudo**)
4. Log message (**pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1001)**)

NIST SP 800-92 states that any record of events in an organizational system and network can be deemed as a log. Let's look at an example of the Linux auth.log, which is used to log user activities such as logins and logouts. The following fields are available:

- **Timestamp** (Dec 29 14:36:00): When did an event occur?
- **Source system** (slingshot): What system did the event take place at?
- **Source process** (sudo): Which process was used?
- **Log message** (pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1001): The actual log entry being provided.

In the above example, the user who has a uid of 1001 (that is actually user sec555) logged on to the system and used sudo to elevate privileges to root.

At a minimum, you will encounter most logs having the following fields present:

- Time
- Source system
- Log message

1. <https://sec555.com/b1qnr>

Linux Logs

Syslog is commonly used in Linux/Unix logging

- The default behavior is to listen using a local socket

Multiple syslog daemons exist

- Default log location is /var/log/
- Log level specified by severity in daemon configuration
- Daemon can be configured to accept and send logs

In comparison to Windows logging, Linux logging is substantially different. This adds to the challenge of system administration, as different platforms require different administration. There are multiple flavors of syslog in use today.

When you hear syslog, you could be thinking about various things, such as:

- **Syslog format:** A standard way of formatting log messages.
- **Syslog daemon:** The system process that writes logs to disk (such as syslogd, syslog-ng, or rsyslog).
- **Syslog protocol:** Sending traffic across a network to be collected to a syslog server.
- **Syslog log file:** A file under /var/log/syslog, which is the default destination for any system messages generated by the syslog daemon.

Common Log Files

/var/log/messages: Global messages (general activity)

/var/log/auth.log: Authentication-related logs (successful and failed logins)

/var/log/boot.log: Boot time events

/var/log/daemon.log: Background process events

/var/log/kern.log: Kernel messages (often used for troubleshooting)

/var/log/cron.log: Events related to scheduled tasks

/var/log/secure: Events related to su or sudo access

/var/log/syslog: Global system log, includes kernel, daemon, other messages

This page lists some of the more common log files found on Linux operating systems.

Note that some of these log files are distribution-specific. But this list is provided as a reference point to the most common log files for your attention, irrespective of focusing on specific distribution differences.

Where Windows uses channels, Linux uses filenames to categorize events. Linux applications also will often log to **/var/log** but do so in a subfolder. Some files of interest are:

- **/var/log/maillog or var/log/mail.log:** If you have a mail server configured, then logs like smtpd or sendmail would end up in this log.
- **/var/log/secure:** Contains authentication failures SSH logins/logouts and failed passwords.
- **/var/log/dmesg:** Display and device driver information (easy to view using dmesg).
- **/var/log/btmp:** Contains all failed login attempts.
- **/var/run/utmp:** Contains information about currently logged-in users, including which terminals are used, system events, current status of the system, and system boot time.
- **/var/log/wtmp:** Keeps a record of login/logout (historical information of utmp).
- **/var/log/lastlog:** Keeps user last login information (view using the lastlog command).

Note: utmp, wtmp, and btmp are binary files, and reading them with a standard command like cat won't properly display content. Instead, use last, lastb, or utmpdump.

Reference:

- <https://sec555.com/1qvc0>

Syslog

Some systems support TCP, but not so common

RFCs support TLS encryption

- Yet, most systems only support syslog over UDP without encryption.

RFC	Publication	Title	Notes
3164	2001	BSD syslog	First syslog RFC
3195	2001	Reliable delivery for syslog	Mappings of the syslog protocol to TCP connections
5424	2009	The syslog protocol	Update which makes RFC 3164 obsolete
5425	2009	Transport Layer Security (TLS) Transport Mapping for Syslog	Use of TLS for securing connections for syslog message transportation
5848	2010	Signed Syslog Messages	Add origin authentication, message integrity, replay resistance and missing message detection
9662	2024	Updates to the Cipher Suites in Secure Syslog	Updates the cipher suites required by syslog and described in RFC 5425 and 6012

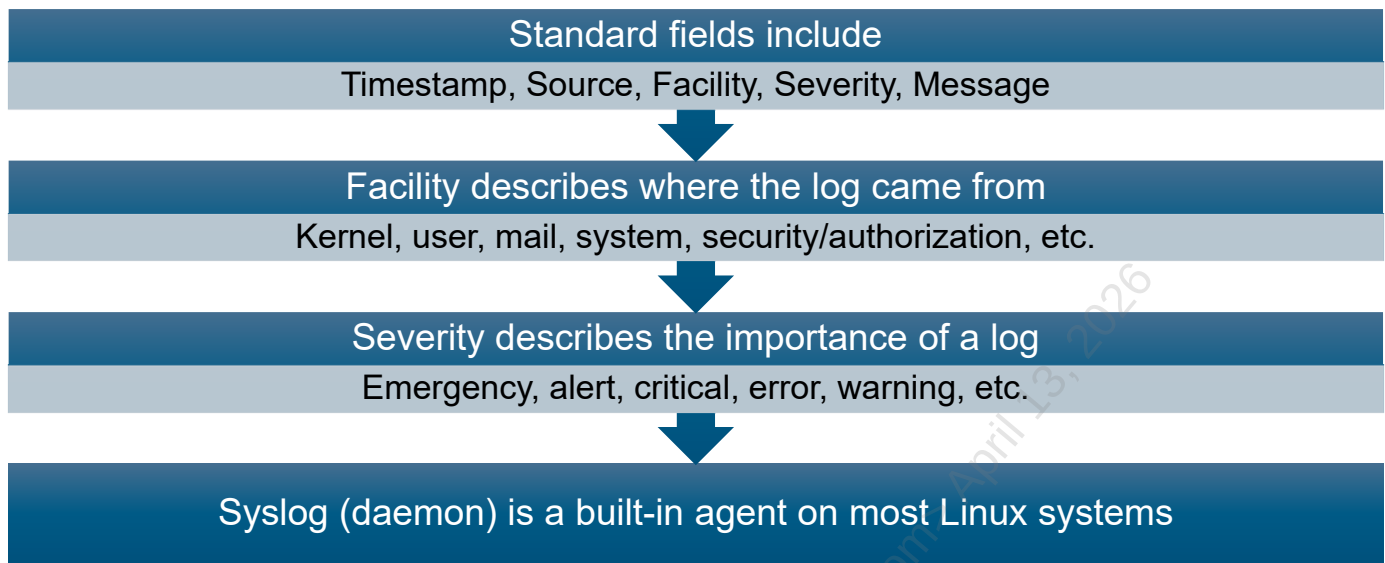
The most well-known and commonly used network protocol for transporting logs is probably syslog. Based on RFC 5424,¹ syslog supports UDP and TCP, as well as optional TLS encryption. However, syslog was adopted during the 1980s (created by Eric Allman for use in the Sendmail project, which was an email routing facility), when everyone felt security was not the primary objective, and was in use long before an RFC was created. In fact, multiple RFCs have been created for syslog, with the most notable first being RFC 3164 (published in 2001), which is on BSD Syslog.² That was followed by the most current of RFC 5424, which is based on the Syslog Protocol. Examples of others include RFC 3195 (reliable delivery for syslog),³ RFC 5425 (TLS transport mapping for syslog),⁴ and RFC 5848 (signed syslog messages).⁵

Due to early adoption by many applications and programmers, syslog messages tend to be inconsistent. Worse yet, initial builds of syslog daemons only supported UDP, and even today, built-in support is limited to UDP, without TLS.

Linux systems are gradually getting more support for TCP and TLS as newer operating system builds include **rsyslog** and **syslog-ng**. These are newer daemons with more built-in capabilities. For instance, **rsyslog** has a timestamp with millisecond accuracy, TCP and TLS support, support for RFCs 5424, 5425, 5426, and it allows buffering of messages if the receiver is not ready.

1. <https://sec555.com/0rbf9>
2. <https://sec555.com/1orcmm>
3. <https://sec555.com/sdmyi>
4. <https://sec555.com/kl5qw>
5. <https://sec555.com/gty93>

Syslog Format



The primary syslog fields are:

- Timestamp
- Source
- Facility
- Severity
- Message

Per RFC 5424,¹ "The timestamp specifies the date and time and must follow a specific time format that includes the date, followed by a capital T, followed by the time, and then either a capital Z for UTC time or the time zone offset, such as -04:00, which would be Eastern Standard Time. The capital T is the separator between the date and the time. Note that the T must be capitalized and if the Z is used to specify UTC, it must also be capitalized."

The source field specifies the system sending the syslog message.

Syslog messages can be routed to files, remote syslog servers, and directly to the console itself. In order to manage log rotation and retention, use `logrotate` as syslog doesn't perform that by itself.

1. <https://www.rfc-editor.org/rfc/rfc5424.html>

Syslog Facility

0	kernel messages	12	NTP subsystem
1	user-level messages	13	log audit
2	mail system	14	log alert
3	system daemons	15	clock daemon (note 2)
4	security/authorization messages	16	local use 0 (local0)
5	messages generated internally by syslogd	17	local use 1 (local1)
6	line printer subsystem	18	local use 2 (local2)
7	network news subsystem	19	local use 3 (local3)
8	UUCP subsystem	20	local use 4 (local4)
9	clock daemon	21	local use 5 (local5)
10	security/authorization messages	22	local use 6 (local6)
11	FTP daemon	23	local use 7 (local7)

The facility specifies the source application or program the message was generated from. Standard facility values are as follows (note that **local use** is for administrator and application purposes):

Numerical Code	Facility
0	kernel messages
1	user-level messages
2	mail system
3	system daemons
4	security/authorization messages
5	messages generated internally by syslogd
6	line printer subsystem
7	network news subsystem
8	UUCP subsystem
9	clock daemon (note 2)
10	security/authorization messages
11	FTP daemon
12	NTP subsystem
13	log audit
14	log alert
15	clock daemon
16	local use 0 (local0)
17	local use 1 (local1)
18	local use 2 (local2)
19	local use 3 (local3)
20	local use 4 (local4)
21	local use 5 (local5)
22	local use 6 (local6)
23	local use 7 (local7)

Syslog Severity

Code	Severity
0	Emergency: System is unusable
1	Alert: Action must be taken immediately
2	Critical: Critical conditions
3	Error: Error conditions
4	Warning: Warning conditions
5	Notice: Normal but significant condition
6	Informational: Informational messages
7	Debug: Debug-level messages

The severity describes the importance of the log, starting with emergency and decreasing in importance down to debug.

Numerical Code	Severity	Example
0	Emergency: System is unusable	Kernel panic, disk corruption
1	Alert: Action must be taken immediately	Root filesystem is full
2	Critical: Critical conditions	Failed disk, application crash
3	Error: Error conditions	Login failed, missing config file, network request failure
4	Warning: Warning conditions	High memory use
5	Notice: Normal but significant condition	service start/stop
6	Informational: Informational messages	DHCP lease, cron job info
7	Debug: Debug-level messages	function entry/exit

You may also see fields such as priority and version.

Priority is a calculation taking the facility number (0–23), multiplying by 8, and then adding the severity value (0–7).

Version number specifies the version of the syslog protocol being used. Other syslog fields also exist, but are less common.

When syslog is used on hosts such as Linux, a configuration file (e.g., `/etc/syslog.conf`), specifies the facility and the severity levels.

Syslog Message Limitations

Inconsistent message formats

Requires additional parsing

Majority of systems are limited in message size

- Syslog over UDP often limited to **1024** bytes (RFC 3164)
 - Fragmentation typically not used with UDP
 - Standard MTU is usually 1500 bytes
- Syslog over TCP often limited to **4096** bytes

Due to the early adoption and varying implementations of syslog, message consistency is almost non-existent. The message layout used by one vendor or application varies dramatically from the next. This requires per-application or device parsing, which adds a lot of overhead. Parsing is not necessarily difficult, but it can be time-consuming.

Another limitation of syslog is the maximum log size supported. In RFC 3164 (mentioned previously), which pertains to BSD syslog, the maximum packet size for syslog over UDP is 1024 bytes. Even though syslog is used on non-BSD systems, many follow suit and **will either drop or truncate a log packet over 1024 bytes**. The reasoning behind this size limit is that UDP does not keep track of packets and, therefore, the loss of a single packet when using fragmentation would garble a message. Due to this and the fact that the standard MTU size is 1500 bytes, 1024 bytes was selected. Note that MTU (maximum transmission unit) is the largest size of a packet (or frame) that can be dispatched in a single network transaction.

For TCP, the size is often limited to 4096 bytes.

While most logs will fit within the 1024- to 4096-byte range, if you use syslog to transport logs, you must know how large your events are going to be. You also need to know if both your syslog client and syslog server are held to these size limits. Many commercial SIEMs adhere to these size limitations. This is interesting, given that Windows logs can be over 30K in size, and yet many agents transmit Windows event logs over syslog.

Syslog Config Example (Ubuntu 22.04 System)

```
auth,authpriv.*      /var/log/auth.log
authpriv.=warning    @10.5.55.10
*.*;auth,authpriv.none /var/log/syslog
cron.warning         /var/log/cron.log
*.!info             /var/log/verbose.log
kern.*              /var/log/kern.log
#lpr.*              /var/log/lpr.log
mail.*              -/var/log/mail.log
```

This example is from the default Ubuntu 22.04 rsyslog configuration file (`/etc/rsyslog.d/50-default.conf`) which has been slightly modified. The traditional syslog format is compatible with the newer rsyslog format. This example follows the traditional syslog format for specifying what gets logged and where. The **left column specifies the facility and severity levels** to log, and the **right column specifies the destination**.

```
auth,authpriv.*      /var/log/auth.log
```

Takes any logs with a facility of auth or authpriv facilities (which relate to authentication events) and sends them to `/var/log/auth.log`. The asterisk referenced means **all severities**.

```
authpriv.=warning    @10.5.55.10
```

Takes any logs with a facility of authpriv and a severity of warning and sends them over UDP port 514 to a syslog server located at IP address 10.5.55.10. The equals sign specifies an exact match on severity, so only a severity of warning will be included, and nothing else.

```
*.*;auth,authpriv.none /var/log/syslog
```

Log all facilities and severities of messages (*.*) (except for auth and authpriv) to `/var/log/syslog`.

```
cron.warning         /var/log/cron.log
```

This takes any logs with the cron facility that have a severity of warning or higher and writes to `/var/log/cron.log`.

```
*.!info             /var/log/verbose.log
```

Takes any logs with any facility that have a severity of info or lower (which would include info and emergency severities) and writes them to `/var/log/verbose.log`.

kern.* /var/log/kern.log

Logs all kernel-related messages to /var/log/kern.log.

#lpr.* /var/log/lpr.log

Log all printer-related messages to /var/log/lpr.log (but it has a # character, which means it's currently commented out).

mail.* -/var/log/mail.log

This takes any mail logs and sends them to /var/log/mail.log.

Note that the dash before /var/log/mail.log means that syslog does not have to flush the log to disk after each log entry, improving efficiency. This is often used for log files that receive a lot of logs. Having to confirm the write to disk could cause unacceptable performance issues. However, should a system crash with this setting enabled, it is possible that logs may not have been committed to the log file on disk and thus, can be lost.

Syslog Example

`/var/log/auth.log (Ubuntu 16.04)`

```
Mar 28 14:43:13 sec555 sudo: nickmitro : 1 incorrect
password attempt ; TTY=pts/1 ; PWD=/var/log ; USER=root ;
COMMAND=/bin/su
```

When transported over network:

```
<81>Mar 28 14:43:13 sec555 sudo: nickmitro : 1 incorrect
password attempt ; TTY=pts/1 ; PWD=/var/log ; USER=root ;
COMMAND=/bin/su
```

`/var/log/auth.log (Ubuntu 22.04)`

```
Mar 28 14:45:51 slingshot su: pam_unix(su:auth):
authentication failure; logname= uid=1001 euid=0
tty=/dev/pts/0 ruser=sec555 rhost= user=nickmitro
```

The top of the page represents a failed login stored in `/var/log/auth.log` of an Ubuntu 16.04 system. Notice the difference is minor between syslog on disk and syslog over the network. On disk, the log facility/category is identified by the filename of `auth.log`. Over the network, this is identified using the syslog PRI field, which is the beginning `<81>`. The integer in this field comprises the facility and severity for the log. To find the facility and severity, a math formula must be applied. The math formula for the PRI field will be described shortly.

Notice that `/var/log/auth.log` does not have anything that describes the severity (debug, info, warning, etc.). This is because the default behavior of rsyslog on Ubuntu 16.04 is to place all severity types for the facility `auth` into `/var/log/auth.log`.

A similar entry from a Ubuntu 24.04 system is also present for comparison:

`/var/log/auth.log`

```
Feb 14 14:36:01 slingshot su: pam_unix(su:auth): authentication
failure; logname= uid=1001 euid=0 tty=/dev/pts/0 ruser=sec555 rhost=
user=nick
```

Syslog Breakdown

```
<81>Mar 28 14:43:13 sec555 sudo: nickmitro : 1 incorrect  
password attempt ; TTY=pts/1 ; PWD=/var/log ; USER=root ;  
COMMAND=/bin/su
```

PRI: <81>

Timestamp: Mar 28 14:43:13

Source host: sec555

Source program: sudo

Message: nickmitro : 1 incorrect password attempt ; TTY=pts/1
; PWD=/var/log ; USER=root ; COMMAND=/bin/su

Sometimes includes process ID, such as CRON[1464].

Syslog messages sent over the network typically consist of five main fields:

1. **PRI:** The PRI field starts at the beginning and is an integer that is calculated to store the facility and severity labels for an event.
2. **Timestamp:** The timestamp is specific to the system generating the event. **If the source system's time is off, then this timestamp will be incorrect.** It is also important to note that the timestamp is specific to the time zone of the source system. This will need to be accounted for if an organization contains systems across multiple time zones.
3. **Source host:** The source machine's hostname.
4. **Source program:** The source program that generated the event log.
5. **Message:** Contains the specifics of the log entry.

Unlike Windows, Linux does not generate a unique event ID specific to a related event. This often requires string searching or adding fields or tags to report on specific events, such as logons. To get maximum value, the message field often requires further breakdown and parsing. This is primarily because syslog does not use structured schemas such as XML. This, combined with the freedom available to programmers on how the message field is used per program, causes a lot of extra labor to make sense of Linux logs.

Syslog PRI

Syslog messages sent over the network store facility and severity in the computed PRI field

PRI is 1–5 digits surrounded by <>.

- Example: <189>

Facility = PRI/8 rounded down to whole number

- Example: $189/8 = 23.625$, so facility = 23 or local7

Severity = PRI – (Facility * 8)

- Example: $189 - (23 * 8) = 5$, so severity = 5 or notice

A new analyst looking at a raw syslog message often tends to ignore the syslog PRI field, thinking it is just noise in the log. This is because it looks out of place. For example:

```
<81>Mar 28 14:43:13 logparse sudo: nickmitro : 1 incorrect password attempt ; TTY=pts/1 ; PWD=/var/log ; USER=root ; COMMAND=/bin/su
```

The <81> looks like it may just be a random number tacked to the front of the log. However, it is the syslog PRI field. This field is an integer that can be mathematically reversed into the numeric facility and severity codes. The math formulas on this slide show how to reverse the integer into the proper codes.

In the example of <81>, the facility would be 10, which is the facility code for security/authorization.

Calculation: **Facility = PRI / 8** = $81/8 = 10.125$ (with the closest whole number being 10).

The severity would be 1, which is the severity code for the alert.

Calculation: **Severity = PRI – (Facility * 8)** = $81 - (10 * 8) = 81 - 80 = 1$

Syslog-ng

One common Linux syslog daemon is syslog-ng

- Capable of handling 600K+ messages a second
- Enhanced with many filtering and custom parsing capabilities
- Able to forward logs to Elasticsearch and Apache Kafka
- Supports transport over UDP, TCP, and SSL/TLS

Configuration file does not conform to syslog format

- The format is clean and easy to read, unlike the traditional syslog format

After syslog came syslog-ng. It was designed by the community to address deficiencies such as the lack of TCP, SSL, and TLS in standard syslog and was one of the first implementations around central log collection. The biggest change for those switching to syslog-ng from syslog (or rsyslog) is that it requires its own syntax, as it has stepped away from the old syslog format. Like many open-source projects, syslog-ng has adapted over time, and in its current release, it has many capabilities.

Why use it?

- Regular expression and composite filter support.
- CSV and JSON support, which makes it integrate seamlessly with modern tools.
- Disk buffering support, to prevent log loss during poor network conditions.
- Able to forward logs to Apache Kafka and Elasticsearch.
- Handles high volumes of logs due to multi-threading support.
- Customized scripting support (e.g., Lua or Python).
- Supports log field enrichment prior to forwarding.

References:

- <https://sec555.com/8cz25>
- <https://sec555.com/sqebm>

Rsyslog

Rsyslog stands for rocket-fast system for log processing

- Handles 1 million+ messages per second to local destinations
- Advanced parsing and filtering
- Great modular plug-in support
- Performance is faster, especially with filtering and parsing
- Complex to configure
- Modular plug-ins can make troubleshooting challenging to perform

Rsyslog is another syslog alternative found in modern Linux systems. **A core principle of rsyslog is that it is built for speed.** This means even extra functionality, such as parsing, is analyzed and modified to make it as fast as possible. An interesting aspect of rsyslog is that it uses the library liblognorm (as opposed to regex) to perform string searches to look for pattern matches. When this is utilized instead of regex, parsing is done at minimal overhead and can sustain high events per second.

Both rsyslog and syslog-ng are great examples of community-maintained projects. In the case of rsyslog, it still supports syslog-ng features like multi-threading and buffering, UDP, TCP, and TLS. In addition:

- Has advanced filtering and parsing (filtering on source or severity or content).
- Can use regular expressions or property-filters.
- Logs can be sent to databases, SIEM tools, cloud services, or remote servers.
- Allows for modular plug-in usage, which means it is very easy to add or remove functionality.
- Legacy syslog is fully supported, and it supports JSON logging.

A few items to consider are:

- It can become complex to configure.
- Advanced feature implementation demands really good familiarity.
- Multi-threading and disk buffering mean more resource usage (both CPU and memory).
- Some legacy applications won't be able to support its advanced features.
- Troubleshooting can be challenging due to its modular nature, which makes pinpointing an issue very tricky at times.

Reference:

- <https://sec555.com/sqebm>

Logrotate

Default log management tool

Performs

- Compression
- Deletion

Configuration file is in plaintext

What happens if the attacker changes the configuration file and alters settings? 😞

```
root@slingshot:/home/sec555# cat /etc/logrotate.conf
# see "man logrotate" for details

# global options do not affect preceding include directives

# rotate log files weekly
weekly

# use the adm group by default, since this is the owning group
# of /var/log/syslog.
su root adm

# keep 4 weeks worth of backlogs
rotate 4

# create new (empty) log files after rotating old ones
create

# use date as a suffix of the rotated file
#dateext

# uncomment this if you want your log files compressed
#compress

# packages drop log rotation information into this directory
include /etc/logrotate.d

# system-specific logs may also be configured here.
```

Log rotation is very important if you want to be able to perform sound investigations. The worst thing that can happen after experiencing a security incident is having no logs to analyze.

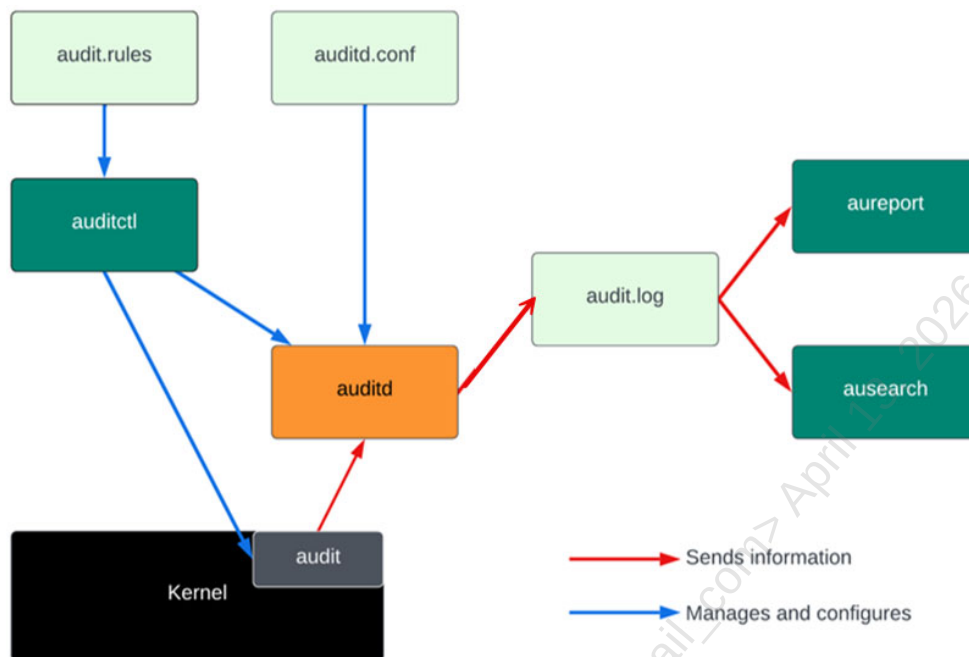
Logrotate can be used to configure how logs will be managed, and that means how often logs get compressed and archived or eventually get deleted. When you have too many logs on a system, you need it to perform this level of housekeeping to reduce the manual labor and potential mistakes that come with it. The default main configuration file (/etc/logrotate.conf) depicted on the page has been taken from a Ubuntu 22.04 system. The options are very easy to read and alter to your needs. # is used to comment lines out.

Also note that there are separate log rotation configuration files under /etc/logrotate.d, for individual applications and services, like apache2, bootlog, ufw, and more. For instance, this is the ufw firewall config:

```
/var/log/ufw.log
{
    rotate 4
    weekly
    missingok
    notifempty
    compress
    delaycompress
    sharedscripts
    postrotate
        -x /usr/lib/rsyslog/rsyslog-rotate ] && /usr/lib/rsyslog/rsyslog-
        rotate || true
    endscript
}
```

- **rotate 4:** Means files are rotated 4 times before any old files are removed. To phrase it differently, the last 4 rotated log files will be kept.
- **weekly:** Log rotate files once a week (you can change this to hourly, daily, monthly, or even yearly).
- **missingok:** Log rotation should keep on going, without reporting errors if a log file is unavailable.
- **notifyempty:** If a file is empty, then it won't be rotated.
- **compress:** Compress old logs (uses the gunzip format - .gz) .
- **delaycompress:** Don't compress files to be rotated until the next rotation begins. For example, `ufw.log` will become `ufw.log.1` and will stay uncompressed for one rotation, while `ufw.log.2` will now become `ufw.log.2.gz`.
- **sharedscripts:** Execute scripts in prerotate or postrotate only one time (the default behavior is to execute scripts separately for each log being rotated).
- **postrotate:** Script to be executed after log rotation (in this example, `/usr/lib/rsyslog/rsyslog-rotate` is checked to assess if it is executable (-x) and if so, it is executed so the file can be closed and reopened for writing later on. The `true` statement allows for preventing any errors if the file is not found, which means in that case, this post-rotate action does nothing.

Auditd



The Linux Audit system, also referred to as auditd, is a Linux utility used for auditing events, such as system calls, file and directory access, and network connection logging. It operates at the kernel level and collects data from the system and stores it in a binary format. It's very granular and you can configure it to your desire. Its basic components are:

- **audit.conf:** Perform configuration, using the audit.conf file (/etc/audit/audit.conf), which is used to set buffering, log rotation and retention, logging locations, and event filtering.
- **audit.rules:** A list of audit rules is located under /etc/audit/audit.rules (comprised of files under /etc/audit/rules.d) and feeds into auditctl, which is what runs the auditing system. This is where specific rules about auditing conditions are written and trigger alerts when related events take place.
- **auditctl:** Used to control the audit system as a whole. It establishes kernel parameters for logging and what events are logged.
- **auditd:** The audit daemon making log message generation possible.
- **audit.log:** Contains log data generated by the audit daemon.
- **ausearch:** Allows for enhanced searching of the audit.log file for detailed log events, based on user-defined characteristics.
- **aureport:** Generates custom reports from querying the audit.log file.

Reference:

- SANS FOR577: Linux Incident Response and Threat Hunting

Image reference:

- <https://sec555.com/5zg4w>

Auditd Rule Creation and Test

- Add audit rule to watch the shadow file

```
auditctl -w /etc/shadow -p rwx -k ShadowFileAccess
```

- Try accessing it for reading

```
cat /etc/shadow
```

- Use ausearch to look for events

```
ausearch -f /etc/shadow -i
```

```
type=PROCTITLE msg=audit(02/15/2025 10:44:47.748:101) : proctitle=cat /etc/shadow
type=PATH msg=audit(02/15/2025 10:44:47.748:101) : item=0 name=/etc/shadow inode=263762 dev=fd:00 mode=file,640 ouid=root ogid=shadow rdev=00:00 nametype=NORMAL c
fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(02/15/2025 10:44:47.748:101) : cwd=/home/sec555
type=SYSCALL msg=audit(02/15/2025 10:44:47.748:101) : arch=x86_64 syscall=openat success=no exit=EACCES(Permission denied) a0=AT_FDCWD a1=0x7ffdb48493cc a2=0_RDON
80 pid=185487 auid=unset uid=nick gid=nick euid=nick suid=nick fsuid=nick egid=nick sgid=nick fsgid=nick tty=pts0 ses=unset comm=cat exe=/usr/bin/cat subj=unconfi
```

There are various rules you can set up on the system using the auditctl command. There are numerous more you can consider.¹ In the above example, a rule was used to monitor for access to the shadow file:

```
auditctl -w /etc/shadow -p rwx -k ShadowFileAccess
```

- **auditctl** is called to add the audit rule.
- **-w** is used to watch a file.
- **/etc/shadow** specific the file's location on the system.
- **-p rwx** is used to specify to watch the file for anyone attempting to read (r), write (w), or execute (x) it.
- **-k** is used to assign a custom tag to the rule for easy searching afterward.
- **ShadowFileAccess** is just used as a name to distinguish this activity.

Once the rule is set up, you can try to access the file using nano, cat, more, less, or any other way. Once you do, a violation will be logged in /var/log/audit/audit.log, which you can find using:

```
Cat /var/log/audit/audit.log | grep Shadow
```

A better way, though, would be to use the ausearch utility to get a better view:

```
ausearch -f /etc/shadow -i
```

If you just want to get a statistical depiction of audit hits, at a high level, use aureport:

```
aureport -f -i --summary
```

1. <https://sec555.com/dnq76>

Sysmon for Linux

Open-source system monitor tool developed to collect security events from Linux environments

- Uses eBPF (Extended Berkeley Packet Filter)
- Transports logs via syslog
- Utilizes same Event IDs as Windows, allowing for easy integration and correlation within the SIEM
- Possible replacement for auditd

Sysmon for Linux was developed by Sysinternals and released in 2021, and had a major update in early 2025. A few benefits to consider are:

- Offers advanced process monitoring (e.g., command argument level).
- Supports file system monitoring visibility (creation/modification/deletion of events, critical file tampering).
- Allows monitoring of network connections.
- Provides structured logs that can be fed easily to a SIEM tool.
- It's quite efficient and performance-friendly.
- Can greatly aid threat hunters and security analysts in detecting and analyzing anomalous behavior.
- Great level of documentation available.

The articles below explain the installation and operational process.

References:

- <https://sec555.com/5m74x>
- <https://sec555.com/dn7sp>
- <https://sec555.com/2sd4f>
- <https://sec555.com/m19pd>

Summary

Understand how syslog works in depth and consider the benefits of using rsyslog and syslog-ng.

Consider the use of auditd and Sysmon for Linux to enhance your system visibility. Choose an appropriate log collection strategy to ensure it fits your environment.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines

- SMTP
- DNS
 - ☐ Lab 2.1: Investigating DNS Logs
- HTTP/HTTPS
 - ☐ Lab 2.2: Investigating HTTP Logs
- Windows Logs
 - ☐ Lab 2.3: Investigating Windows Logs
- Linux Logs
- Host Firewalls and Login Events
 - ☐ Lab 2.4: Using Auditd

This area is intentionally left blank.

Module Objectives



The student will benefit from understanding the format of both Windows and Linux host-based firewall logs and understand what login events of interest to monitor for.

This area is intentionally left blank.

Host-Based Firewall Logging

Windows

Default: **Windows Firewall**

Configuration over GUI and CLI

Application-based filtering

Logging options

- Security channel
- Log file

Cost: Free

Linux

Default: **iptables**

Only allows configuration via CLI

Packet-level filtering

Logging options

- Log file (location varies by OS)

Cost: Free

Monitoring host-based firewalls is a high-volume problem. A single machine can easily generate over 25,000 events an hour. Granted, this is only around 5–8 EPS, but add that up for each device, and this is a large volume to control. To make the problem worse, a standard system is likely generating over 100 blocked connection logs an hour.

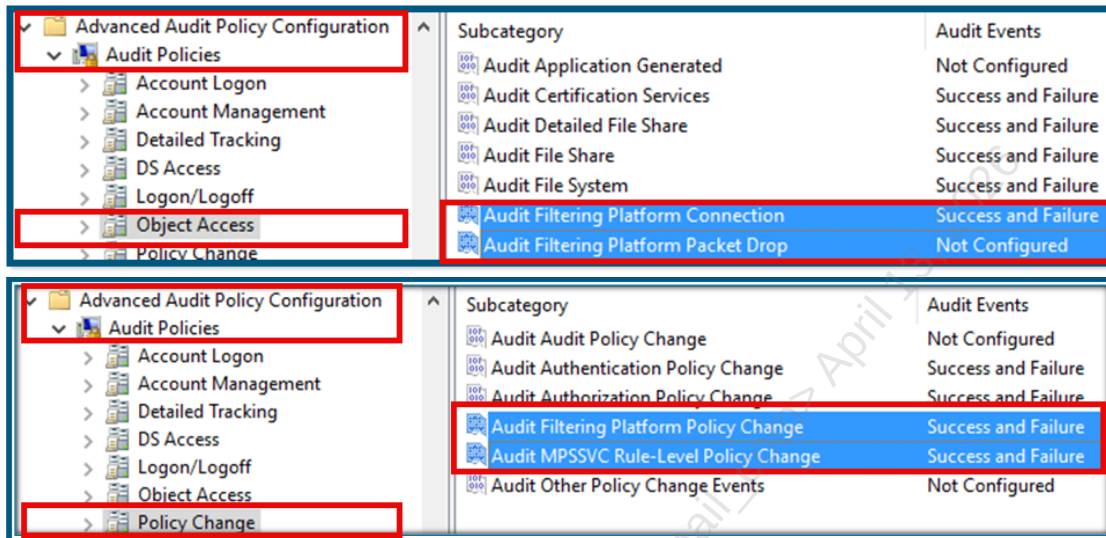
If a firewall blocks something, should it not be investigated? Yet, is a person capable of investigating 100 blocks per machine per hour? Not really, but we can do something to make this more approachable.

Two of the most common firewalls are the Windows Firewall and Linux iptables. This is primarily because they are built into their respective operating systems. Note that while this module focuses on these two native host-based firewalls, the concepts apply to pretty much any third-party host-based firewall as well. At a high level, they are both great options that are already built into the OS.

The Windows Firewall allows you to configure it using the GUI, while iptables only supports CLI configuration. The Windows Firewall is set to block any unwanted traffic out of the box, while iptables doesn't have any default rules and expects the administrator to set those up. Although the Windows Firewall can work on both endpoints and servers, it's mostly optimized for endpoints, while iptables has been optimized for both. The Windows Firewall has been created with a view to persist after system restarts (rules are stored in the Windows registry), while iptables rules are not persistent since they are stored in RAM. Ruleset needs to be saved to enable rule persistence.

Windows Firewall Audit Policies

- Firewall logging is spread across multiple policies
 - Audit policies enable logging to Security channel



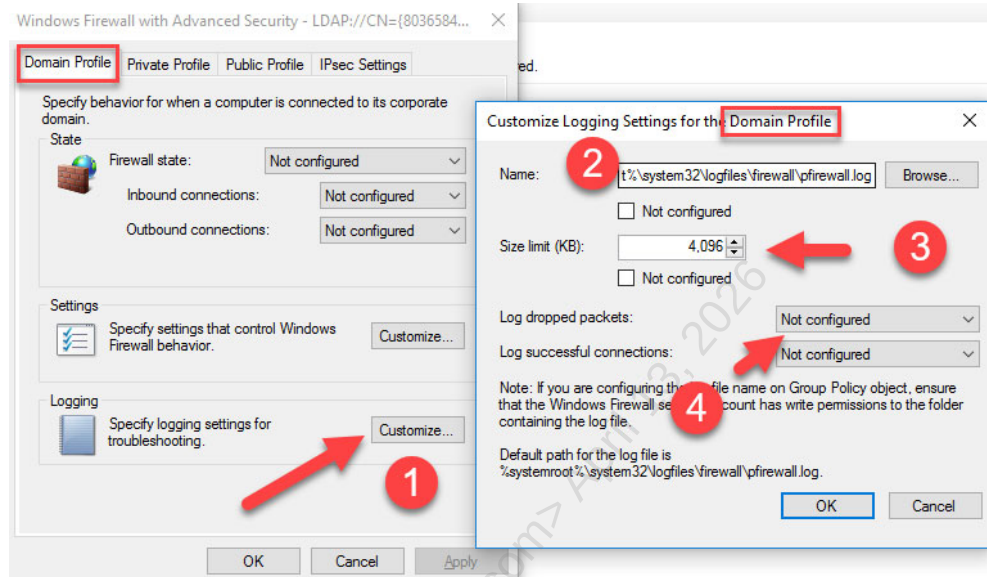
The first method to enable logging for Windows Firewall is to enable settings in the Object Access and Policy Change audit policies:

1. **Audit Filtering Platform Connection:** Logs based on connections or connection attempts, local port binds, and applications or services listening over a network port. You need to be very careful with this setting, as the volume of events will be very high. But you can granulate the specific connections or ports and only log, for example, successful actions. Example event IDs: 5156 – network connection was permitted (filtering allows it), 5157 – network connection was blocked.
2. **Audit Filtering Platform Packet Drop:** Logs at the packet level. This means the Packet Drop policy is much more likely to have repetitive entries. To minimize data gathered while not really losing value, it is recommended to use the **Audit Filtering Platform Connection** over **Audit Filtering Platform Packet Drop**. Example event IDs: 5152: Windows Filtering Platform blocked a packet.
3. **Audit Filtering Platform Policy Change:** You can enable the logging of changes made to the Windows Filtering Platform (WFP), which is an API collection that filters network traffic and enforces security rules. Examples: Changes to IPSEC policies, packet filtering rules, firewall rule modifications. This includes the firewall service being shut down. Example event IDs: 5153 – Network packet allowed, 5159: Security policy settings were changed.
4. **Audit MPSSVC Rule-Level Policy Change:** MSPVC is the Microsoft Protection Service, so this setting reports changes to specific firewall rules and offers enhanced configuration granularity. Example event IDs: 4946 – rule was added to the Windows Firewall exception list, 4949 – Windows Firewall settings restored to default.

The other policy entries about policy change are used to record events related to modifications to the Windows Firewall. This includes things such as the creation or modification of firewall rules, deletion of firewall rules, or even changing the state of the firewall (such as turning it off).

Windows Firewall Log File(s)

- A log file per network profile can be specified
 - Or can share a single file
- Provides more detail than event channel log



The second method of logging for Windows Firewall is to enable logging to a text-based log file. This file can be set to log dropped packets and/or successful connections. It also allows specifying a size limit in which it will keep rotating the log contents according to the max allowed size. It is typically recommended to increase this size as the default 4 MB is fairly small for a log file.

The setting is under Local Security Policy > Windows Defender Firewall with Advanced Security > Domain Profile tab > (go to the Logging section at the bottom) Customize > Untick "Not configured" and select the desired log file (the default path is **%systemroot%\system32\logfiles\firewall\pfirewall.log**). Note that this relates to the Domain profile. If you need to do the same for private and public profiles, you need to go to the other tabs and repeat the same.

If you want to log dropped packets and successful connections to get enhanced visibility and understand the root cause behind potentially persistent dropped packets and/or network connections, then switch "**Log dropped packets**" and/or "**Log successful connections**" to **Yes**.

Enabling the log file component of Windows Firewall can be done manually—per host—or it can be done with group policy.¹

1. <https://sec555.com/8zho9>

pfirewall.log Example

- Extra data
 - TCP Flags
 - Size
 - Ack #
 - Syn #

action	protocol	src-ip	dst-ip	src-port	dst-port	size	tcpflags	tcpsyn	tcpack	tcpwin	icmptype	icmpcode	info	path
DROP	TCP	10.0.0.108	10.0.0.10	37782	49153	44	S	2255999555	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	37783	49153	44	S	2255934018	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	37782	49152	44	S	2255999555	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	37783	49152	44	S	2255934018	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	61627	49152	44	S	109291553	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	61628	49152	44	S	109226016	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	61627	49153	44	S	109291553	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.108	10.0.0.10	61628	49153	44	S	109226016	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46353	49152	44	S	1376660998	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46354	49152	44	S	1376726535	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46355	49152	44	S	1376529924	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46353	49153	44	S	1376660998	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46354	49153	44	S	1376726535	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	46355	49153	44	S	1376529924	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33677	49152	44	S	601886679	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33678	49152	44	S	601952214	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33679	49152	44	S	602017749	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33677	49153	44	S	601886679	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33678	49153	44	S	601952214	0	1024	-	-	RECEIVE	
DROP	TCP	10.0.0.106	10.0.0.10	33679	49153	44	S	602017749	0	1024	-	-	RECEIVE	
DROP	UDP	10.0.0.108	239.255.255.250	57046	1900	165	-	-	-	-	-	-	RECEIVE	
DROP	UDP	10.0.0.108	239.255.255.250	57046	1900	165	-	-	-	-	-	-	RECEIVE	
DROP	UDP	10.0.0.108	239.255.255.250	57046	1900	165	-	-	-	-	-	-	RECEIVE	
DROP	UDP	10.0.0.108	239.255.255.250	57046	1900	165	-	-	-	-	-	-	RECEIVE	
DROP	UDP	10.0.0.108	239.255.255.250	57046	1900	165	-	-	-	-	-	-	RECEIVE	

This page demonstrates a sample log file when the firewall has been configured to log to a file.

In it, the contents show multiple dropped packets along with some extra details not found in the first method of logging.

This method includes additional information, such as the TCP flags involved. In the above picture, all the TCP connections were dropped on the initial SYN packet.

Note that reading the contents of this file requires administrator privileges. For example, opening the file with Notepad will generate an error. However, running Notepad as administrator and then opening the file works.

The power of this technique is providing detailed connection information and the ability to log per network profile, since you can configure a different log file per profile to exist (one for domain, one for public, and one for private). The downsides are that it only logs connection information and not event IDs or other details that would go to the Security channel, in addition to you having to make arrangements to collect the related log files on your tools (e.g., SIEM).

On the other hand, logging in the Security channel means including any firewall changes, application, and port bindings, logging if the firewall is disabled, seeing event IDs, and many other auditable items. The downside is that you get much less connection-related detail.

But you can always mix and match by choosing both of the approaches, which will give you the best value, provided that's what fits your needs.

iptables

Linux systems almost universally use iptables

- Some include iptables management programs like UFW

The feature set includes:

- Stateful firewall capabilities
- Inbound and outbound control
- Support for multiple types of address translation

No central reporting is available

- Logging is also disabled by default

The most common Linux host-based firewall is iptables. It is built into just about every Linux operating system today. While some seem to be using a different firewall, they often are iptables. For example, Ubuntu users use UFW (Uncomplicated Firewall), but UFW is actually a management program that sets up and controls iptables.

iptables is stateful inspection supportive and can control inbound and outbound connections. Where it is a bit stronger is its ability to get granular on firewall rules, such as what TCP flags are involved, as well as its support for many forms of network address translation. Like the Windows Firewall, logging is disabled by default.

iptables is configured using a set of rules. They are added, deleted, and modified using the CLI. Not only can it accept or drop traffic, but it can also modify or redirect packets elsewhere. In order to be able to perform traffic logging, it uses a separate module called ULOG, which is what sends messages to the syslog facility, and they end up getting written to disk.

Reference:

- SANS FOR577: Linux Incident Response and Threat Hunting

iptables Rules

```
# Inbound Rules: Allow SSH (22/tcp) and traffic we initiated
iptables -A INPUT -i eth0 -p tcp --dport 22 -m state \
--state NEW,ESTABLISHED -j ACCEPT
iptables -A INPUT -p tcp -m state --state ESTABLISHED -j ACCEPT
iptables -A INPUT -p udp -m state --state ESTABLISHED -j ACCEPT
iptables -A INPUT -p icmp -m state --state ESTABLISHED -j ACCEPT
#Block a specific IP
iptables -A INPUT -s $1.1.1.1 -j DROP
# Outbound Rules: Any sourcing TCP, UDP
iptables -A OUTPUT -p tcp -m state --state NEW,ESTABLISHED -j ACCEPT
iptables -A OUTPUT -p udp -m state --state NEW,ESTABLISHED -j ACCEPT
```

Rules for iptables are designed around the concept of chaining. This concept allows for a packet or connection to hit a specific rule and then be sent over to a different chain (series of related rules). There are two types of chains:

- **Built-in:** Include keywords such as INPUT, OUTPUT, FORWARD.
- **User-defined (also known as custom):** They're used to create custom processing rules for packets.

Rules in a given chain are processed in order, and once a match is found, a pre-defined action is taken. Some common arguments are:

- **-A:** Append rule to chain (like input or output chains)
- **-D:** Delete rule
- **-j:** Action to perform if the packet matches the rule
- **-i:** Applicable interface
- **-m:** Uses the state module to allow connection state tracking
- **-p:** Protocol
- **-s:** Source IP address
- **-d:** Destination IP address
- **--sport:** Source port
- **--dport:** Destination port

References:

- <https://sec555.com/f51m2>
- <https://sec555.com/gqzof>
- SANS SEC401: Security Essentials - Network, Endpoint, and Cloud

iptables Rules with Logging

```
#Log dropped traffic
iptables -N LOGGING
iptables -A INPUT -j LOGGING
iptables -A LOGGING -m limit --limit 2/min -j LOG --log-prefix "BLOCK" \
--log-level 4
iptables -A LOGGING -j DROP
```

- This sends anything not matching a rule to the LOGGING chain.
 - **-m limit --limit 2/min** logs two packets per minute.
 - **-j LOG** can be added to any rule and is what enables logging.
 - **--log-prefix** is an optional string to add to logs.
 - **--log-level** specifies the syslog severity to set.

To add default deny logging to iptables, you can run the commands given on this page (feel free to adjust to your needs).

They first create a new chain called LOGGING.

Then, a rule is appended to the bottom of the INPUT chain so that:

If no prior INPUT rules match, traffic gets sent over to the LOGGING chain. This chain then logs and blocks all traffic.

Be careful to ensure that your configuration meets what you are trying to achieve. For example, maybe two packets per minute is a low limit for you, and you want to increase it. Maybe you need to drop all traffic and only log HTTP drops at debug level (7). In that case, you can adjust this rule, as per below:

```
iptables -N LOGGING
iptables -A INPUT -p tcp --dport 80 -j LOGGING
iptables -A LOGGING -m limit --limit 2/min -j LOG --log-prefix
"HTTP_DROP " --log-level 7
iptables -A LOGGING -j DROP
```

Note that the log location differs in each distro (e.g., Ubuntu stores these under /var/log/syslog).

References:

- <https://sec555.com/z39ns>
- SANS FOR577: Linux Incident Response and Threat Hunting

Blocked Connections

Blocked Connections

- But a default network will be full of blocked connections

This is primarily due to:

- Vulnerability scanners or network monitoring systems
- Windows default settings cause lots of noise
- Broadcast and multicast traffic

Options

- Prevent traffic or filter logs

Establishing reporting capabilities requires log collection. However, the immediate problem is that the volume of logs is high. If additional purchases for expanding the log aggregator or backend storage are not allowed, then this problem is rather challenging.

Either traffic needs to be prevented from occurring, or logs need filtering. Now the question is, which is it?

Take any preventive device and monitor when it blocks something, and you now have a detective device. Granted, some things should be prevented and the logs ignored, but many times, if you are blocking something, you want to know about it. Adding this level of monitoring is great for catching things such as internal pivoting attempts.

Post-compromise, an adversary typically has to perform internal reconnaissance and pivot internally. Monitoring something as simple as host-based firewalls provides an additional means to catch this.

Although it may seem counterintuitive to log dropped traffic, the logic is to do so to uncover issues. This will allow you to find misconfigurations, uncover rogue insiders (e.g., the admin who thought it would be great to run an aggressive Nmap scan on the network over the weekend), and, most importantly, external attacks or reconnaissance attempts that may be of interest.

In an incident the author responded to a few years ago, the attacker managed to breach the network and steal the company backups (unencrypted, sadly), but that was only possible because for weeks on, they were scanning the environment, and no one paid any attention to that traffic. Do you know why? Because it was dropped at the firewall and no one thought it was of interest. But the attacker managed to find a vulnerability in a Confluence server, allowing them to exfiltrate valuable data.

The key to this technique is to filter out that traffic from your logs, thus keeping only the interesting dropped traffic. If it's not misconfiguration or expected traffic, then there may be something there worthy of attention. This will take time at the beginning to configure, but it can be smooth sailing after that.

Listening Processes

Blocked connections are now monitored

- Yet host-based firewalls have other value-add capabilities, such as identifying processes listening for connections

Useful for tuning firewall rules

- Also identifies malicious programs

A process listening for incoming connections is a service (regardless of OS specifics)

The primary reason for monitoring host-based firewalls is to discover blocked connections. Yet host-based firewall logs have other uses. Why waste?

If you think back to the Windows Firewall audit options, there are logs created any time an application attempts to listen for network connections. This means it is possible to discover which applications exist in an environment that listens for network connections. This provides an opportunity for another allow list check. It would be nice if you could be notified anytime a new process listens for network connections.

Listening Applications

The Windows Filtering Platform has permitted an application or service to listen on a port for incoming connections.

Application Information:

Process ID: 8244

Application Name: \device\harddiskvolume3\windows\system32\windowpowershell\v1.0\powershell.exe

Are you expecting PowerShell listening for network connections?

Allowed application

Windows Firewall blocked an application from accepting incoming connections on the network.

Profiles: Public

Application: C:\windows\system32\windowpowershell\v1.0\powershell.exe

Hopefully not, so you decide to block it from now on.

Blocked application

Host-based firewalls are positioned to see more than just network traffic. Because they are processes within their respective operating systems, they can also see which process is accepting or sending network traffic. This can be used to monitor processes that accept network connections.

This page demonstrates this by showing two events:

- The first log shows PowerShell being permitted to listen for network connections. The question is, in a normal environment, would you anticipate powershell.exe as listening for network connections? Most likely not.
- The second log rightfully shows PowerShell as being blocked from accepting incoming connections.

Modern-day attacks heavily use PowerShell. Being able to see it listen for network connections is basic, easy, and just may well save an organization.

This technique works best as an allow list technique but should also be doubled as a deny list. For example, powershell.exe would probably make the deny list. When you are reviewing traffic, you should start considering creating two buckets. One batch of events you find will reflect applications and events going in the allow bucket while the second batch will go to the deny bucket. When in doubt, it's recommended to pursue further investigation of that application or activity to identify if it should be blocked.

Separate Use Cases

Not all logs need to be collected

- Key host-based firewall logs are great to centralize.
- Others should be enabled but stay local.

Firewall event logs can be sent to SIEM

Local log files can record everything "in case it is needed"

The performance impact is minimal

Log files exist and can be later replayed

Allows staying tactical while being prepared for response

In the example of host-based firewall logs, the goal is to continue providing tactical data to the SIEM while providing logs for response activities. For instance, events from the security channel can be set only to log certain things, such as blocked connections. These can then be forwarded to the SIEM. Then, the log file option (e.g., of the Windows Firewall) can be enabled and set to log everything. These log files do not get sent to the SIEM and are enabled for forensics or incident-handling purposes.

The side benefit of enabling logs for forensics/incident handling is that they can often be replayed down the road. While it is not cost-effective to send every log to a SIEM, it would be nice to be able to do so ad hoc. If more verbose logs existed that could be manually picked up and sent to a SIEM for response procedures, the response activities would greatly benefit from the searching capabilities of the SIEM. Do note, though, that if any logs are kept on a machine and the attacker manages to get access to that machine, they can clear or potentially manipulate the logs. So, you should choose wisely as to which logs you will end up enabling, which of those will end up staying local only (if any), and which will end up making their journey to the SIEM tool. Parsing out things you don't need always makes the volume of data less, which decreases the cost of commercial solutions.

A best-case scenario would be scripts that would automatically collect certain logs and feed them into a SIEM. SOF-ELK is a distribution that allows you to easily collect and feed logs into an open-source SIEM for forensics purposes.¹

1. <https://sec555.com/hsl0n>

Login Events (1)



This area is intentionally left blank.

Login Events (2)

Login events are critical and should be monitored

- Credentials are intended to allow authorized access
- And to provide accountability

Yet credentials are often used and abused

Myth: Preferred method for attacks is exploitation

Truth: Compromised credentials are commonly used in attacks

Attacks do not always come from exploitation or things like client-side phishing attacks. Many targeted attacks heavily involve the use of valid credentials. Mainstream attacks typically involve compromising a single system and then pivoting throughout the environment using legitimate but stolen credentials. It is also possible for an attacker to brute force credentials by using techniques such as password spraying of internet-facing servers (such as Outlook Web Access). Once valid credentials are discovered, they can be used against internet-facing VPNs that do not require multifactor authentication.

The point is that credentials are a key attack vector. Verizon reported that almost 50% of EMEA breaches relate to credential compromises. In addition, it seems that 68% of global breaches relate to non-malicious actions by humans, which boils down to human error or social engineering attacks (which commonly focus on harvesting credentials).¹ Verizon's 2023 data breach report showed 86% of breaches used stolen credentials.² The IBM-X-Force Threat Intelligence report states a 71% increase in attack volume using valid credentials.³ These numbers are massive!

1. <https://sec555.com/sxbpt>
2. <https://sec555.com/2xylq>
3. <https://sec555.com/341ch>

Windows Logon Types

Each logon type can be used to identify the method of logon

- Type 2 Interactive: Local console/keyboard
- Type 3 Network: File sharing
- Type 5 Service: Service startup
- Type 10 RemoteInteractive: Remote Desktop

This can be used to identify events of interest

- Such as successful RDP access to a desktop where RDP is not normally used

There are many Windows logon types. They are described in detail below:

Logon Type	Description
2	Interactive: This is what most people think of and do. It is when someone logs on to a system locally, most commonly from a keyboard.
3	Network: This type of logon is when credentials are used to access a remote computer such as browsing files on a file share.
4	Batch: This is used for when a scheduled task using credentials is launched.
5	Service: This is for when a service that is using credentials is started.
7	Unlock: When a workstation or server is locked and then unlocked, this logon type is recorded.
8	NetworkCleartext: This logon type is recorded when a logon uses credentials that are sent in cleartext. This is most often in conjunction with IIS using basic authentication.
10	RemoteInteractive: This logon type is recorded when a logon is performed using remote desktop or Terminal Services.
11	CachedInteractive: This logon type is recorded when a system that is not able to reach a domain controller successfully logs on using cached credentials.

The fact that Windows breaks out the type of logon is extremely beneficial for monitoring. Multiple sites online break out the various logon types and describe conditions under which they are recorded. One such description is found on Ultimate Windows Security.¹

1. <https://sec555.com/t25pr>

Special Logons

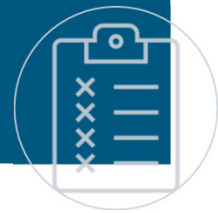
- **Computer accounts:** Active Directory machines are constantly authenticating to domain controllers
- **Service accounts:** "Special" accounts used to run services or scripts (some built-in, some user-created)

These two account types comprise the majority of logons.



- Computer accounts distinguished by a \$ in the user field ending
- Service accounts need to be tracked

Both create large amounts of successful logons and failures.



Within Windows, there are logons for more than user accounts. Because Windows uses centralized logging and allows for things such as mutual authentication, computer accounts are constantly performing authentication. This occurs naturally between domain members and domain controllers.

Windows and Linux both employ service accounts.

These are accounts used for specific tasks of programs. For example, many network services, such as Bind DNS, will often run under a service user account called **bind** on Linux. If a user account is created to perform a specific task, such as for credentialed vulnerability scans, it is also considered a service account. Basically, any purpose-built account used to perform a task gets classified as a service account.

Because of their use, service accounts are treated with special respect. For example, you are likely to fail an audit if you do not enforce password rotation every 90 days for user accounts. Yet, an auditor will typically ignore password rotation for a service account. Unfortunately, service accounts can be low-hanging fruit for attackers and penetration testers. To give you a taste of how challenging this can be, a Ubuntu 22.04 machine has about 40 system accounts with an out-of-the-box deployment.

High-Volume Problem

Logon success/failure is a high-volume problem

- Average # of logons/logoffs per system can exceed 100 per hour (need to establish a clipping level to monitor)
- Lots of logon events are of little to no value

Process of filtering is similar to other high-volume logs

- Find the noise, eliminate or filter it
- Then deploy tactically awesome detections

First, you need to understand what logs you are dealing with

Just like host-based firewalls (that were discussed earlier), logon events can be another high-volume problem.

Take Windows event ID 4624, which relates to successful account logons.

It can average over 100 logon events per hour, per system. Most of these logon events are of little to no value because they just tell you a user had logged on to the environment successfully. But isn't that what you expect to happen under normal circumstances?

As previously mentioned, these can be cleaned up by finding the noise and either eliminating it or filtering events appropriately.

Linux Login Event

A login event from Linux is simple

- Logged per local system (typically)
- Includes username and source (local vs. remote login)
- Logs success or failure

Easy to understand and use

- Gets more complicated with centralized logins

Linux login events are simple in nature. They include the username, success or failure, and where the login is coming from, such as from a local terminal (tty) or a remote program like SSH. Authentication in Linux is controlled by Linux Pluggable Authentication Modules (PAM). This is typically logged to /var/log/auth.log or /var/log/secure.

Because Linux systems tend to use decentralized logons and the methods for authenticating are simplified in comparison to Windows, their events are more straightforward and easier to understand.

Too Many Logins

Too much of a good thing is a bad thing

- Users with excessive successful logins to systems are suspicious.
- Threshold needs determining.
- Exceptions will need to be made for some accounts.
 - Delegation finds interactive sessions.
 - Impersonate finds things like scripts crawling the network.

Can find repetitive logons such as from pass-the-token

When thinking of login monitoring, most people immediately think about brute force attacks, which are attempts to constantly log in to accounts until successfully guessing the right password. While lots of failed logins are, of course, bad, so are too many successful logins.

Users are creatures of habit. They tend to log in at certain times of the day and only to certain machines. While some users, such as IT, regularly log in to multiple systems, most individuals will need to log in to only a small number of systems. If suddenly an account is being used to log in to excessive amounts of systems, it is a good sign that something bad is happening. Here are a few examples to consider:

- If, on average, you have 1,000 successful logins a day, why do you now start seeing 1,500?
- Are you expecting legitimate users connecting from X country/location?
- Who would connect using a TOR exit node?
- If they never connected at 03:00 a.m. for the last 5 years, would they start to do so now for a whole month (assuming they are based in the same location)?
- They were only using Edge for years, and now they have started using Iceweasel.
- Would a backup service account log on to the main domain controller interactively?
- Someone logged in to a system, and they were terminated a week ago.

The list can keep on going, but the key takeaway is one: **Just because they logged in, it doesn't mean it's legitimate.**

Too Many Failures

Counterpoint is too many login failures

- But what is "too many"?
- Standard user account could be 10+
- Service account should be 0 (should be no failures)

The rules change depending on the situation

- 1 failure per account x 50 = password spraying
 - Look for threshold per source
- 10 failures to a user from internet = it depends

Of course, too many login attempts is bad. If one IP address is generating thousands of login attempts, it's safe to assume you are being brute forced.

The key thing is to understand what is considered too many logins and how to monitor.

For example, if someone were trying to log in to 1,000 user accounts using a password of "password", it would likely bypass most alert systems. This type of attack is a form of brute forcing called password spraying, but since each account only gets one attempt, it is often ignored, even though it is not stealthy in any way, shape, or form. Again, this is an example of brute forcing.

Think about it. What would a normal brute force attack look like? The attacker targets the root account with 1,000 passwords. That means that you have tens of passwords hitting that account every minute, which would create an alert in most cases. But how about the attacker turns the tables? They can use the word password against 1,000 accounts. The logic is that by the time they run through all of these, enough time will have passed to go back to the beginning of the accounts list and try again, because you would only get sporadic attempts of incorrect passwords per account per minute. That may very well go undetected.

What do you do?

Instead of only monitoring failed logins by users, include monitoring failed logins by source address.

Other things should be monitored as well.

For example, an end user may accidentally enter their password incorrectly five times. In fact, it could even be more than that. However, a service account should never fail to log in. If even one failed login occurs, then either something is misconfigured or something bad is happening. Knowing the situations and context around your environment is what makes a difference. Technically, even this can be evaded. Assume you are an attacker with thousands of compromised systems under your control. Realistically, you could point all of them at a login page and tell them each to try a single user and password.

© SANS Institute 2025

If the first password does not work, all these systems could then issue a second password and continue in this manner. This would show up as one login per user per device per attempt. Worse yet, this is likely to result in a successful login within the first couple of password guesses. It just depends.

So, how do you catch a massively distributed password spray attack?

Monitor failed logins in total. Statistics work on your side as well as the enemies'. Create dashboards showing total items, such as:

- Total of failed logins
- Total failed logins from internal sources
- Total failed logins from external sources

This provides some additional context, which allows you to understand the possibility that an account is being misused.

References:

- <https://sec555.com/ajf6v>
- <https://sec555.com/58a7n>

Summary

Host-based firewall logs can prove tremendously useful. Remember to always look at blocked connections and identify the root cause for blocked traffic. Weed out misconfigurations and expected behavior to identify signs of potential attacks.

It is vital to monitor both successful and unsuccessful logins to your environment to have a clear depiction of any malicious aspect. Remember to know your account types and identify legitimate accounts vs. potentially misused ones.

This area is intentionally left blank.

Using Auditd

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Contacts and Resources

AUTHOR

Nick Mitropoulos

nick.mitropoulos@scarlet-dragonfly.com

@NickMitropoulos

www.linkedin.com/in/nickmitropoulos

Nick is the CEO of Scarlet Dragonfly, a company that handles a range of security services like SOC, incident handling, and vulnerability management, to name a few. He has over 16 years of experience in various industries, including consulting, education, telecommunications, healthcare, investment banking, financial, energy, oil and gas, insurance, pharmaceutical, law, military, and news. His breadth of experience, combined with his love for teaching, is crucial in helping his students better grasp the various facets of security.

Nick started collecting hacking newspaper articles back in the 80s when security was just starting to grow, and he was immediately hooked. He is a certified ISC2 instructor and a GIAC advisory board member, and he has an MSc (with distinction) in Advanced Security and Digital Forensics from Edinburgh Napier University and a BSc (with distinction and the highest held graduation grade of the department) from the Technological Educational Institute of Thessaly. He is also a faculty member of the SANS Technology Institute, an NSA Center of Academic Excellence in Cyber Defense, and a multiple winner of the National Cyber League competition. Nick has a great passion for teaching and has trained many individuals in a variety of security topics, mainly focusing on SIEM, security operation centers, incident response, and security certification courses.

Nick has worked for a variety of companies (including the Greek Ministry of Education, AT&T, F5 Networks, JP Morgan Chase, KPMG, and Deloitte) and has provided critical advice to many clients that want to improve their current security posture by creating new security operation centers and protocols, developing vulnerability management programs, managing third parties and security vendors, testing incident handling and forensic teams, redesigning perimeter security solutions, and developing custom-tailored training courses for company staff and specialized security teams. One of the greatest challenges for most security professionals is ensuring they have an in-depth understanding of all fundamental concepts of security. Nick loves teaching classes that help students obtain the knowledge required to secure their environments and enhance their personal security posture to protect themselves in an ever-growing digital arena.